Research on Hardware Primitives for Root-of-Trust in Secure Systems



RONALDO ENRIQUE SERRANO PEÑA

Department of Computer and Network Engineering

The University of Electro-Communications

A dissertation submitted for the degree of
Doctor of Engineering

19th January 2023

This page intentionally left blank.

Research on Hardware Primitives for Root-of-Trust in Secure Systems

APPROVED

Prof. Cong-Kha PHAM, Chairman

Prof. Koichiro ISHIBASHI

Prof. Yasutada OHAMA

Prof. Akashi SATOH

Prof. Ryo ISHIKAWA

Associate Prof. Bagus SANTOSO

Assistant Prof. Trong-Thuc HOANG

Data approved by Chairman: _____

This page intentionally left blank.

Copyright © 2022 Ronaldo Enrique Serrano Peña
All Rights Reserved.

This page intentionally left blank.

I would like to dedicate this dissertation to my mother, who encouraged me to pursue higher education.

"La creación intelectual es el más misterioso y solitario de los oficios humanos."

— Gabriel García Márquez

This page intentionally left blank.

Acknowledgements

First of all, I would like to express my deep respect and appreciation to my advisor Prof. Cong-Kha PHAM for his mentorship throughout the research process. I would also like to express my sincere appreciation to the committee members, including Prof. Koichiro ISHIBASHI, Prof. Yasutada OHAMA, Prof. Akashi SATOH, Prof. Ryo ISHIKAWA, Assoc. Prof. Bagus SANTOSO, and Asst. Prof. Trong-Thuc HOANG for their critical questions and valuable comments on this dissertation.

Second, the completion of this dissertation is due largely to the support of my friends in the PHAM laboratory. I would like to thank Ckristian DURAN and Marco SARMIENTO for their sincere assistance. I also thank Ba-Anh DAO, Anh-Tien LE, Binh Kieu-Do-NGUYEN, Khai-Duy NGUYEN, and DANG Tuan Kiet for their countless hours of brainstorming and helping.

Third, I would like to thank Dr. Kuniyasu SUZAKI and Mr. Akira TSUKAMOTO, researchers at the National Institute of Advanced Industrial Science and Technology (AIST), for their guidance and support.

Fourth, the research was supported by the New Energy and Industrial Technology Development Organization (NEDO). It was also supported by the VLSI Design and Education Center (VDEC), the University of Tokyo, collaborating with Synopsys, Inc., Cadence Design Systems, Inc., and Mentor Graphics, Inc.

Fifth, I sincerely thank the University of Electro-Communications for providing me with an excellent opportunity to experience my study and life in Japan.

Last but not least, I am grateful to my entire family, especially my mother, Gloria Peña Vargas, my father, Manuel Enrique Serrano Angarita, and my uncle, Luis Enrique Peña Vargas, for offering their love and sharing their wisdom with me throughout my life. In addition, I am grateful to Anjanette Asistin Sengoku and her family, for their support during my stay in Japan.

This page intentionally left blank.

和文要旨

和訳 セキュアシステムにおけるルートオブトラストのための ハードウェアの基本的要素に関する研究

セラノ　ペニャ　ロナルド　エンリケ

過去数十年間計算機などを含む電子システムは速度の向上と消費電力の削減に重点を置いてきた。しかし、近年、システムにおける重要な情報管理により、セキュリティが重要視されている。したがって、重要な情報に対する攻撃を緩和または軽減するためのプロセスがシステムに実装されることが不可欠である。これらのプロセスは、いくつかの安全なハードウェアの基本的要素（以降、ハードウェアプリミティブと呼ぶ）を使用して、起動プロセス、認証、および暗号化に焦点を当てている。ハードウェアプリミティブは、暗号化アルゴリズムとさまざまな物理現象を使用して構成され、乱数生成、鍵生成、および安全な格納のためにに使用される。セキュアハードウェアプリミティブは、基本的なルートオブトラスト(RoT) を提供し、システムの主要な実行ステージで重要な情報がその都度保護される。暗号化アルゴリズムは、システム間で情報を送信するための安全な方法を提供する。現在、コンピュータネットワークのエンドツーエンドの安全なチャネの確保に最も使用されている暗号化プロトコルは、Transport Layer Security (TLS) である。TLS は通常、Advanced Encryption Standard (AES) に基づく暗号を使用して情報を交換する。さらに、サイドチャネル攻撃とアルゴリズムの複雑さによって引き起こされるさまざまな脆弱性のため、ハードウェアアクセラレーションがこれらの問題を軽減する唯一の方法となっている。TLS 1.3 は ChaCha20 に基づく暗号を導入して、AES 暗号とは異なる別のソリューションを提供し、ChaCha20-Poly1305 Authenticated Encryption with Associated Data (AEAD) のスキームが可能となってきた。

非決定論的な物理現象によって生成された静的および動的エントロピを利用して、真正乱数生成器 (True Random Number Generator: TRNG) と物理的複製不能関数 (Physically Unclonable Function: PUF) を構築することが可能である。これらの物理現象は、従来の回路の望ましくない影響によって緩和され、低エントロピ抽出と信頼性の再現実装問題によって引き起こされるセキュリティの問題が発生する。このため、システムのセキュリティに重大なダメージを防ぐために、低エントロピ抽出を緩和し、物理現象の安定性を保証するようキャリブレーションプロセスやヘルステストなどの対策が実装されている。

また、RoT の重要な部分を代表し、起動プロセスで使用されるキーはシステムの重要な情報の格納に関しては、これらのキーが破損しないことを保証する必要がある。不揮発性メモリNVRAMは、鍵導出アプリケーションにおいて、 RoT で使用される動的鍵に必要不可欠である。さらに、RoT はワンタイムプログラム(OTP)メモリを用いて静的鍵を格納し、認証用の不変鍵を提供するようにしている。しかし、不揮発性メモリは製造工程の追加が必要で、実装コストを増加させることになる。

本論文は、安全なシステムの RoT で使用されるハードウェアの基本的要素について提案している。特定用途向け集積回路(ASIC) およびプログラム可能な集積回路FPGAに対して、周波数崩壊に基づいてTRNGを実装するための新しい回路を提案し、物理現象から抽出されるエントロピを改善している。さらに、これらの回路は、プロセス、電圧、および温度の変動に対する耐性を持ち、健全なエントロピソースが保証される。また、新しい統合のNVRAM TRNG-PUF は、標準 CMOS製造技術によるシングルフローティングゲートSFGにより、センスアンプの準安定性を利用している。NVRAMのSFGを用いて TRNG 応答を調整し、メタスタビリティに基づくTRNGに生じる低エントロピ抽出が軽減される。また、アンチヒューズビットセルを使用して、標準的な CMOS 製造技術でプロセスを追加することなく、統一型OTP-PUF メモリの提供を可能とした。最後に、TLS 1.3 と互換性のある ChaCha20-Poly AEAD ハードウェア構造が提案され、ハードウェア効率の点でAESに基づく暗号とは異なるソリューションが提供できるようになった。さらに、新しい AEAD 構造は、ラッチを使用した FPGA 実装で生じるミスマッチを利用する PUF と統合され、AEAD 実装でのワンタイム鍵生成に新しい機能を加えている。

本論文で提案した新しい安全なハードウェアプリミティブは、それぞれの起動プロセス、認証、および暗号化などのアプリケーションの品質を向上させるためにFPGAや標準 CMOS テクノロジを用いて実装された。セキュリティで保護されたシステムで RoT の各必須ハードウェアプリミティブを実装する際、先端テクノロジを使用する必要がなく、標準 CMOS テクノロジで実装可能な回路を提案した。全体システムにおける RoT 領域が占有オーバーヘッドを削減することができた。

This page intentionally left blank.

Abstract

In the last decades, electronic systems have focused on increasing speed and reducing power consumption. However, in recent years security has taken importance due to the critical information management in the systems. Therefore, procedures are implemented into the systems to mitigate or reduce the attacks against critical information. The procedures focus on the booting process, authentications, and encryptions, using some secure primitives. The primitives are constructed with cryptography algorithms and different physical phenomena used to extract some properties for random number generation, key generation, and secure storage, respectively. The secure primitives provide the essentials Root-of-Trust (RoT). As a result, critical information is protected in the main working steps of the system.

Cryptography algorithms provide a secure method to send information between systems. Currently, the most used cryptography protocol for end-to-end secure channels in computer networks is Transport Layer Security (TLS). The TLS typically uses ciphers based on the Advanced Encryption Standard (AES) to exchange information. In addition, due to the different vulnerabilities caused by side-channel attacks and the algorithm's complexity, hardware acceleration is the only way to mitigate these problems. Nonetheless, TLS 1.3 introduces the ciphers based on ChaCha20 to provide another solution different from AES ciphers, resulting in the ChaCha20-Poly1305 Authenticated Encryption with Associated Data (AEAD).

The static and dynamic entropy generated by non-deterministic physical phenomena can be exploited to obtain a True Random Number Generator (TRNG) and a Physical Unclonable Function (PUF). Nevertheless, these physical phenomena are mitigated due to the undesirable effects in the conventional circuits, resulting in a security problem caused by low entropy extraction and reliability implementation problems. In this way, to prevent a disastrous scenario in the system's security, some strategies, calibration steps, and health tests are implemented, mitigating the low entropy extraction and guaranteeing the physical phenomena.

Finally, the storage of the critical information of the system, like keys used in the booting process, represents a vital part of the RoT, ensuring the non-corruption of these keys. In this way, a Non-Volatile Random Access Memory (NVRAM) is requested for dynamic keys used in the RoT, used in key derivations applications. In addition, a One-Time Program memory (OTP) is requested by the RoT to store static keys, providing an immutable key for

authentication purposes. Nevertheless, the non-volatile memories request an additional layer in the implementation, increasing the cost of the application.

This dissertation presents a group of hardware primitives used in the RoT for secure systems. A novel group of strategies to implement a TRNG based on frequency collapse in Application-Specific Integrated Circuits (ASIC) and Field Programmable Gate Arrays (FPGA) is proposed, improving the entropy extracted from the physical phenomena. In addition, the strategies provide resistance against Process, Voltage, and Temperature (PVT) variations and guarantee a healthy entropy source. Besides, a novel unified NVRAM-TRNG-PUF exploits the metastability in the sense amplifier using a Single Floating Gate (SFG) in standard CMOS technology. The SFG of the NVRAM is used to calibrate the TRNG response, mitigating the low entropy extraction presented in TRNG based on metastability. Also, a unified OTP-PUF memory is presented without additional masks in standard CMOS technology, using an anti-fuse bit cell. Finally, a ChaCha20-Poly AEAD hardware construction compatible with TLS 1.3 is presented, providing a different solution than a cipher based on AES in terms of hardware efficiency. In addition, a novel AEAD construction is unified with a PUF exploiting the mismatch presented in an FPGA implementation using latches, providing a new feature in the One Time Key generation in the AEAD implementation. The novel secure primitives presented in this dissertation are implemented to improve the quality of the respective application, generating the option to use in bad environments for secure primitives like FPGA and standard general-purpose CMOS technology.

Contents

Acknowledgements ix

Contents xvi

List of Abbreviations xx

List of Figures xxii

List of Tables xxiv

1 Introduction 1
- 1.1 What is the Root of Trust (RoT) in a Secure Systems? 1
- 1.2 Motivation and Key Contributions 2
  - 1.2.1 Motivation 2
  - 1.2.2 Key Contributions 4
- 1.3 Dissertation Layout 5

2 Literature Review 9
- 2.1 Overview 9
- 2.2 Key Contributions comparison with related works 13
  - 2.2.1 True Random Number Generator (TRNG) 13
  - 2.2.2 Cryptography primitives compatible with Transport Layer Security version 1.3 14
  - 2.2.3 Unified hardware primitives implementation 14

3 Proposed TRNG exploiting the Frequency Collapse in MMRO 16
- 3.1 TRNG based on Frequency Collapse 16
  - 3.1.1 Physical Phenomena 18
  - 3.1.2 Analytical Model 20
- 3.2 TRNG Implementation 20
  - 3.2.1 FPGA Implementation 20
  - 3.2.2 ASIC Implementation 21
- 3.3 Results and Comparison 22
  - 3.3.1 FPGA Results 22
  - 3.3.2 ASIC Results 26
- 3.4 Summary 31

4 Proposed Non-Volatile Random-Access Memory (NVRAM) in Standard CMOS Technology 32

4.1 NVRAM in Standard CMOS Technology . . . 32
4.1.1 Physical Phenomena NVRAM . . . 32
4.1.2 ASIC implementation . . . 33
4.1.3 Measured Results . . . 35
4.2 Unified NVRAM-TRNG-PUF . . . 36
4.2.1 Physical Phenomena of Unified TRNG-PUF . . . 37
4.2.2 Analytical Model of Unified TRNG-PUF . . . 38
4.2.3 Unified NVRAM-TRNG-PUF Measured Results . . . 40
4.3 Summary . . . 45

5 Proposed One Time Program (OTP) memory in Standard CMOS Technology 47
5.1 OTP memory in Standard CMOS Technology . . . 47
5.1.1 Physical Phenomena . . . 48
5.1.2 AF OTP bit cell ASIC implementation . . . 48
5.1.3 High-Voltage Driver . . . 50
5.1.4 Measurement Results and Comparison . . . 52
5.2 Unified OTP-PUF . . . 54
5.2.1 Results and Comparison . . . 55
5.3 Summary . . . 57

6 Proposed ChaCha20-Poly1305 Authenticated Encryption with Additional Data (AEAD) 59
6.1 ChaCha20 primitive . . . 59
6.1.1 Unified ChaCha20-PUF . . . 62
6.1.2 Results and Comparison . . . 63
6.2 Poly1305 primitive . . . 67
6.2.1 Results and Comparison . . . 68
6.3 Authenticated Encryption with Additional Data (AEAD) . . . 70
6.3.1 Results and Comparison . . . 73
6.4 Summary . . . 76

7 Conclusion and Future Work 78
7.1 Conclusion . . . 78
7.1.1 Achievements . . . 78
7.1.2 Limitations . . . 79
7.2 Future Work . . . 79
7.2.1 Develop High-Voltage Analog Circuits . . . 79
7.2.2 Improve the area efficiency in the non-volatile arrays . . . 79
7.2.3 Verify the side-channel attacks in the AEAD primitives 79

Bibliography 92

A Related Publications 94
A.1 Journal . . . 94
A.2 Conference . . . 94

B Full Publication List 95

B.1 Journal 95
B.2 Conference 96

Author Biography **97**

This page intentionally left blank.

List of Abbreviations

AAD	**A**dditional **A**uthenticated **D**ata
AEAD	**A**uthenticated **E**ncryption with **A**ssociated **D**ata
AES	**A**dvanced **E**ncryption **S**tandard
AF	**A**nti-**F**use
AMRV	**A**bsolute **M**aximum **V**oltage **R**atio
ASIC	**A**pplication-**S**pecific **I**ntegrated **C**ircuits
CMOS	**C**omplementary **M**etal-**O**xide-**S**emiconductor
CSA	**C**arry-**S**ave-**A**dder
CURE	**CU**stomizable and **R**esilient **E**nclaves
DAR	**D**ouble-**A**dd-**R**educe
DRC	**D**esign-**R**ule-**C**heck
DT	**D**irect **T**unneling
EEPROM	**E**rasable **P**rogrammable **R**ead-**O**nly **M**emory
ESD	**E**lectro-**S**tatic **D**ischarge
FF	**F**lip-**F**lop
FG	**F**loating **G**ate
FN	**F**owler-**N**ordheim
FPGA	**F**ield-**P**rogrammable **G**ate **A**rray
FSM	**F**inite **S**tate **M**achine
GE	**G**ate **E**quivalent
GARO	**GA**lois **R**ing **O**scillator
H2L	**H**igh-to-**L**ow
IEC	**I**nternational **E**lectrotechnical **C**ommission
IID	**I**ndenpendent and **I**dentically **D**istributed
ISA	**I**nstruction **S**et **A**rchitecture
ISO	**I**nternational **O**rganization of **S**tandardization
KDF	**K**ey **D**erivation **F**unction
LUT	**L**ook**U**p **T**able
L2H	**L**ow-to-**H**igh
MAC	**M**essage **A**uthentication **C**ode
MCV	**M**ost **C**ommon **V**alue
MMRO	**M**ulti-**M**odal **R**ing **O**scillator
MulAcc	**Mul**tiplier and **Acc**umulator
MTP	**M**ulti-**T**ime **P**rogrammable
NVM	**N**on-**V**olatile **M**emory
NVRAM	**N**on-**V**olatile with **R**andom **A**ccess **M**emory
NIST	**N**ational **I**nstitute of **S**tandards and **T**echnology
OTK	**O**ne **T**ime **K**ey
OTP	**O**ne **T**ime **P**rogram

PDK	**P**rocess and **D**esign **K**it
PFD	**P**hase and **F**requency **D**etector
PUF	**P**hysical **U**nclonable **F**unction
PVT	**P**rocess **V**oltage **T**emperature
QR	**Q**uarter **R**ound
RRAM	**R**esistive **R**andom **A**ccess **M**emory
RO	**R**ing **O**scillator
RoT	**R**oot-**o**f-**T**rust
SFG	**S**ingle **F**loating **G**ate
SGX	**S**oftware **G**uard e**X**tension
SNM	**S**tatic **N**oise **M**argin
SoC	**S**ystem **o**n **C**hip
SRAM	**S**tatic **R**andom **A**ccess **M**emory
TEE	**T**rusted **E**xecution **E**nvironment
TCB	**T**rusted **C**omputing **B**ase
TDDB	**T**ime-**D**ependent **D**ielectric **B**reakdown
TLS	**T**ransport **L**ayer **S**ecurity
TS	**T**hreshold **S**witching
TRNG	**T**rue **R**andom **N**umber **G**enerator
UDC	**U**p-**D**own **C**ounter
VT	**V**oltage **T**hreshold

List of Figures

1.1 RoT hardware primitives in a secure system. 3

2.1 Literature review summary. 9

3.1 Architecture of the MMRO and conventional RO used in the TRNG implementation. 16
3.2 Architecture of the TRNG based on frequency collapse. 17
3.3 Architecture of the PFD circuit. 17
3.4 Frequency collapse phenomenon in the phase domain. 18
3.5 Frequency collapse in the time domain. 18
3.6 Frequency collapse in the frequency domain. 19
3.7 Placement restriction and FPGA primitive used to implement the MMRO. 21
3.8 Suggestion proposed for ASIC implementation in the TRNG construction. 22
3.9 TRNG implementation with the respective suggestion in FPGA. 23
3.10 ASIC implementation of the TRNG in ROHM 0.18μm CMOS technology with and without placement constraints. 27
3.11 Minimum entropy with PVT variations of the TRNG implementation without placement suggestions. 28
3.12 Minimum entropy with PVT variations of the TRNG implementation with the placement suggestions. 29

4.1 Cross-section of NVRAM with and without control gate methodology. 33
4.2 Schematic differential NVRAM bit cell. 33
4.3 ASIC implementation of an NVRAM bit cell in ROHM 0.18μm CMOS technology. 34
4.4 Retention characteristics of the programmed bit cell. 36
4.5 Architecture of typical (a) and proposed (b) TRNG-PUF based on metastability. 37
4.6 Stable and unstable modes in the sense amplifier caused by the NVRAM. 37
4.7 Metastable zone in the unified NVRAM-PUF-TRNG implementation. 38
4.8 Metastability response of sense amplifier based on latches. . . 39
4.9 Unified NVRAM-TRNG minimum entropy with PVT variations. 40
4.10 ASIC implementation of an NVRAM array in ROHM 0.18μm standard CMOS technology. 43

4.11 Inter- and Intra-die Hamming distance of unified NVRAM-PUF in nominal conditions. 43

5.1 PMOS AF micrograph in a 0.18μm standard CMOS technology. 49
5.2 PMOS AF characterization with current limitations. 50
5.3 CMOS devices cross section in a Standard technology. 50
5.4 Schematic of the high-voltage driver implementation. 51
5.5 ASIC implementation of the high-voltage driver in ROHM 0.18μm standard CMOS technology. 52
5.6 ASIC implementation of the 128-bit OTP array in ROHM 0.18μm standard CMOS technology. 53
5.7 Layout of the basic 2 OTP bit cell with the sense amplifier. . . 55
5.8 Inter- and Intra-die Hamming distance of unified OTP-PUF in nominal conditions. 55

6.1 Architecture of the ChaCha20 primitive. 61
6.2 Quarter Round operation architecture. 61
6.3 State of the art summary in PUF based on ring oscillators. . . . 62
6.4 Metastability response in balanced and unbalanced latches. . . 62
6.5 Schematic of PUF implementation in ChaCha20 core. 63
6.6 Placement constraints proposed in the PUF in FPGA implementation. 64
6.7 Micrograph of ChaCha20 core on 0.18μm GP CMOS technology. 65
6.8 Hamming distances response of PUF based on unbalanced latches. 66
6.9 Architecture of Poly1305 primitive. 68
6.10 ASIC implementation of the Poly1305 primitive in ROHM 0.18μm CMOS technology. 69
6.11 AEAD construction based on ChaCha20 and Poly1305 primitives. 70
6.12 AEAD interaction with the memory of the system. 71
6.13 Procedure of ChaCha20-Poly1305 AEAD compatible with TLS version 1.3. 72
6.14 Typical and proposed OTK generation procedures. 72
6.15 ChaCha20-Poly1305 AEAD ASIC implementation in ROHM 0.18μm CMOS technology. 74
6.16 ChaCha20-Poly1305 AEAD run time comparison. 75

List of Tables

2.1 Summary and comparison with the state of the art of TRNG in an FPGA implementation. . . . 13
2.2 Summary and comparison with the state of the art of TRNG in an ASIC implementation. . . . 13
2.3 Summary and comparison with the state of the art of AEAD primitive in FPGA and ASIC implementations. . . . 14
2.4 Summary and comparison with the state of the art of unified primitives . . . 15

3.1 NIST SP800-90B Initial test results of TRNG based on Frequency Collapse with Non-IID assumption in FPGA implementation. 24
3.2 NIST SP800-90B Restart test results of TRNG based on Frequency Collapse with Non-IID assumption in FPGA implementation. . . . 24
3.3 Summary and comparison of minimum entropy. . . . 25
3.4 NIST SP800-22 test results of TRNG based on Frequency Collapse in FPGA implementation. . . . 25
3.5 Performance summary and comparison in FPGA. . . . 26
3.6 NIST SP800-90B results with non-IID assumption. . . . 27
3.7 NIST SP800-90B restart results with non-IID assumption. . . . 28
3.8 Minimum entropy comparison of ASIC implementation. . . . 29
3.9 NIST SP800-22 results with PVT variations. . . . 30
3.10 Summary and Comparison Results of the TRNG in a ASIC implementation. . . . 30

4.1 Operational voltages of the NVRAM bit cell. . . . 35
4.2 Operational results of the NVRAM implemented . . . 35
4.3 NIST SP800-90B results with IID assumption. . . . 40
4.4 Minimum entropy comparison of Unified NVRAM-TRNG. . . . 41
4.5 NIST SP800-22 results with PVT variations. . . . 42
4.6 Quality results of the Unified NVRAM-PUF implementation. . 44
4.7 Summary and comparison of the NVRAM-TRNG-PUF implementation . . . 44

5.1 Performance summary and comparison. . . . 53
5.2 Operational results of the OTP memory implemented. . . . 54
5.3 Quality results of the Unified OTP-PUF implementation. . . . 56
5.4 Summary and comparison of the OTP-PUF implementation . 56

6.1 Performance summary and comparison in the FPGA implementation. 64
6.2 Comparison results of ChaCha20 primitive in ASIC implementation 65
6.3 Comparison of the PUF quality in FPGA implementation. . . . 66
6.4 Performance summary and comparison of Poly1305 in FPGA implementation 68
6.5 Comparison results of Poly1305 primitive in ASIC implementation 69
6.6 Performance summary and comparison of AEAD implementation in FPGA. 73
6.7 Comparison of AEAD compatibles with TLS version 1.3 73
6.8 Summary and comparison in ASIC implementation 75

Chapter 1

Introduction

1.1 What is the Root of Trust (RoT) in a Secure Systems?

Root-of-Trust (RoT) is the fundamentals of security in a system, providing the infrastructure and methods to generate and store keys used in cryptography operations like encryption, signing, and authenticated key exchange, respectively. A secure RoT typically includes hardware primitives to achieve the objectives of the operations in the security system. The secure system is proposed based on the Trusted Computing Base (TCB), proposing a Trusted Execution Environment (TEE) model in different computer architectures. For example, some industrial TEE models are proposed to generate a secure zone with secure primitives inside the system, like Intel Software Guard eXtension (SGX) [1–4], ARM TrustZone [5], and AMD Secure Encrypted Virtualization (SEV) [6]. On the other hand, academic TEE models are proposed based on RISC-V Instruction Set Architecture (ISA) [7] generating solutions like Hell-Five MultiZone [8], TIMBER-V [9], Keystone [10], Sanctum [11], CUstomizable and Resilient Enclaves (CURE) [12], and an Isolated Heterogeneous Architecture for Key Scheduling [13].

The hardware primitives in the RoT present a principal challenge in all the TEE models explained due to the hardware primitives requesting special technologies to reproduce a physical phenomenon in a random number generation, key generation or non-volatility storing key. In this way, multiple chips in different technologies are necessary to connect in the package or use enclave peripherals in the system, generating vulnerabilities in the security system. Therefore, the hardware primitives like True Random Number Generator (TRNG), Physical Unclonable Function (PUF), a One-Time Program (OTP) memory, Non-Volatile Random Access Memory (NVRAM), and cryptography accelerators need to be implemented in the same chip to providing a secure robustness system.

The TRNG and PUF are the first hardware primitives needed in the RoT of a secure system. The main challenge of these primitives is the nonconventional implementations due to these primitives being based on undesirable physical phenomena in a circuit. Typically, the TRNG and PUF primitives

need a calibration step to increase or reduce the noise, and process variations [14–17], respectively. In addition, these hardware primitives are implemented in a standalone peripheral into the secure system, generating an area overhead in the system. However, the TRNG and PUF primitives are recently unified with some essential system modules like a Static Random Access Memory (SRAM) [18–20], reducing the area overhead caused by hardware primitives of the RoT in the system. In another way, the primitives based on Non-Volatile Memory (NVM) request technology with additional masks and processes due to the high voltage used in the application. The NVRAM and OTP primitives are typically implemented using non-standard technology to endure the electrical requirements. Nevertheless, some approaches demonstrate the reliability of NVM primitives in standard CMOS technology [21–30]. Therefore, these primitives in a standard technology create the opportunity to reduce costs. In addition, the system's robustness is increased by the integrated implementation in the same chip. Finally, the cryptography algorithms can provide authentication and communication procedures with others systems. For example, Transport Layer Security (TLS) provides the most used end-to-end secure channel in computer networks [31]. Typically, the ciphers used in TLS are based on the Advanced Encryption Standard (AES) with AES-CCM and AES-GCM implementations. However, a new approach based on ChaCha20-Poly1305 is provided in TLS version 1.3, generating another option different than ciphers based on AES.

1.2 Motivation and Key Contributions

1.2.1 Motivation

Two decades ago, a system's efforts were focused on increasing performance in terms of throughput and transistors per area and reducing the power consumption of the implementation. Nonetheless, the security of the systems takes importance due to the vital information contained. Therefore, an RoT is created to provide security in the procedures for booting, authentication, and data transmission in the system [1–6, 8–11, 32]. However, the physical phenomena of hardware primitives in RoT denote a challenge to obtain a homogeneous system implementation.

The principal approaches provided by the hardware primitives in an RoT into a system are key generation, key storage and external data exchange, as shown in Figure 1.1. The architecture of the trusted system with RoT typically follows the recommendations provided by International Organization for Standardization (ISO) and the International Electrotechnical Commission (IEC) [33]. The generation of the different keys of the system needs to follow some recommendations provided by the National Institute of Standards and Technology (NIST) in SP800-133 [34] and SP800-57 [35–37], respectively. The standard advice has a different class of key for the procedures in the secure system, for example, the device, derived, root and manufacturer keys. The device key is provided with the process variation of the implementation,

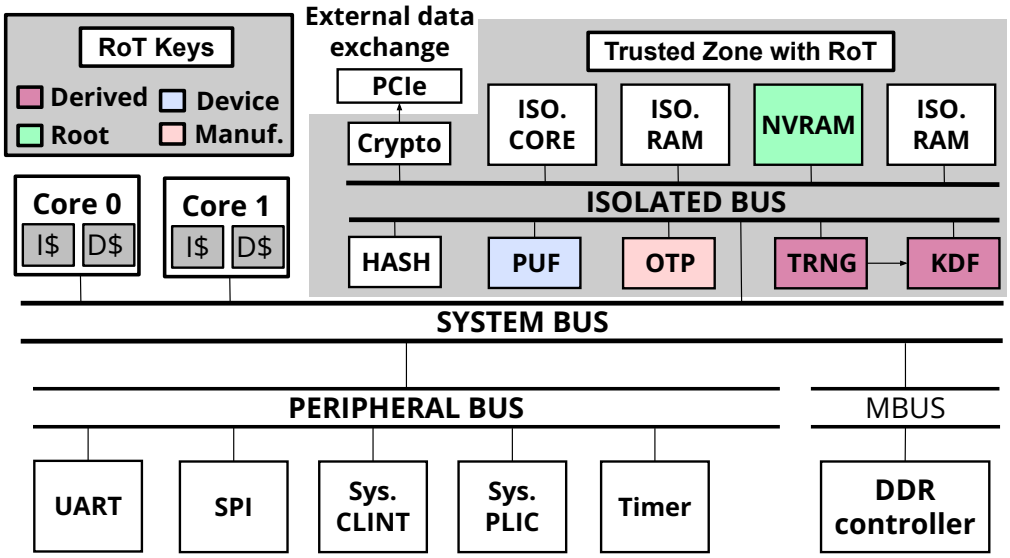


FIGURE 1.1: RoT hardware primitives in a secure system.

generating an identification per each device provided by the PUF primitive. In addition, a derived key needs to be generated using a KDF module with a random number provided by a TRNG primitive. In another way, the RoT needed an NVM to store the manufacturer and root key. The manufacturer key is stored using an OTP memory primitive, guaranteeing immutability in time. Moreover, the root key must be stored in an NVM in the trusted zone of the secure system, using an NVRAM typically. Finally, cryptography and hashes functions are used to exchange external information and authentication procedures, supporting the different attacks [38–40].

This dissertation aims to explore and propose hardware primitives to be used in an RoT in a secure system, exploiting the different undesirable effects in integrated circuits to obtain a homogeneous implementation in different implementation environments. The principal aims can be summarized as follows:

1. Provide an implementation of TRNG primitive used for derived key generation applications in FPGA and ASIC implementations.
2. Provide an implementation of PUF primitive used for device key generation applications in FPGA and ASIC implementations.
3. Provide an implementation of NVM for manufacturer key storage application using an OTP memory primitive in standard CMOS technology.
4. Provide an implementation of NVM for root key storage application using an NVRAM primitive in standard CMOS technology.
5. Propose an AEAD cryptography primitive to provide a secure channel for data exchange applications.

6. Study and evaluate the option to unify some primitives to reduce the resources overhead of the RoT in a secure system in FPGA and ASIC implementation.

1.2.2 Key Contributions

Due to the challenges in implementing each essential primitive of the RoT in a secure system, this thesis describes the problems and solutions to obtain the essential hardware primitive in standard CMOS technology, eliminating the necessity to use expensive technology. In addition, some of the primitives are implemented in an FPGA, exploiting the undesirable effects in this implementation environment. Finally, this research presents some unified approaches of hardware primitives, reducing the area overhead of the RoT in the system. The work of this dissertation can be categorized into four significant contributions as follows:

1. **TRNG based exploiting the Frequency Collapse in MMRO:** The TRNG based on frequency collapse is presented for FPGA and ASIC implementation, proposing some suggestions to reproduce the physical phenomena, improving the minimum entropy extracted. In FPGA implementation, the suggestion of inherent mismatch variations on the devices guaranteeing the physical phenomena occurs. Therefore, a high minimum entropy TRNG can be used in an RoT in FPGA implementation. The ASIC implementation denotes a high minimum entropy extraction with PVT variations, demonstrating the health and robustness of the entropy source. One ROHM-180nm chip was made and measured the TRNG exploiting the frequency collapse in MMRO.
2. **A Unified NVRAM-TRNG-PUF in Standard CMOS Technology:** The proposed unified implementation into an NVRAM creates the option to have three principal primitives of the RoT in one module, reducing the area overhead in the system. The NVRAM application is reproduced into a standard CMOS technology without additional masks and processes. Besides, the TRNG and PUF functions are unified without additional circuits, using the erase and program process in the NVRAM module. The TRNG function passed the entropy and statistical test provided by the NIST SP800-22 and SP800-90B. On the other hand, the PUF function exhibits the characteristic based on the uniformity, uniqueness and reliability quality results in a key generation application. The unified implementation allows storing the key in an NVM primitive. Two ROHM-180nm chips were made, one to characterize the differential NVRAM bit cell and the other to implement an NVRAM array with the analog circuits necessary to connect with a secure system.
3. **A Unified OTP-PUF in Standard CMOS Technology:** The OTP memory implementation can provide a method to introduce a manufacturer key due to the NVM intrinsic characteristic of the break oxide phenomenon. The OTP can be programmed only once, reducing the possibility of editing the manufacture key storage in the RoT of the system.

Besides, the OTP implementation is unified with a PUF primitive, creating the option to store a key based on the process variation, generating the chip key identification. The PUF function generates the device key exploiting the metastability of the sense amplifier when the OTP bit cell and the cell reference are programmed, showing enough inter and intra-chip Hamming distances for this application. Two ROHM-180nm chip was made, one to characterize the oxide breakdown phenomena into the Anti-fuse (AF) OTP bit cell and the second to implement the OTP memory array with different analog circuits to integrate into a system.

4. **ChaCha20-Poly1305 AEAD compatible with TLS version 1.3:** An Authenticated Encryption with Additional Data (AEAD) based on cryptography ChaCha20-Poly1305 primitives is proposed, providing a different solution than typical AEAD based on Advanced Encryption Standard (AES) for Transport Layer Security (TLS) version 1.3. The AEAD construction is compatible with TLS, the most used cryptography protocol for an end-to-end secure channel for computer networks. The hardware primitives and AEAD implementations are implemented in FPGA and ASIC, denoting a competitive performance with the others options in TLS version 1.3. In addition, a new approach is proposed to generate the One-Time Key (OTK) in the AEAD construction, unifying the ChaCha20 and PUF primitives. One ROHM-180nm chip was made to implement the ChaCha20, Poly1305 and AEAD primitives into a secure system.

1.3 Dissertation Layout

The dissertation is divided into seven chapters as follows:

1. **Chapter 1:** Introduction

   This chapter describes the hardware primitives related to this dissertation, presenting the problems to obtaining a Root-of-Trust (RoT) for secure systems. Moreover, this work's motivation is related to the problems and benefits of the RoT in homogeneous secure system implementation. Finally, this work's key contributions and dissertation layout are presented.

2. **Chapter 2:** Literature Review

   This chapter presents the state-of-the-art hardware primitives used in an RoT in a secure system, describing the novel topology in each primitive. In addition, a summary and comparison of the key contributions are depicted. Finally, the comparison of the key contributions of this work with the related works is presented.

3. **Chapter 3:** TRNG Exploiting the Frequency Collapse in MMRO

This chapter presents the proposed TRNG based on frequency collapse in MMRO. The implementation is divided into parts. First, the FPGA implementation describes the suggestions and constraints to reproduce the physical phenomena in this environment. Second, a suggestion in the ASIC implementation is related to reducing the mismatch variations and increasing the minimum entropy normalized of the entropy source. Besides, the entropy source shows robustness against PVT variations with the suggestion applied in the ASIC implementation. The FPGA and ASIC implementation passed the statistical and entropy test provided by the NIST SP800-22 and SP800-90B, respectively. One ROHM-180nm chip contains a microcontroller based on a RISC-V processor with the TRNG based on frequency collapse connected into the system.

4. **Chapter 4:** A Unified NVRAM-TRNG-PUF in standard CMOS technology

   This chapter implements an NVRAM into a standard CMOS technology using the Fowler-Nordheim (FN) phenomena into a Single Floating Gate (SFG) methodology without additional masks or processes. In addition, TRNG-PUF primitives are unified into the NVRAM implementation, exploiting the metastability in the sense amplifier of the implementation. The implementation presents a PUF response when the charges in the two SFG of the differential NVRAM bit cell are erased, generating a biased metastability response. The PUF implementation shows competitive uniformity, uniqueness and reliability results compared with other unified implementations. The TRNG response needs a calibration step based on the health test of the NIST SP800-90b to compensate for the process variations in the sense amplifier to obtain an unbiased metastability response. The TRNG implementation passes the statistical and entropy test provided by the NIST SP800-22 and SP800-90B. Two ROHM-180nm chips are made, one for characterizing the FN phenomena in an NVRAM bit cell and the second to implement the NVRAM array with the basic analog circuits to connect to the system.

5. **Chapter 5:** A Unified OTP-PUF in standard CMOS technology

   This chapter implements an OTP memory into a standard CMOS technology using the oxide breakdown phenomena in Anti-Fuse (AF) bit cells. Besides, a PUF primitive is unified, exploiting the metastability in the sense amplifier of the implementation. The PUF function is reproduced when the AF OTP bit cell and the reference bit cell are programmed. In addition, the PUF exhibits quality results for device key generation applications. Two ROHM-180nm chips are made, the first chip is implemented to characterize the oxide breakdown phenomena in the OTP bit cell, and the second chip is implemented the OTP array with the analog circuits to connect into a system.

6. **Chapter 6:** ChaCha20-Poly1305 Authenticated Encryption with Additional Data (AEAD)

In this chapter, a ChaCha20-Poly1305 Authenticated Encryption With Additional Data (AEAD) compatible with TLS version 1.3 is constructed to provide a primitive to exchange information with other systems. The architecture of the ChaCha20, Poly1305 and AEAD primitives are presented, reporting the results for FPGA and ASIC implementations. Moreover, a unified ChaCha20-PUF implementation is presented to improve the security of the One-Time Key (OTK) generation in the AEAD application. One ROHM-chips was made to implement the ChaCha20, Poly1305 and ChaCha20-Poly1305 AEAD primitives into a secure system.

This page intentionally left blank.

Chapter 2

Literature Review

2.1 Overview

This chapter related the state-of-the-art hardware primitives used to construct an RoT in a secure system. Figure 2.1 illustrates the literature review summary of the related work for TRNG, PUF, NVRAM, OTP and cryptography hardware primitives implementation. In addition, the contributions of this dissertation are described. The unified implementations in each hardware primitive are highlighted in purple. Finally, the unified In-memory implementations are sorted into NVRAM, OTP, SRAM, RRAM and TS NVM primitives, respectively.

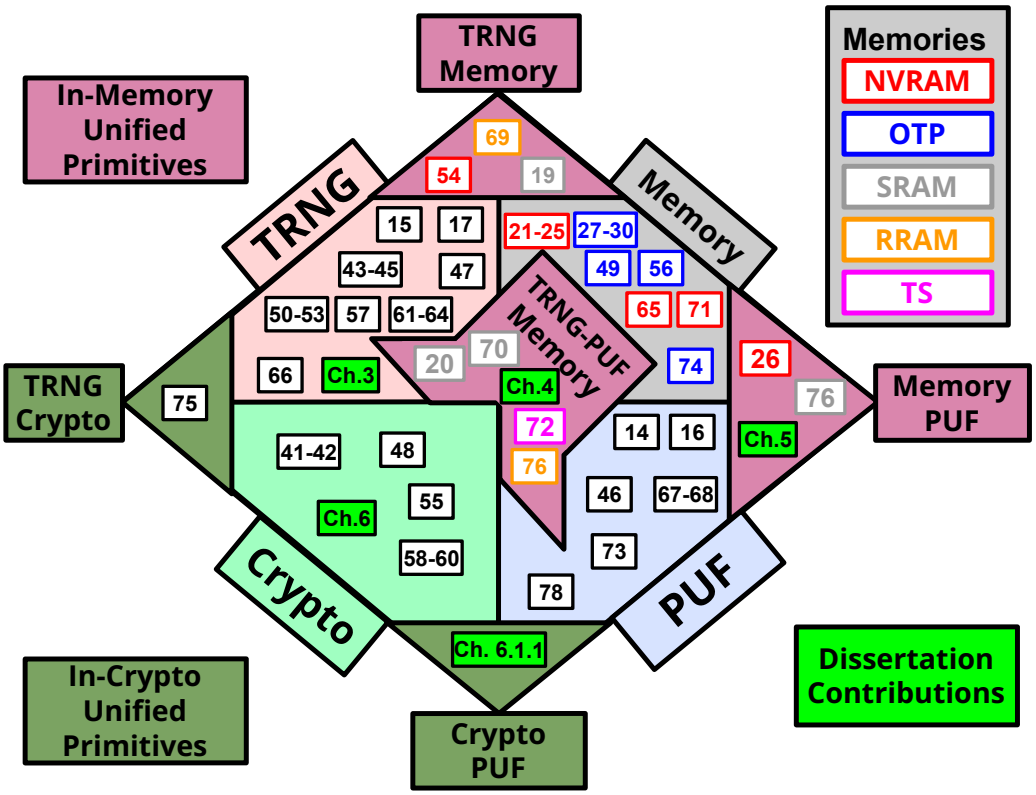


FIGURE 2.1: Literature review summary.

- K. Ohsaki *et al*. (1994) [21]: reported an NVM using a single poly in an Electrically Erasable Programmable Read-Only Memory (EEPROM) implementation in standard CMOS technology.
- R. McPartland *et al*. (2000) [22]: reported an NVM flash memory in a standard CMOS process for low-density applications.
- J. Kim *et al*. (2003) [27]: described a 3-T OTP ROM cell array using the oxide breakdown phenomena in standard CMOS technology.
- J. Raszka *et al*. (2004) [23]: showed an NVM in standard CMOS technology for security applications.
- L. Henzen *et al*. (2008) [41]: report a hardware implementation of Salsa20, ChaCha and Rumba cryptography algorithms.
- I. A.-Badillo *et al*. (2008) [42]: reported the results of AES-CCM crypto core in an FPGA implementation.
- S. K. Mathew *et al*. (2012) [43]: showed a TRNG based on metastability in latches for high-performance applications.
- N. D. Phan *et al*. (2013) [28]: described a 2-Kb OTP implementation for passive RFID application in standard CMOS technology.
- K. Yang *et al*. (2014) [44]: presented the TRNG based on frequency collapse using an odd number of stages in an MMRO.
- T.-K. Kuan *et al*. (2014) [45]: reported a TRNG based on metastability in latches for energy efficiency applications.
- K. Yang *et al*. (2015) [14]: used an MMRO for PUF application, increasing the process variations in the entropy source.
- C. Li *et al*. (2015) [24]: described an NVM with Multi-Time Programmable (MTP) using a pseudo-differential architecture.
- Z. Liu *et al*. (2015) [29]: reported an OTP memory based on gate oxide breakdown phenomena in standard CMOS technology.
- S.-K Park *et al*. (2016) [25]: characterized a single-poly NVM array in standard CMOS technology.
- K. Yang *et al*. (2016) [15]: reported the TRNG based on frequency collapse with an even number of stages in MMRO.
- N. Torii *et al*. (2016) [46]: described a PUF evaluation based on latches in a 40-nm ASIC implementation.
- S. K. Mathew *et al*. (2016) [47]: reported a TRNG based on multiple entropy sources in 14-nm FinFET CMOS.
- V.-P Hoang *et al*. (2016) [48]: described a compact and low-power AES-CCM in ASIC and FPGA implementation.

- S. Chung *et al*. (2016) [49]: implemented an OTP memory based on fuse methodology from 0.35-μm to 28-nm CMOS technology.
- J. Cartagena *et al*. (2016) [50]: related the first implementation of a TRNG based on frequency collapse in an FPGA implementation.
- C. Li *et al*. (2017) [51]: described a TRNG based on metastability in an FPGA implementation.
- S. Tao *et al*. (2017) [52]: presented a TRNG based on metastability in ternary values latches.
- M. A. Sarkisla *et al*. (2018) [53]: presented a TRNG based on metastability in flip-flops in an FPGA implementation.
- B. Ray *et al*. (2018) [54]: reported a unified NVRAM and TRNG based digitizing of the noise in each bit line.
- M. Rodriguez *et al*. (2018) [55]: presented an AES-GCM in an FPGA implementation for wire-speed application.
- S. Chung *et al*. (2019) [56]: implemented a 4Kx8 OTP memory based on fuse methodology on 22-nm FD-SOI.
- S. Park *et al*. (2019) [57]: described a TRNG with an external entropy source based on a natural radioisotope decay.
- P.-S Yeh *et al*. (2019) [19]: presented a unified SRAM-TRNG with self-convergent trimming with in-cell storage.
- G. Kanda *et al*. (2019) [58]: presented a high-throughput, low-area ChaCha20-Poly1305 AEAD hardware implementation in ASIC and FPGA.
- J. Pfau *et al*. (2019) [59]: presented a ChaCha 8/12/20 crypto core in an FPGA implementation.
- S. Koteshwara *et al*. (2019) [60]: reported an architecture optimization of AES-GCM crypto core in an FPGA implementation.
- J. Cañada *et al*. (2020) [30]: related an on-chip self-powered OTP memory in standard CMOS technology.
- Y. Lu *et al*. (2020) [61]: presented a TRNG based on jitter quantization in an FPGA implementation.
- H. Gomez *et al*. (2020) [62]: described a TRNG based on frequency collapse in MMRO using multiple entropy sources.
- Y. Luo *et al*. (2020) [63]: showed a high-performance TRNG based on chaotic cellular automata implementation.
- S. T. Chandrasekaran *et al*. (2020) [64]: showed a TRNG based on the noise into a ring oscillator in a stochastic delta-sigma modulator..
- S.-K Park *et al*. (2020) [65]: characterized an NVRAM array based on the single poly implementation in standard CMOS technology.

- J. Ardila *et al*. (2020) [26]: described a unified NVRAM-PUF using the initial charges in the NVRAM bit cell in standard CMOS technology.
- J. Yang *et al*. (2020) [66]: implemented a TRNG based on the stochastic short-term recovery of charge-trapping FinFET.
- D. Deng *et al*. (2020) [67]: reported a PUF implementation using a ring oscillator with hybrid logic gates.
- M. Garcia-Bosque *et al*. (2020) [68]: related the proposal and analysis of PUF based on Galois Ring Oscillator (GARO) in an FPGA implementation.
- F. Pebay-Peyroula *et al*. (2020) [69]: implemented a unified RRAM and TRNG exploiting the resistive variations in 40-nm CMOS technology.
- Q. Zhou *et al*. (2021) [16]: explored the PUF-Controlled PA spectral using a conventional ring oscillator with a digital-to-analog converter.
- S. Taneja *et al*. (2021) [70]: presented the novel unification of SRAM-TRNG-PUF for ubiquitous hardware security.
- D. Lane *et al*. (2021) [71]: reported the concept of ULTRA-RAM on GaAs substrates to obtain an NVRAM application.
- Q. Ding *et al*. (2021) [72]: described a unified TRNG and PUF into a Threshold Switching Array in a 3D NbOx implementation.
- T. Chen *et al*. (2021) [17]: implemented a TRNG based on metastability in flip-flops implementing an integrated entropy monitor.
- A. A. Zayed *et al*. (2021) [73]: presented a low-power PUF implementation based on mismatch variation in MMRO using FinFET technology.
- S.-Y S. Chou *et al*. (2021) [74]: presented a 16Kb Anti-Fuse OTP memory in high-K metal-gate 5-nm FinFET technology.
- S. Taneja *et al*. (2021) [75]: presented a unified crypto core and TRNG based on metastability in latches.
- S. Taneja *et al*. (2022) [20]: presented an extension of the unified SRAM-TRNG-PUF using jitter accumulation and leakage current for the TRNG and PUF primitives, respectively.
- B. Gao *et al*. (2022) [76]: presented a unified RRAM-PUF-TRNG based on process variations and digitizing the noise in RRAM bit cells for PUF and TRNG functions, respectively.
- J.-W Nam *et al*. (2022) [77]: presented a unified SRAM-PUF employing a body voltage control technique.
- J. Park *et al*. (2022) [78]: described a strategy in a PUF based on MMRO increasing the mismatch variations in the implementation.

2.2 Key Contributions comparison with related works

2.2.1 True Random Number Generator (TRNG)

The state-of-the-art TRNG primitive is divided into two parts due to the different environment implementations, changing the conditions to reproduce the physical phenomena of the entropy source.

TABLE 2.1: Summary and comparison with the state of the art of TRNG in an FPGA implementation.

Research	Year	Entropy Source	Physical Phenomena	Innovation
[50]	2016	Multi-Modal Ring Oscillator	Frequency Collapse	The first implementation of a TRNG based on Frequency collapse in FPGA is reported.
[51]	2017	Latch	Metastability	A high-throughput low-resources TRNG implementation.
[53]	2018	Ring Oscillator	Wake-up and shut-down time	A reduction of resource utilization compared to typical TRNG based on RO is described.
[57]	2019	Radioisotope	Natural decay	Utilization of external entropy source with a non-conventional phenomenon is reported.
[61]	2020	Jitter	Flip-Flop	A Robust TRNG against PVT using a calibration circuit in FPGA is described.
This work [79] Chapter 3.2.1	2021	Multi-Modal Ring Oscillator	Frequency Collapse	The problems and solutions to implement a TRNG based on frequency collapse are described, obtaining a high minimum entropy.

Table 2.1 compares the state-of-the-art with the contributions of this thesis to implement a TRNG in an FPGA environment. This research provides the problems and solutions to implement a TRNG based on frequency collapse in Multi-Modal Ring Oscillator (MMRO) in an FPGA

TABLE 2.2: Summary and comparison with the state of the art of TRNG in an ASIC implementation.

Research	Year	Entropy Source	Physical Phenomena	Innovation
[43]	2012	Latch	Metastability	A high throughput TRNG in 45-nm CMOS technology for a high-performance microprocessor.
[44]	2014	Multi-Modal Ring Oscillator	Frequency Collapse	The novel TRNG based on frequency collapse with odd stages is presented in 28-nm and 65-nm CMOS technology with a noise generator.
[45]	2014	Latch	Metastability	A TRNG with low-energy consumption is presented with an offset calibration.
[15]	2016	Multi-Modal Ring Oscillator	Frequency Collapse	A TRNG based on frequency collapse with even stages is presented in 40nm and 180nm implementations, showing robustness against PVT variations.
[47]	2016	Multiple entropy sources	Mestastability	A TRNG with multiple entropy sources is described in 14nm FinFET CMOS technology.
[52]	2017	Latch	Metastability	A TRNG based on Ternary values latches is presented in 65nm CMOS technology.
[62]	2020	Multi-Modal Ring Oscillator	Frequency Collapse	A TRNG based on frequency collapse using multiple entropy sources based in odd stages MMRO is reported in 18.0nm CMOS technology.
[63]	2020	Cellular Automata	Chaotic System	A high-performance TRNG based on Chaotic Cellular automata is presented in 40nm CMOS technology.
[64]	2020	Delta-Sigma Modulator	Jitter	A TRNG exploiting the jitter in clock reference in an analog circuit is reported in 65nm CMOS technology.
[66]	2020	Charge Trapping	Short-term Recovery	A novel TRNG based on short-term recovery into a charge-trapping FinFET is implemented in 14nm CMOS technology.
[17]	2021	Flip-Flop	Metastability	A TRNG with an integrated entropy monitor is implemented in 130nm CMOS technology.
This work [80] Chapter 3.2.2	2022	Multi-Modal Ring Oscillator	Frequency Collapse	Proposes suggestions to obtain a robust and healthy entropy source in TRNG based on MMRO in 180nm ASIC implementation.

implementation, applying constraints to reduce the mismatch in the TRNG response described in chapter 3.2.1.

Additionally, Table 2.2 compares the related works in an ASIC implementation. This work proposes a suggestion for implementing the entropy source in TRNG based on frequency collapse in MMRO. The suggestion demonstrates a robust and healthy entropy source implementation against PVT variations with high minimum entropy. This key contribution is explained in Chapter 3.2.2.

2.2.2 Cryptography primitives compatible with Transport Layer Security version 1.3

Transport Layer Security (TLS) is the most used cryptography protocol to provide an end-to-end secure channel in computer networks. TLS version 1.3 provides a new primitive different than the cipher based on Advance Encryption Standard (AES). The ChaCha20-Poly1305 AEAD is introduced to mitigate the security deficiencies discovered in the last ciphers in the cryptography protocol. Table 2.3 compiles the AEAD primitive comparison with the related works compatible with TLS version 1.3. The AEAD primitive proposed in this thesis is explained in chapter 6.

TABLE 2.3: Summary and comparison with the state of the art of AEAD primitive in FPGA and ASIC implementations.

Research	Year	Ciphers	Innovation
[41]	2008	ChaChaX-Salsa20-Rumba	A hardware implementation of ChaChaX, Salsa20 and Rumba primitives are reported in 180nm CMOS technology.
[42]	2008	AE-CCM	An FPGA implementation of AES-CCM AEAD is reported for wire-speed cryptography applications.
[48]	2016	AES-CCM	A compact and low-power AES-CCM in FPGA and 65nm ASIC implementation is reported for wireless body area network applications.
[58]	2019	AEAD ChaCha20-Poly1305	A high-throughput low-area ChaCha20-Poly1305 AEAD compatible with TLS 1.3 is reported in FPGA and 180nm ASIC implementation.
[60]	2019	AEAD AES-GCM	An architecture optimization of AES-GCM AEAD is reported for an FPGA implementation.
[59]	2019	ChaChaX	A hardware perspective is presented in the ChaCha8/12/20 primitives in an FPGA implementation.
This work [81] Chapter 6	2022	AEAD ChaCha20-Poly1305	ChaCha20-Poly1305 AEAD hardware primitive compatible with TLS version 1.3 is described, supporting full and fragmented blocks in FPGA and 180-nm ASIC implementation.

2.2.3 Unified hardware primitives implementation

The unification of the hardware primitives is a hot topic in the state-of-the-art due to the area reduction compared to the primitive into a standalone implementation. The undesirable effects in the system's basic modules are used to unify the hardware primitives, obtaining TRNG and PUF primitives. Table 2.4 summarizes the unified primitives

TABLE 2.4: Summary and comparison with the state of the art of unified primitives

Research	Year	Unified primitives	Innovation
[26]	2020	TRNG PUF	A unified NVRAM and PUF implementation is reported to exploit the initial values in the floating gate bit cell.
[72]	2021	TS TRNG PUF	A unified TS-TRNG-PUF is implemented in a 3D NbOx bit cell, exploiting the noise and process variations.
[75]	2021	TRNG Crypto	A unified TRNG-Crypto core is reported in 40nm CMOS technology exploiting the metastability in latches.
[20]	2022	SRAM TRNG PUF	A unified SRAM-TRNG-PUF implementation is reported in 28nm CMOS technology for ubiquitous hardware security.
[76]	2022	RRAM TRNG PUF	A unified RRAM-TRNG-PUF implementation is described in 40nm technology for IoT applications.
[77]	2022	SRAM PUF	An SRAM implementation is unified with a PUF primitive using the body voltage control technique.
This work [82] Chapter 4	2022	NVRAM TRNG PUF	A unified NVRAM-TRNG-PUF is implemented, exploiting the metastability in the sense amplifier in standard CMOS technology.
This Work Chapter 5	2022	OTP PUF	An OTP memory implementation is unified with a PUF primitive exploiting the process variation in standard CMOS technology.
This work [83] Chapter 6.1.1	2022	Crypto PUF	A unified ChaCha20-PUF is reported, exploiting the metastability in unbalanced latches in FPGA implementation.

proposed with the related work. This thesis proposes unified OTP-PUF primitives exploiting the process variations in the oxide breakdown phenomena, obtaining a metastable response in the sense amplifier in the memory related in chapter 4. In addition, a unified NVRAM-TRNG-PUF primitive is implemented, exploiting the metastability in the sense amplifier of the memory using the NVRAM bit cell to determine the mode operation described in chapter 5. Finally, a ChaCha20 crypto core and PUF primitives are unified, exploiting the metastability in latches in an FPGA implementation described in chapter 6.1.1.

Chapter 3

Proposed TRNG exploiting the Frequency Collapse in MMRO

This chapter's primary goal is to explain the fundamentals of the TRNG based on frequency collapse, describing the physical phenomena in the entropy source with the respective analytical model. The FPGA and ASIC implementation of the TRNG is described highlighting the problems and solutions of the respective implementation. In addition, some strategies are proposed to improve the minimum entropy, obtaining a robust and healthy implementation. Next, the results and comparison with the related works are presented. Finally, the summary of the chapter is described.

3.1 TRNG based on Frequency Collapse

The frequency collapse phenomena are exploited in Multi-Modal Ring Oscillator (MMRO), extracting the entropy necessary to generate a random number. The number of edges in an MMRO determines the frequency oscillation when each edge consists of a nand cell with the respective delay cells. Figure 3.1 illustrates the three edges of MMRO used in the entropy source. The

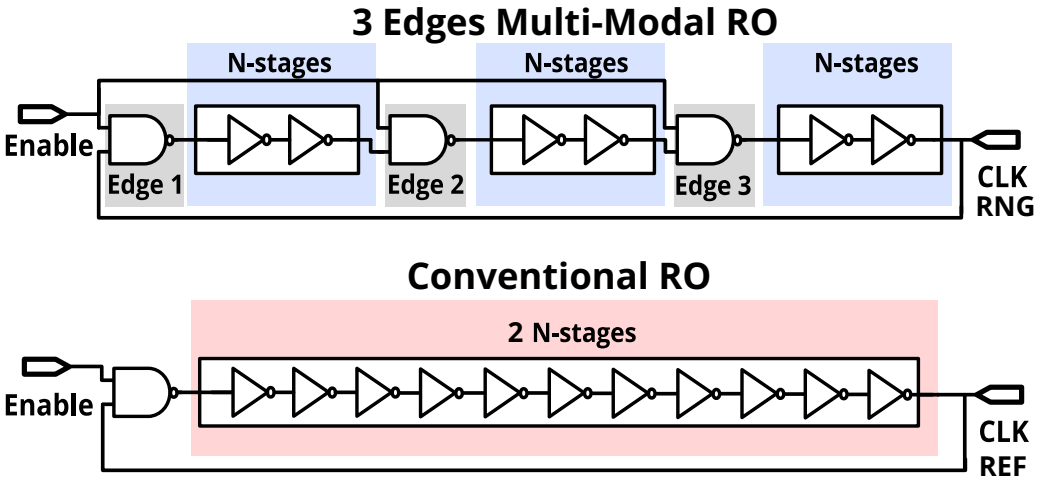


FIGURE 3.1: Architecture of the MMRO and conventional RO used in the TRNG implementation.

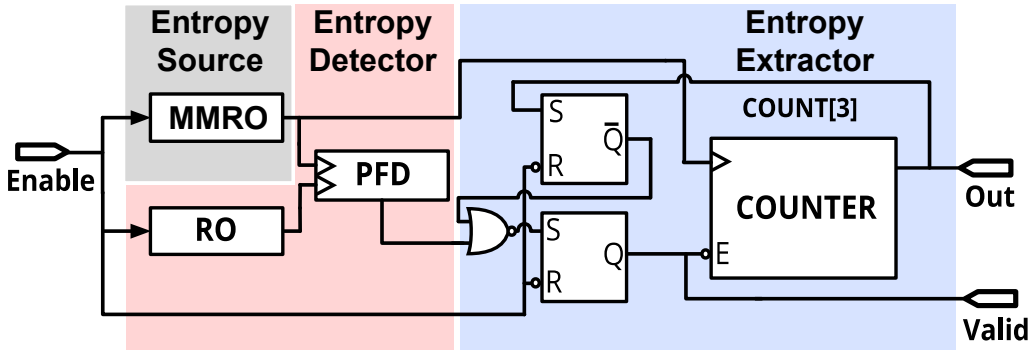


FIGURE 3.2: Architecture of the TRNG based on frequency collapse.

number of stages into an edge denotes the number of inverters used by delay cells. However, an even number of inverted is necessary to guarantee oscillation in the MMRO. On the other hand, a conventional Ring Oscillator (RO) is used to generate the clock reference in the TRNG. Nevertheless, the conventional RO needs to use two times the number of stages compared to one stage in the MMRO, ensuring a higher frequency after the frequency collapse in the MMRO.

Figure 3.2 shows the block diagram of the TRNG based on frequency collapse. The TRNG architecture consists of an *Entropy Source*, *Entropy Detector* and *Entropy Extractor*, respectively. The *Entropy Source* is implemented using an MMRO, exploiting the non-deterministic physical phenomena to generate entropy. The *Entropy Detector* catches the frequency collapse in the MMRO compared with a clock reference, using Phase and Frequency Detector (PFD). The PFD triggers a signal when the frequency of the MMRO is less than the clock reference. Finally, an *Entropy Extractor* digitizes the entropy generated in the MMRO using an 8-bit counter. A latch is used to manage the enable of the counter, mitigating false positives in the count using the fourth bit of the counter. Besides, the counter uses the clock generated in the entropy source.

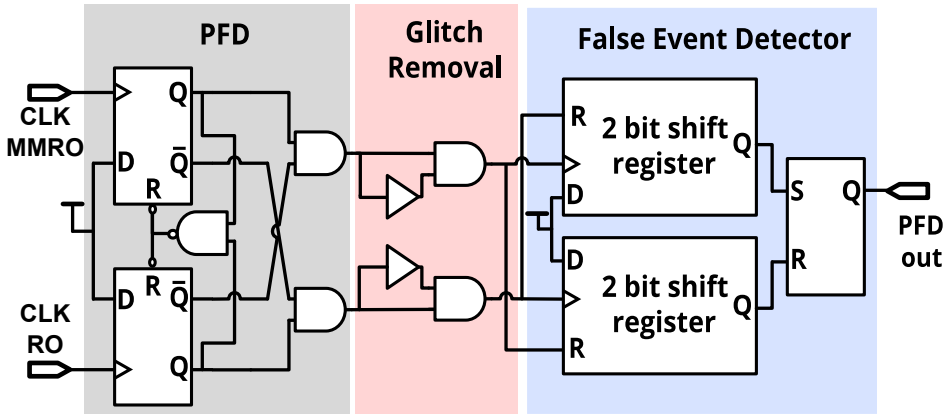


FIGURE 3.3: Architecture of the PFD circuit.

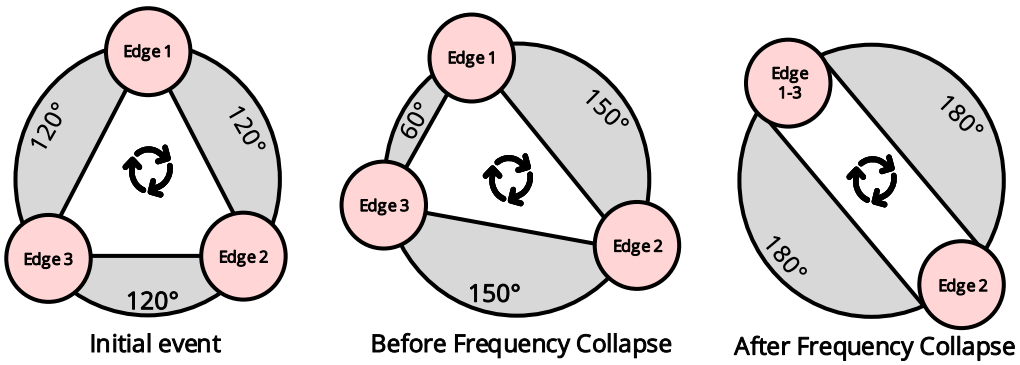


FIGURE 3.4: Frequency collapse phenomenon in the phase domain.

The PFD architecture is shown in Figure 3.3. The full digital PFD can detect any change between *CLK MMRO* and *CLK RO*. Nevertheless, a *Glitch Removal* is implemented due to the signal deformation causing the jitter accumulation. In addition, a *False Event Detector* is added to eliminate the undesirable effects caused for the full digital implementation of the PFD. Finally, a latch controls the output of the PFD, determining when the collapse frequency phenomena occur.

3.1.1 Physical Phenomena

The frequency collapse phenomenon is shown in Figure 3.4. The *Initial Event* of the three edges MMRO is illustrated, highlighting the balanced phase in all pulses introduced in each edge. However, the jitter breaks the phase balance,

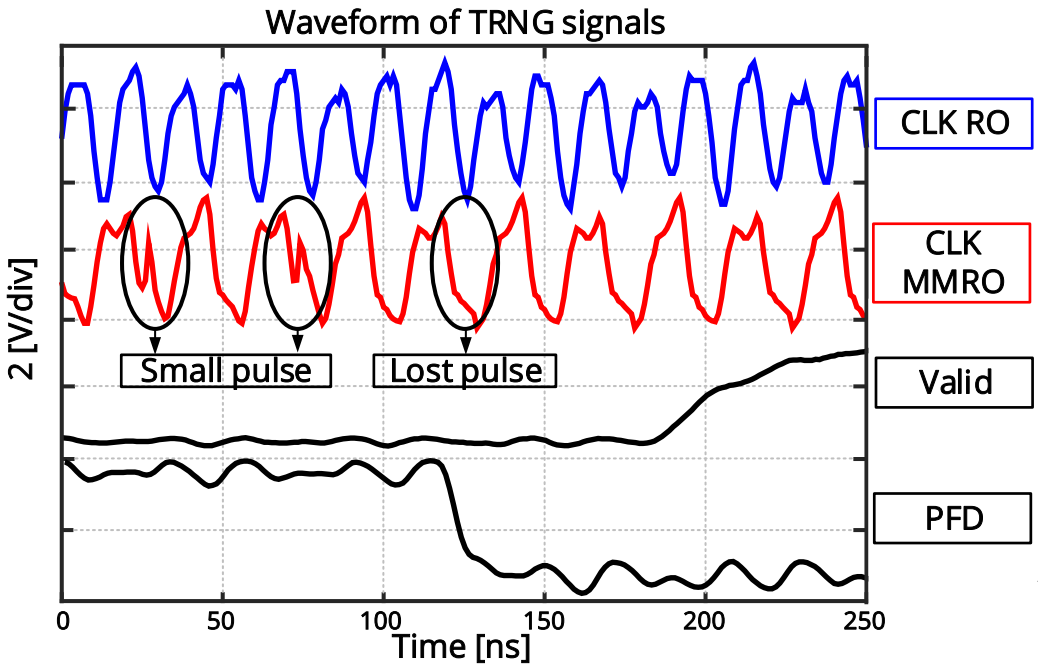


FIGURE 3.5: Frequency collapse in the time domain.

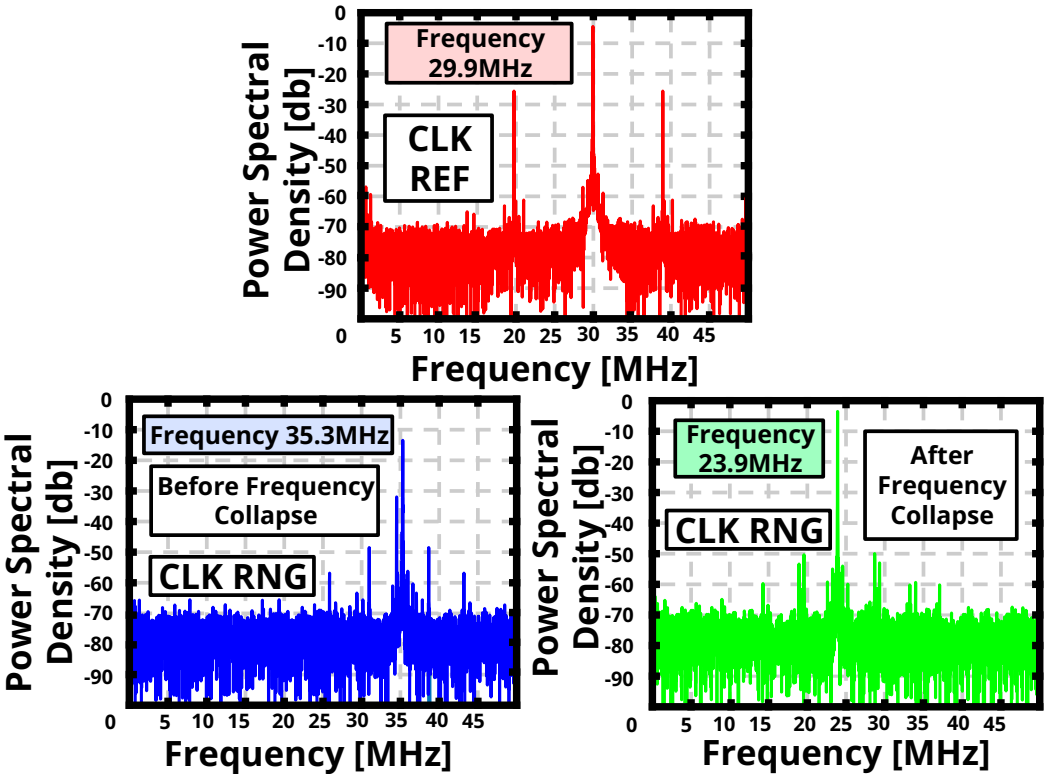


FIGURE 3.6: Frequency collapse in the frequency domain.

causing the approaching of some pulses before the frequency collapse. The frequency collapse phenomenon occurs when two pulses are mixed, reducing the oscillation frequency in the entropy source. The entropy generated is the time necessary to obtain a random frequency collapse in the MMRO.

Figure 3.5 illustrates the frequency collapse phenomenon measured in the time domain. The *CLK MMRO* highlighted in red shows the frequency collapse phenomenon before and after it occurs. The unbalance generated by the jitter into the MMRO causes the mixing of two pulses in the MMRO. At this moment, the PFD detects the difference between the entropy source and clock reference, deactivating the enable of the counter in the *Entropy Extractor*. However, some cycles are necessary due to the *False Event Detector* in the PFD. After the frequency collapse occurs, the signal *Valid* is triggered to indicate that the random number is already generated.

Figure 3.6 exhibits the frequency collapse phenomenon measured in the frequency domain. The physical phenomenon of the entropy source is demonstrated with the power spectral density of the *CLK RNG*. The frequency of *CLK RNG* is reduced by 32.29% after the physical phenomenon occurs. Therefore, the oscillation frequency of the *CLK REF* needs to be in the rank between frequency before and after frequency collapse happens. The reference frequency is calibrated with the number of stages in the conventional RO, typically 2N stages of the entropy source.

3.1.2 Analytical Model

In the analytical model for odd edges in the entropy source, the pulse changes the path in each oscillation, reducing the relationship between rising and falling delays compared with the even edges entropy source [15]. Therefore, the analytical model proposed is based on the noise and mismatch in FPGA [79] and ASIC [80], respectively, where the difference between the FPGA and ASIC models is the noise introduced by external signals added in the ASIC implementation. At the time any pulse arrives, the output (T_p) of the MMRO is used to model the physical phenomena, as shown in (3.1).

$$T_p = \sum_{i=nk}^{nk+k/3} [t_0 + \delta_{m(n,i)}] + \sum_{j=1}^{nk+k/3} [\delta_{n(n,j)} + \delta_{e(n,j)}] - \sum_{i=1}^{nk} [\delta_{n(n,i)} + \delta_{e(n,i)}] \quad (3.1)$$

Here, the k and n denote the numbers of edges and cycles, respectively. Besides, the t_0 represents the nominal delay of one pulse in the MMRO. The mismatch is presented by $\delta_{m(n,i)}$. However, in ASIC and FPGA implementations, the mismatch variations can be taken by a constant due to the fixation after fabrication or implementation ($\delta_{m(n,i)} = C$), respectively. The undesirable effects introduced by the external signals are depicted by $\delta_{e(n,i)}$ and $\delta_{e(n,j)}$. These effects can be approximated by $\delta_{e(n,i)} = \delta_{e(n,j)} \sim N(0, \rho^2)$, where ρ depicts the variance of the random inferences of the external signals. On the other hand, the $\delta_{n(n,i)}$ and $\delta_{n(n,j)}$ are the noise introduced into the MMRO. Concluding, the three edges model for MMRO (3.2) demonstrates the variance increase linearly with the number of cycles when $\delta(n,j) = \delta(n,i) \sim N(0, \sigma^2)$ [44].

$$T_p \sim N(\frac{k}{3}[t_0 + C], (2nk + \frac{k}{3})[\sigma^2 + \rho^2]) \quad (3.2)$$

3.2 TRNG Implementation

This section explains the strategies and suggestions in FPGA and ASIC implementation to reduce the mismatch in the MMRO. The suggestions for each implementation are based on the analytical model described in section 3.1.2.

3.2.1 FPGA Implementation

In FPGA implementation, the entropy source presents a problem in reproducing the physical phenomena due to the intrinsic mismatch presented in the FPGA primitives used to construct the MMRO. Figure 3.7 illustrates the suggestions and the FPGA primitive used to reproduce the MMRO. The MMRO is reproduced using a 6-bit Look-Up-Table (LUT) for the NAND and delay cells in the entropy source. However, the LUT primitive is constructed based on a RAM-based function, adding mismatch variations in the FPGA

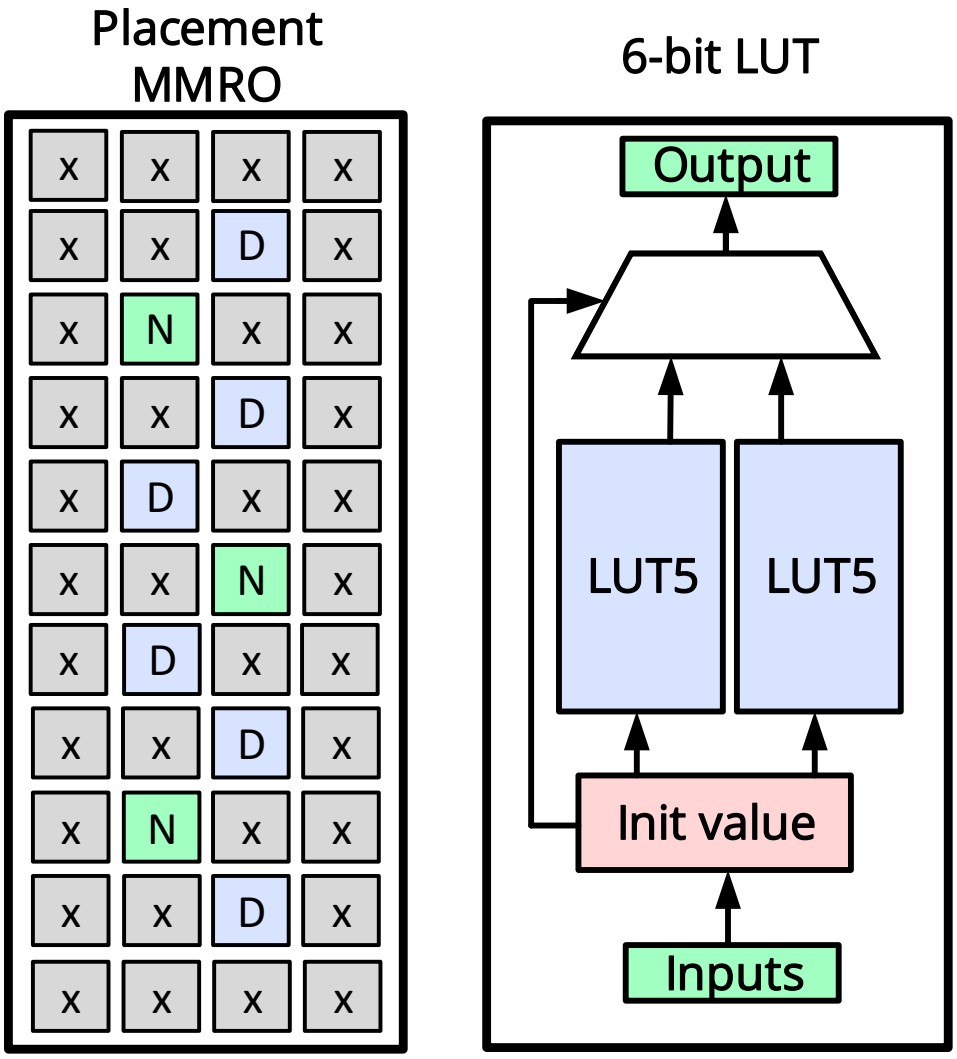


FIGURE 3.7: Placement restriction and FPGA primitive used to implement the MMRO.

implementation. Therefore, a placement restriction is applied to mitigate the undesirable effect added by the FPGA primitive. The NAND (N) and Delay (D) cells are placed in different slices with the same LUT in each slice. In addition, the placement is distributed to obtain the same route length as possible in the MMRO. In another way, an initial value used in each LUT6 is the same in NAND and delay cells in the MMRO.

3.2.2 ASIC Implementation

In ASIC implementation, the entropy source applies a similar placement restriction to FPGA suggestion. However, the ASIC provides better management of the cells in the MMRO. Figure 3.8 shows the ASIC suggestion proposed to reduce the undesirable effects in the entropy source. The MMRO and conventional RO are placed in rows to reduce the wire length between each NAND and delay cells. Additionally, a placement blockage highlighted in gray is applied near the MMRO and conventional RO to reduce the noise

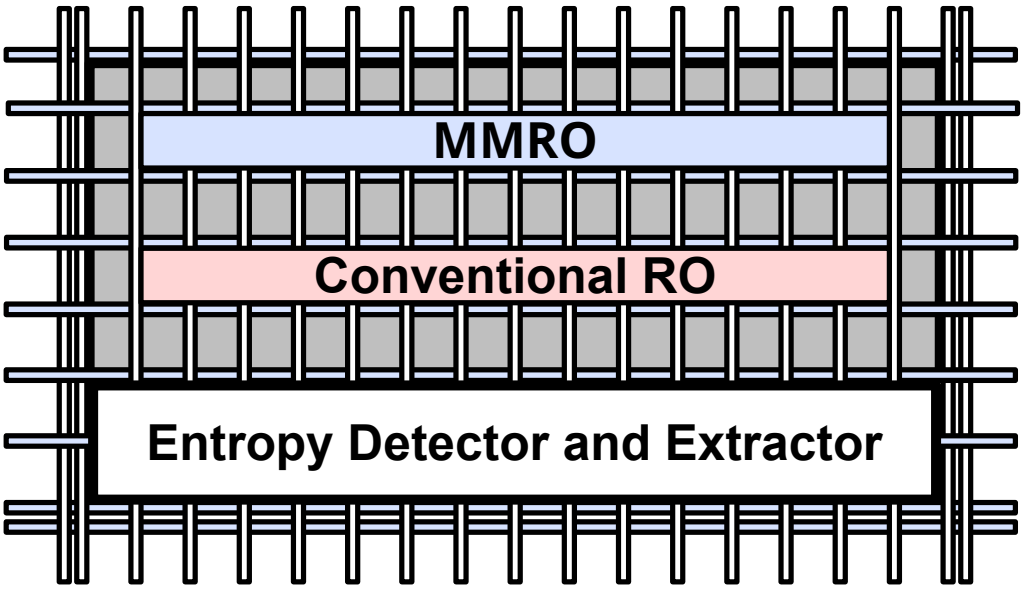


FIGURE 3.8: Suggestion proposed for ASIC implementation in the TRNG construction.

introduced by the external signal from another part of the TRNG. Finally, the *Entropy Detector* and *Entropy Extractor* are placed and routed in an isolated area. The proposed suggestions are to reduce the variations between each cell in the MMRO, increasing the noise contribution to the physical phenomenon.

3.3 Results and Comparison

This section report the results of the TRNG implemented in FPGA and ASIC in terms of area, power and minimum entropy, respectively. Besides, statistical and entropy tests are applied using the NIST SP800-22 [84] and SP800-90B [85] to determine the quality of the entropy source implemented.

3.3.1 FPGA Results

Figure 3.9 illustrates the layout of the TRNG based on frequency collapse in FPGA implementation. The TRNG was implemented in Xilinx Artix-7 XC7A100TCSG324 FPGA. The secure primitive is implemented inside a system based on a RISC-V processor used for measuring purposes. A three entropy source and clock references with different numbers of stages are implemented. The *Entropy Detector* and *Entropy Extractor* are implemented without restriction. However, the suggestion for FPGA implementation is described in section 3.2.1. A placement blockage guard the MMRO and the conventional RO implementation. Besides, the LUT and *INIT Value* per each LUT is the same for the NAND and delays cells into the entropy source. The suggestion applied in the FPGA implementation ensures the physical phenomena in the entropy source occur.

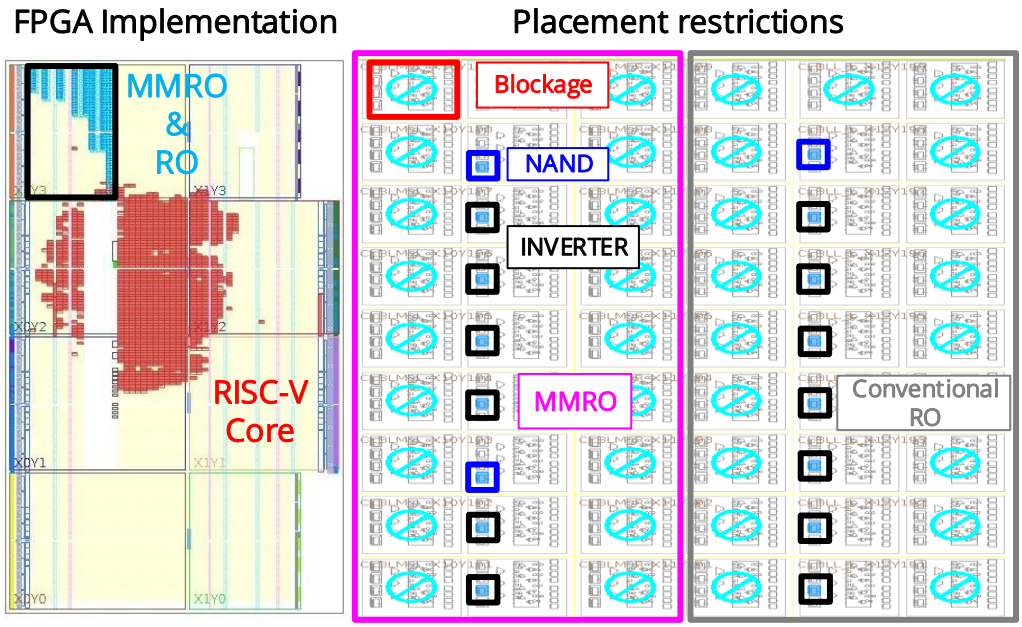


FIGURE 3.9: TRNG implementation with the respective suggestion in FPGA.

The entropy test described in NIST SP800-90B needs to classify the Independent and Identically Distributed (IID) assumption of the entropy source to apply the respective test, preventing the overestimation of the minimum entropy [86, 87]. According to the analytical model for FPGA implementation described in section 3.1.2, the entropy source presents a non-IID assumption due to the high quantity of mismatch in the FPGA environment. Table 3.1 shows the non-IID NIST SP800-90B test in the entropy source implemented in FPGA. The minimum of all estimators determines the minimum initial entropy (H_I). Nevertheless, in the initial non-IID test, a single and long sequence is used to determine the H_I. Therefore, a restart test needs to be applied to prevent an overestimation. The minimum initial entropy obtained is $H_I = 0.8923 * 8 = 7.1432$.

Table 3.2 depicts the NIST SP800-90B restart test results. The estimators presented in the restart test are the same except for Collision, Markov and Compression estimators. The data used in this test is obtained, restating 1000 times. One thousand samples are collected in the entropy source for each restart with $\alpha = 0.01$. Additionally, the MMRO passed all sanity checks in the FPGA implementation. The row and column data sets are constructed based on the samples recollected in the restart test, applying the estimators in these data sets. A minimum entropy is obtained in the restart test applied in the row (H_R) and columns (H_C), respectively. The minimum entropy results are obtained by $H_{min} = min(H_R, H_C, H_I)$. The minimum entropy resulting in the FPGA implementation is $H_{min} = 0.8929 * 8 = 7.1432$.

TABLE 3.1: NIST SP800-90B Initial test results of TRNG based on Frequency Collapse with Non-IID assumption in FPGA implementation.

Non-IID estimators		**1-bit**		**8-bit**	
		P_{value}	**Est.**	P_{value}	**Est.**
Most common value		0.5015	0.9956	0.0042	7.8566
Collision		0.5230	0.9351	—	—
Markov	**P0**	0.4989	0.9957	—	—
	P1	0.4989			
	P2	0.4989			
	P3	0.4989			
Compression		0.0244	0.8929	—	—
t-Tuple		0.5176	0.9501	0.0048	7.7188
Longest repeated substring		0.5008	0.9976	0.0048	7.3538
Multi most common		0.5007	0.9980	0.0040	7.9528
Lag prediction		0.5007	0.9980	0.0039	8.0000
Multi MMC prediction		0.5014	0.9959	0.0040	7.9503
LZ78Y prediction		0.5015	0.9957	0.0040	7.9506

TABLE 3.2: NIST SP800-90B Restart test results of TRNG based on Frequency Collapse with Non-IID assumption in FPGA implementation.

Non-IID estimators in restart test	**8-bit row**		**8-bit column**	
	P_{value}	**Est.**	P_{value}	**Est.**
Most common value	0.0043	7.8566	0.0043	7.8566
t-Tuple	0.0061	7.3538	0.0061	7.3538
Longest repeated substring	0.0048	7.7188	0.0041	7.9436
Multi most common	0.0040	7.9528	0.0041	7.9214
Lag prediction	0.0041	8.0000	0.0041	7.9218
Multi-MMC prediction	0.0040	7.9503	0.0041	7.9434
LZ78Y prediction	0.0040	7.9506	0.0040	7.9445

TABLE 3.3: Summary and comparison of minimum entropy.

	This work	**[61]** 2020	**[57]** 2019
Type of data	Non-IID	Non-IID	Non-IID
Number of bits	8	1	8
Minimum entropy normalized†	0.8929	0.8888	0.7207
Type of entropy source	Noise	Noise	Radio isotope
Physical phenomenon	Frequency collapse	Coherent sampling	Natural decay

† The ideal and maximum value is 1.

Table 3.3 shows the summary results and comparison regarding the minimum entropy of TRNG based on different physical phenomena. The minimum entropy obtained in the TRNG based on frequency collapse is compared with TRNG based on coherent sampling in FF and natural decay in radioactive isotope with a non-IID assumption, demonstrating a competitive minimum entropy.

TABLE 3.4: NIST SP800-22 test results of TRNG based on Frequency Collapse in FPGA implementation.

Statistical test		
Estimators	P_{value}	**Rate**
Frequency	0.3342	96
Block frequency	0.5544	99
Cumulative sums	0.6248	96
Cumulative sums	0.2142	96
Runs	0.0909	99
Longest run	0.7198	100
Rank	0.6579	99
FTT	0.1719	100
Non-overlap template	0.7598	99
Overlap template	0.1296	98
Approx. entropy	0.7792	99
Universal	0.0909	97
Random excursions	0.4522	98
Random excursions variant	0.6575	99
Serial	0.5956	97
Serial	0.8165	97
Linear complexity	0.1088	97

TABLE 3.5: Performance summary and comparison in FPGA.

	This work	[50] 2016	[51] 2017	[53] 2018	[61] 2020
LUT	62	409^{+}	220	236	128*
FF	21	36	0	0	104*
Bit rate [Mb/s]	1.1 to 9.1	N/A	30	N/A	127
Entropy source	Multi-modal RO	Multi-modal RO	Cross-coupled	RO	Vernier interval
Physical phenomenon	Frequency collapsed	Frequency collapse	Metastability	Wake-up & shutdown	Coherent sampling

* Resources are reported in slices, data are converted to LUT and FF.
$^{+}$ The authors implement four entropy sources.

Table 3.4 depicts the FPGA implementation applying the NIST 800-22 statistical test. The test is applied with a level of significance $\alpha = 0.01$, indicating that one would expect a sequence in 100 sequences to be rejected. Besides, a P-value greater than 0.01 is mandatory to consider the random sequence with a confidence of 99%. The non-overlap template, random excursions and random excursion variant are reported passed when all sub-tests are approved. The data is generated by the Xilinx Artix-7 FPGA implementation in nominal conditions.

Table 3.5 compares the TRNG implementation in terms of performance and resource utilization in FPGA implementation. The implementation occupies 62 LUTs and 21 FFs, reporting less resource utilization than the related TRNG in FPGA. On the other hand, the TRNG implemented presents a 1.1 up to 9.1 Mb/s of bit rate. The bit rate is reported in a range due to the characteristics of the physical phenomenon.

3.3.2 ASIC Results

Figure 3.10 illustrates the micrograph of the TRNG implemented in ROHM $0.18\mu m$ standard CMOS technology with and without the placement suggestions described in section 3.2.2. The TRNG with and without the placement suggestion is implemented with an SoC based on a RISC-V processor for measurement purposes. The TRNG implementation with the placement suggestion occupies $25600\mu m^2$ with four entropy sources when each has the respective clock reference. All MMROs with the conventional RO represent 50% of the area occupied in the TRNG implementation. The suggestion applied in the implementation produces a 42% area overhead per entropy source compared with a typical digital implementation.

The TRNG implemented in ASIC with the placement suggestion is analyzed with a non-IID assumption based on the analytical model described in 3.2.2. Table 3.6 reports the entropy test results in four chips in nominal conditions (1.8V and $25°C$). The entropy sources denote a $H_I = 0.9890 * 8 = 7.9120$

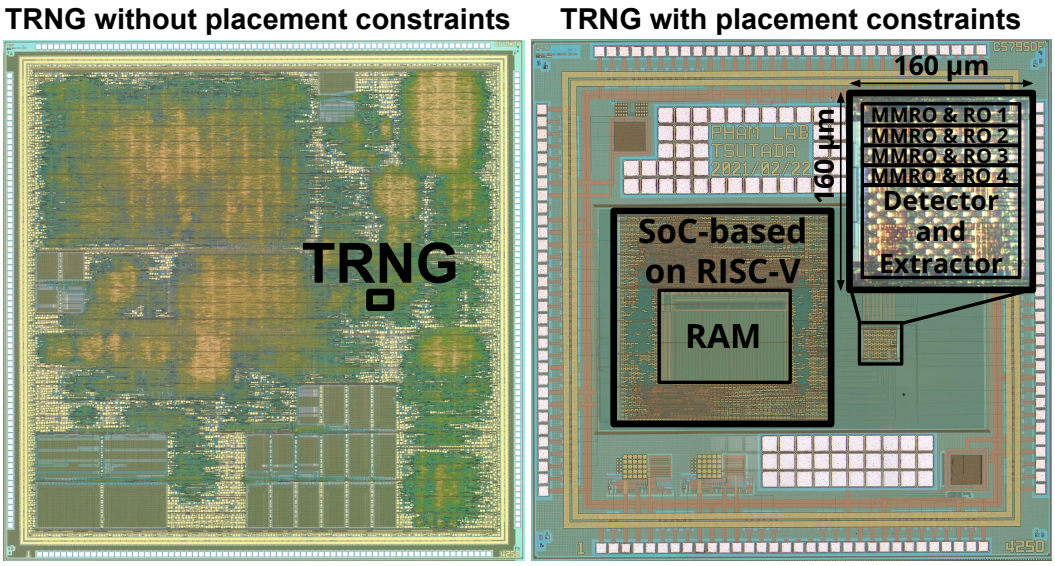


FIGURE 3.10: ASIC implementation of the TRNG in ROHM $0.18\mu m$ CMOS technology with and without placement constraints.

in the four chips measured. The implementation also passes the restart test with four chips, reported in Table 3.7. The row and column entropy are $H_R = 0.9912 * 8 = 7.93$ and $H_C = 0.9912 * 8 = 7.93$, respectively. The minimum entropy resulting in the implementation in nominal conditions is $H_{min} = 0.9890 * 8 = 7.9120$.

TABLE 3.6: NIST SP800-90B results with non-IID assumption.

Non IDD estimators		V = 1.8[V] T= 25[°C]							
		CHIP #1		CHIP #2		CHIP #3		CHIP #4	
		P_{value}	Est.	P_{value}	Est.	P_{value}	Est.	P_{value}	Est.
Most common value		0.5009	0.9971	0.5011	0.9966	0.5010	0.9955	0.5014	0.9959
Collision		0.5031	0.9910	0.5004	0.9988	0.5006	0.9982	0.5002	0.9994
Markov	P0	0.5005	0.9984	0.5001	0.9997	0.5010	0.9956	0.5003	0.9991
	P1	0.4994		0.4994		0.4989		0.4997	
	P2	0.5005		0.5001		0.5010		0.5003	
	P3	0.4994		0.4994		0.4989		0.4997	
Compression		0.0163	0.9898	0.0159	0.9958	0.0161	0.9927	0.0163	0.9898
t-Tuple		0.5038	0.9890	0.5035	0.9899	0.5026	0.9925	0.5013	0.9962
Longest repeated substring		0.5004	0.9986	0.5038	0.9890	0.5005	0.9984	0.5009	0.9974
Multi most common		0.5008	0.9975	0.5000	0.9997	0.5005	0.9984	0.5002	0.9994
Lag prediction		0.5001	0.9995	0.5004	0.9986	0.5005	0.9984	0.5008	0.9976
Multi MMC prediction		0.5006	0.9980	0.5009	0.9972	0.5014	0.9959	0.5003	0.9991
LZ78Y prediction		0.5009	0.9973	0.5010	0.9968	0.5014	0.9957	0.5007	0.9979

TABLE 3.7: NIST SP800-90B restart results with non-IID assumption.

Non IID estimators in restart test	V=1.8[V] T=25[°C]															
	CHIP #1				CHIP #2				CHIP #3				CHIP #4			
	ROW		COLUMN		ROW		COLUMN		ROW		COLUMN		ROW		COLUMN	
	P_{value}	Est.	P_{value}	Est.	P_{value}	Est.	P_{value}	Est.	P_{value}	Est.	P_{value}	Est.	P_{value}	Est.	P_{value}	Est.
Most common value	0.0041	7.93	0.0041	7.93	0.0040	7.96	0.0040	7.96	0.0040	7.96	0.0040	7.96	0.0041	7.95	0.0041	7.95
t-Tuple	0.0040	7.96	0.0040	7.96	0.0040	7.96	0.0040	7.96	0.0040	7.96	0.0040	7.96	0.0041	7.94	0.0041	7.94
Longest repeated substring	0.0039	7.99	0.0039	7.99	0.0039	7.99	0.0039	7.99	0.0039	7.99	0.0039	7.99	0.0040	7.96	0.0039	7.99
Multi most common	0.0039	7.97	0.0039	7.98	0.0039	7.99	0.0039	7.98	0.0040	7.96	0.0040	7.96	0.0039	7.98	0.0039	7.99
Lag prediction	0.0040	7.96	0.0040	7.96	0.0040	7.96	0.0040	7.96	0.0039	7.98	0.0040	7.96	0.0041	7.95	0.0040	7.96
Multi-MMC prediction	0.0039	7.98	0.0039	7.97	0.0039	7.98	0.0039	7.98	0.0039	7.98	0.0039	7.99	0.0040	7.96	0.0039	7.98
LZ78Y prediction	0.0039	7.97	0.0039	7.98	0.0039	7.99	0.0039	7.99	0.0039	7.98	0.0039	7.99	0.0039	7.98	0.0041	7.95

Figure 3.11 depicts the minimum entropy normalized of the TRNG implemented without the placement suggestions, exhibiting a 0.9005 of minimum entropy with -10-10°C and 1.2-V conditions. On the other hand, Fig. 3.12 illustrates the minimum entropy normalized of the implementation with the placement suggestion, reporting a 0.9789 of minimum entropy normalized with -10-10°C and 1.2-V conditions. The minimum entropy is measured with Process, Voltage and Temperature (PVT) variations. The four chips are measured in 1.2V until 2.0V and -10°C until 125°C for voltage and temperature variations, The initial non-IID estimators and the restart test are applied for

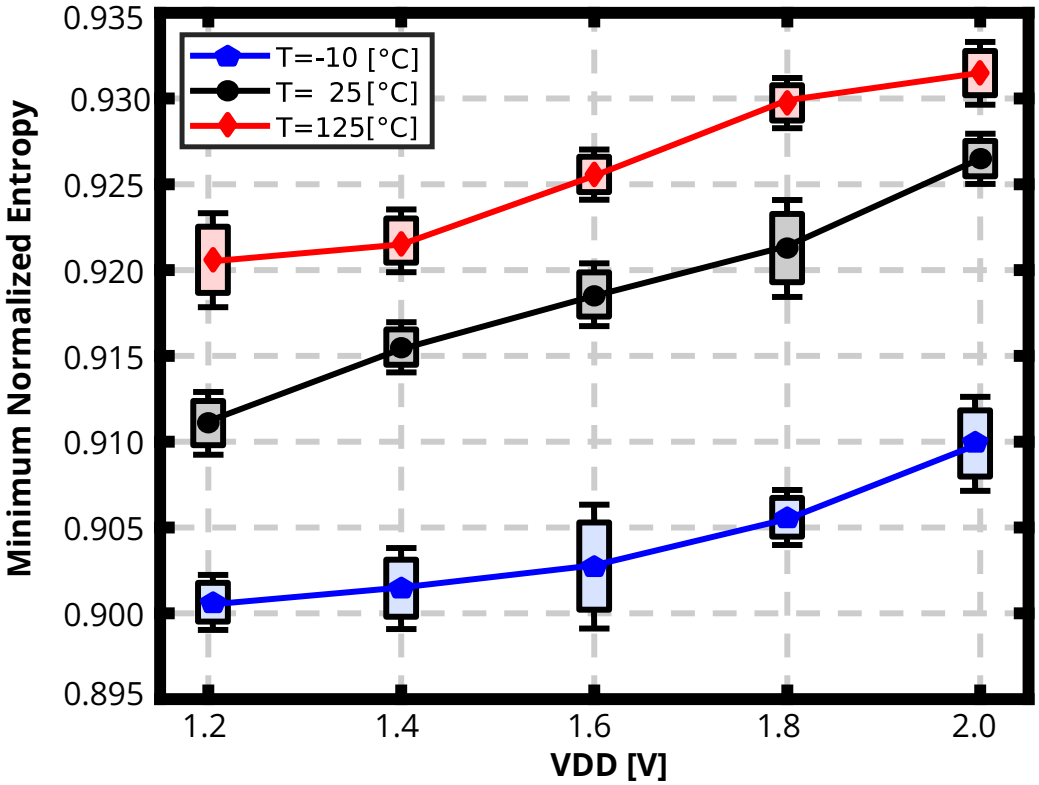


FIGURE 3.11: Minimum entropy with PVT variations of the TRNG implementation without placement suggestions.

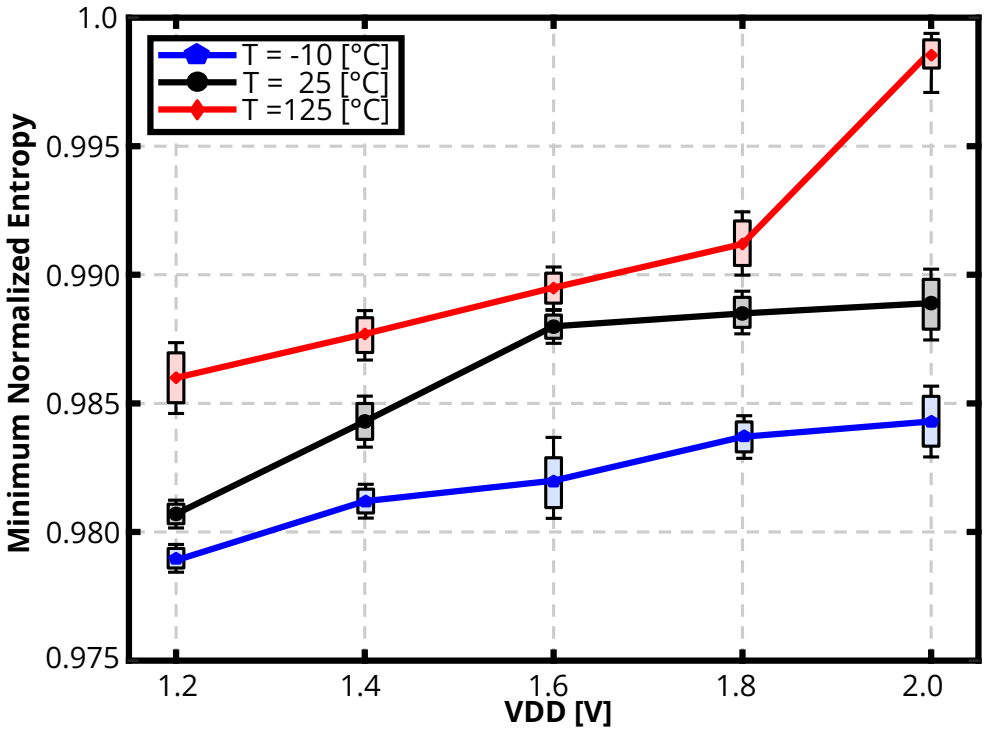


FIGURE 3.12: Minimum entropy with PVT variations of the TRNG implementation with the placement suggestions.

each point in the PVT variations to obtain the minimum entropy. The worst point of minimum entropy is expected due to the noise's dependency on the entropy source's physical phenomena.

TABLE 3.8: Minimum entropy comparison of ASIC implementation.

	Type of Data	Min. Entropy Normalized †	Physical Phenomena	PVT Variations
This Work	Non-IID	0.978	Frequency Collapse in MMRO	Yes
[46] 2016	IID	0.991	Metastability in Latches	Yes
[47] 2016	IID	0.992	Multiple Entropy Sources	Yes
[52] 2017	Non-IID	0.407	Metastability in Latches	Yes
[63] 2020	IID	0.883	Self-time in Chaotic system	No
[64] 2020	IID	0.995	Jitter in Analog-RO	Yes
[66] 2020	IID	0.996	Charge-Trapping in FinFET	Yes
[69] 2020	IID	0.996	Resistance variations in RRAM	No
[70] 2021	IID	0.993	Leakage in SRAM	Yes
[76] 2022	IID	0.997	Resistance variations in RRAM	Yes

† The ideal and maximum value is 1.

TABLE 3.9: NIST SP800-22 results with PVT variations.

Statistical test	PVT	CHIP # 1			CHIP #2			CHIP #3			CHIP #4		
	V[V]	1.2	1.8	2.0	1.2	1.8	2.0	1.2	1.8	2.0	1.2	1.8	2.0
	T[°C]	-10	25	125	-10	25	125	-10	25	125	-10	25	125
Frequency	P_{value}	0.3190	0.7597	0.8343	0.6993	0.5955	0.3345	0.2022	0.4280	0.4288	0.8051	0.5141	0.4011
	Rate	99	96	99	100	98	99	97	97	99	99	100	99
Block frequency	P_{value}	0.2368	0.4372	0.3838	0.6371	0.8513	0.5141	0.5544	0.1025	0.6155	0.1626	0.6579	0.4559
	Rate	99	100	99	100	99	100	97	99	100	99	99	100
Cumulative sums	P_{value}	0.6579	0.2896	0.4559	0.0805	0.3669	0.1372	0.0909	0.3955	0.1916	0.9942	0.6993	0.8558
	Rate	99	97	99	99	100	99	98	98	99	99	100	98
Cumulative sums	P_{value}	0.5194	0.6371	0.8343	0.2054	0.5749	0.4749	0.1087	0.5749	0.4190	0.2757	0.7254	0.3504
	Rate	99	96	99	100	97	99	98	99	98	98	100	98
Runs	P_{value}	0.9834	0.6579	0.2622	0.8977	0.0219	0.8676	0.7791	0.1296	0.4749	0.2022	0.6786	0.2526
	Rate	99	99	99	100	100	100	98	99	100	98	98	99
Longest run	P_{value}	0.5141	0.4372	0.1626	0.8676	0.9357	0.6971	0.7399	0.1372	0.6993	0.5141	0.5544	0.4011
	Rate	100	99	99	100	99	99	99	100	98	100	99	99
Rank	P_{value}	0.5749	0.4591	0.7399	0.6786	0.0269	0.3898	0.5341	0.2622	0.4559	0.8180	0.8831	0.7542
	Rate	99	98	100	99	100	100	97	99	100	99	98	100
FFT	P_{value}	0.7570	0.1025	0.3504	0.4749	0.3838	0.1815	0.2757	0.0288	0.1916	0.1455	0.2022	0.1757
	Rate	100	99	100	98	99	99	97	99	99	98	98	98
Non-overlap template	P_{value}	0.8977	0.8977	0.9716	0.7981	0.6371	0.9914	0.3838	0.5141	0.9240	0.7981	0.4372	0.4943
	Rate	99	98	100	99	99	99	99	99	99	99	100	100
Overlap template	P_{value}	0.3345	0.4372	0.3504	0.5955	0.6163	0.5544	0.2757	0.5341	0.4190	0.1916	0.7197	0.2519
	Rate	100	100	99	99	99	99	100	98	99	99	99	99
Approx. entropy	P_{value}	0.9882	0.9914	0.9992	0.9812	0.9944	0.9991	0.9878	0.9911	0.9994	0.9892	0.9914	0.9985
	Rate	100	99	100	100	100	100	99	100	100	99	100	99
Universal	P_{value}	0.7791	0.1372	0.7981	0.8165	0.8343	0.3190	0.4190	0.4578	0.3838	0.7597	0.4943	0.3838
	Rate	99	96	100	99	98	99	99	99	99	99	100	99
Random excursions	P_{value}	0.1815	0.3504	0.2248	0.2622	0.4749	0.5544	0.5749	0.4559	0.5944	0.2519	0.7399	0.5716
	Rate	99	99	99	99	98	99	99	99	99	99	98	100
Random excursions variant	P_{value}	0.9978	0.5341	0.5341	0.5341	0.2757	0.4943	0.4190	0.4190	0.3190	0.6394	0.6579	0.3144
	Rate	99	100	100	100	99	99	99	98	99	100	100	99
Serial	P_{value}	0.6163	0.1153	0.4559	0.8977	0.9357	0.7981	0.7791	0.7399	0.5749	0.2022	0.8558	0.2248
	Rate	100	100	99	99	100	99	99	99	100	99	100	99
Serial	P_{value}	0.2622	0.6786	0.4559	0.3190	0.7981	0.1087	0.1453	0.3669	0.1815	0.6126	0.8513	0.5341
	Rate	98	100	100	99	99	100	98	98	100	99	98	99
Linear complexity	P_{value}	0.9357	0.5544	0.6371	0.5141	0.2622	0.9435	0.5733	0.3041	0.9558	0.3190	0.4943	0.6371
	Rate	99	99	99	98	98	100	99	100	99	99	98	100

Table 3.8 compares the minimum entropy obtained with different physical phenomena in an ASIC implementation. The minimum entropy compared with the other implementations is the worst value with the PVT variation results. The frequency collapse phenomena report a 1.31% to 1.9% less minimum entropy normalized compared to some related implementations. However, these implementations are analyzed with an IID assumption, resulting

TABLE 3.10: Summary and Comparison Results of the TRNG in a ASIC implementation.

	Technology [nm]	Power [mW]	Bit rate [Mb/s]	Energy [pJ/bit]	Area [μm^2]	Normalized Area $^+$ [10^3 F^2]	Entropy Source
This work	180	0.56† - 1.88‡	7.3† - 9.2‡	77.2† - 204.3‡	16000 (25600*)	493.82	Multi-modal RO
[44] 2014	65	0.159	2.8	56.79	960	227.21	Multi-modal RO
	28	0.54	23.16	23.32	375	478.31	
[47] 2016	14	1.5	162.5	9.23	1008	5142.85	Multiple Entropy Sources
[62] 2019	180	0.34 - 0.42	1.6 - 3.7	92 - 264	8700	268.51	Multi-modal RO
[63] 2020	40	0.528	1600	0.33	270	168.75	Chaotic-Cellular Automata
[64] 2020	65	0.36	52	6.9	60000	14201.18	Stochastic Delta Sigma Modulator
[70] 2021	28	34.56	3.6	9.6 - 17.2	15400	19642.85	SRAM memory

† Entropy source with 16 stages. ‡ Entropy source with 2 stages.
* Including 3 additional entropy sources.
$^+$ Normalized Area = (area)/(minimum feature size of the process)^2

in a possible overestimation in the minimum entropy value.

Table 3.9 depicts the statistical test results with PVT variations. The P-values for the non-overlap template, random excursion and random excursion variant are the average of the P-values of all sub-tests. The significance level applied in the statistical test is $\alpha = 0.01$. The TRNG implemented passed all estimators in the statistical test provided by the NIST SP800-22.

Table 3.10 shows the summary and comparison results of the ASIC implementation. The frequency collapse presents a non-deterministic value in the time to collapse. Therefore, the bit rate and energy efficiency of the TRNG are presented in a range, reporting 7.3 until 9.2-Mb/s and 77.2 until 204.3-pJ/bit using the 16 and 2 delay stages, respectively. Finally, the TRNG based on frequency collapse reports a 493.82-10^3F^2 of area normalized.

3.4 Summary

A TRNG based on frequency collapse is presented in FPGA and ASIC implementation, proposing some suggestions in each implementation to improve the minimum entropy, respectively. In FPGA implementation, the suggestions described in section 3.2.1 guarantee the physical phenomena occurs due to the native mismatch in the primitives used in the FPGA implementation. The TRNG is evaluated in Xilinx Artix-7 XC7A100TCSG32 FPGA in nominal conditions. On the other hand, two ROHM-180nm chip was made with and without the placement suggestions described in section 3.2.2, demonstrating the improvement of the minimum entropy with PVT variations. In addition, the analytical model used to propose the suggestions is described in section 3.1.2. The NIST SP800-90B entropy test and NIST SP800-22 statistical test are applied in both implementations, passing all sub-tests.

Chapter 4

Proposed Non-Volatile Random-Access Memory (NVRAM) in Standard CMOS Technology

This chapter's primary goal is to discuss the NVRAM implementation, explaining the physical phenomena to implement in standard CMOS technology. Moreover, the results of the NVRAM implementation are reported. On the other hand, the NVRAM bit cell is exploited to obtain a PUF and TRNG implementation, obtaining a unified PUF-TRNG-NVRAM. The unified PUF-TRNG uses the metastability phenomena in the sense amplifier of the NVRAM bit cell. Next, the results of the PUF-TRNG are presented and compared with the related works. Finally, the summary of the chapter is described.

4.1 NVRAM in Standard CMOS Technology

4.1.1 Physical Phenomena NVRAM

Figure 4.1 shows the cross-section of an NVRAM in standard and non-standard CMOS technology. The physical phenomenon used to create a non-volatile memory is the Fowler-Nordheim (FN) tunneling electrons, applying high voltages into floating gates [88–90]. In non-standard CMOS technology, the NVRAM is fabricated using additional layers, applying the control gate methodology for tunneling the electrons into a Floating Gate (FG) [54, 71, 91]. A conventional control gate methodology in non-standard CMOS technology is shown in Figure 4.1.a). On the other hand, the NVRAM implementation in a standard CMOS technology without additional layers is shown in Figure 4.1.b). The implementation consists of a Single Floating Gate (SFG) constructed with PMOS transistors without a control gate methodology [21–26]. However, the SFG needs more transistors for the program and erase steps in comparison with the control gate methodology.

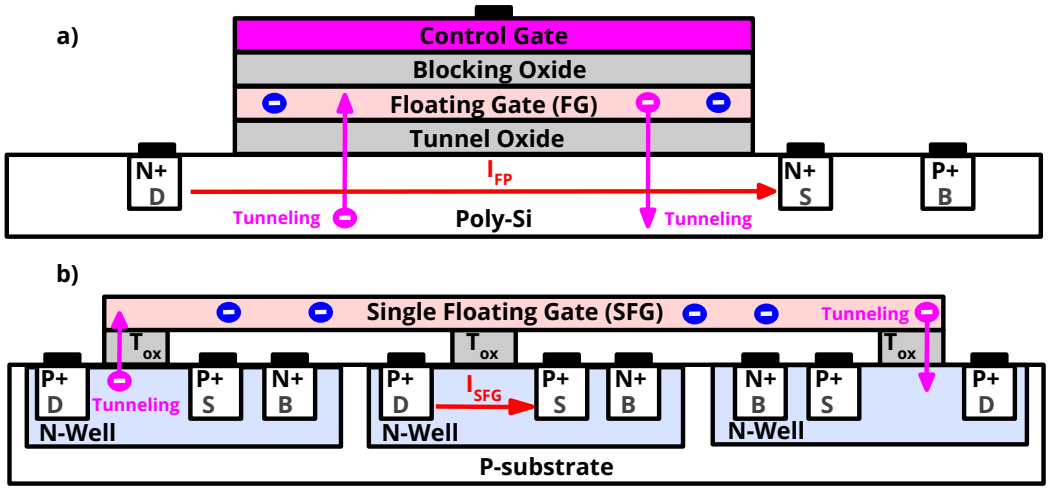


FIGURE 4.1: Cross-section of NVRAM with and without control gate methodology.

4.1.2 ASIC implementation

Figure 4.2 shows the schematic of the NVRAM bit cell with a sense amplifier for a measured purpose The bit cell implementation highlighted in gray is implemented with ten transistors (M_{1-10}) to construct a differential response. The fundamental of the NVRAM bit cell without a control gate methodology

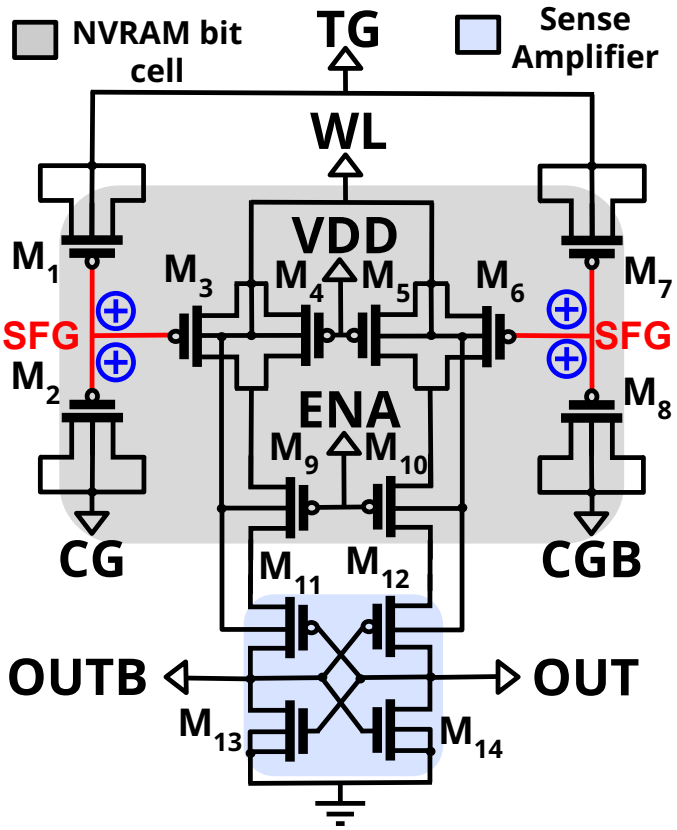


FIGURE 4.2: Schematic differential NVRAM bit cell.

consists of a coupling capacitor (C_c), tunneling capacitor (C_t), and read transistor (T_R) connected in the SFG. The differential bit cell uses six transistors to implement the C_c ($M_{2,8}$), C_t ($M_{1,4}$) and T_R ($M_{3,6}$), respectively. Besides, transistors ($M_{9,10}$) are used to enable the read step. On the other hand, a sense amplifier based on cross-coupled inverted (M_{11-14}) is implemented to read the value of the differential NVRAM bit cell. The FN phenomena are controlled using the CG, CGB, TG and WL signals in the bit cell implementation.

Figure 4.3 illustrates an NVRAM bit cell implementation in a ROHM 0.18μm standard CMOS technology. The implementation consists of a bit cell of an NVRAM based on SFG with the respective sense amplifier circuit. The differential bit cell occupies 476 μm^2 with 14.69 ($10^2 F^2$) area normalized. The bit cell with the sense amplifier is implemented using 3.3-V transistors to support the high-voltage necessary in the FN phenomena. The transistor used for C_t and T_R is implemented with 360nm of with and length, respectively. However, the C_c has a 2.6μm width and 360nm length dimensions.

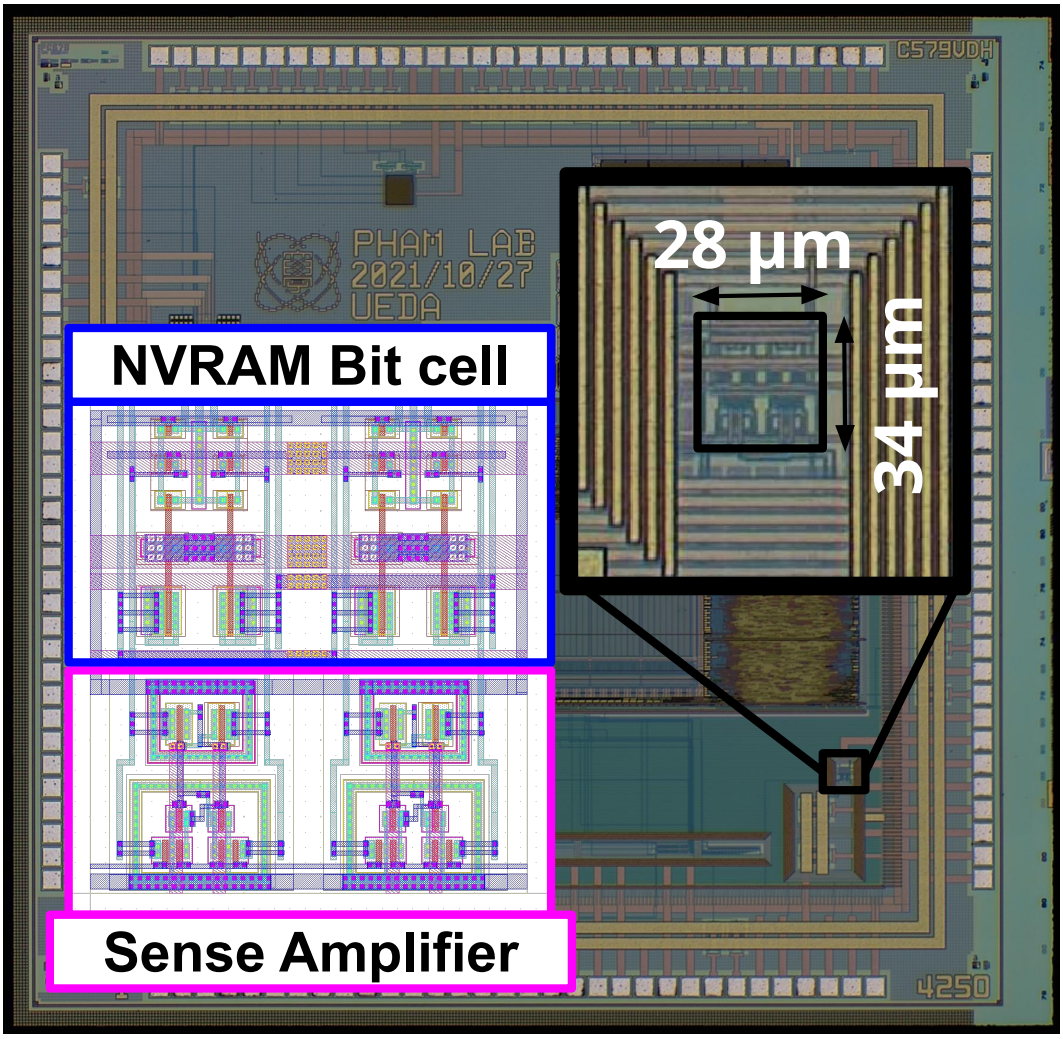


FIGURE 4.3: ASIC implementation of an NVRAM bit cell in ROHM 0.18μm CMOS technology.

TABLE 4.1: Operational voltages of the NVRAM bit cell.

Mode	VDD [V]	ENA [V]	CG [V]	CGB [V]	WL [V]	TG [V]
Read	3.3	0	0	0	3.3	0
Program	3.3	3.3	8.5	8.5	0	0
Erase	3.3	3.3	0	0	0	8.5

Table 4.1 depicts the operational voltages required for the NVRAM function. The principal step functions in the NVRAM bit cell are *Program*, *Erase* and *Read*. First, the *Program* step needs the high-voltage (8.5-V) for the FN phenomena to introduce the charges into the SFG, applying the high-voltage in CG or CGB depending on the value to store. Second, the *Erase* step also needs the high-voltage in the TG pin to remove the charges stored in the SFG. In the *Program* and *Erase* steps, a 3.3-V is applied in the pins ENA and VDD, respectively. Finally, the *Read* step applies a 3.3-V in the WL pin, enabling the current used in the sense amplifier. However, this step needs a 0-V applied in the ENA pin in compared to the *Program* and *Erase* steps.

4.1.3 Measured Results

Table 4.2 depicts the operational results of the differential NVRAM bit cell implemented. The NVRAM is compared with other implementations in different standard CMOS technologies. The minimum program/erase voltage shows a linear relationship with the tunnel oxide of the technology used in the implementation. In the ROHM 180nm implementation, the length of the tunnel oxide is 70-Å requiring 8.5-V to program/erase steps. On the other hand, the high voltage in the *Program* and *Erase* must be applied for 15-ms, ensuring the correct function. In addition, the time necessary in the *Read* step is 5-ns.

TABLE 4.2: Operational results of the NVRAM implemented

	This work	[24] 2015	[22] 2000	[21] 1994
Technology [nm]	180	130	250	800
Tunnel Oxide [Å]	70	62	100	150
Programming Time [ms]	15 @8.5-V	2 @10-V	100 @7-V	100 @9-V
Erase Time [ms]	15 @8.5-V	1 @10-V	1000 @10-V	100 @17-V
Read Access Time [ns]	5 @3.3-V	—	—	—
Min. Program/Erase Voltage [V]	8.5	8	10	17

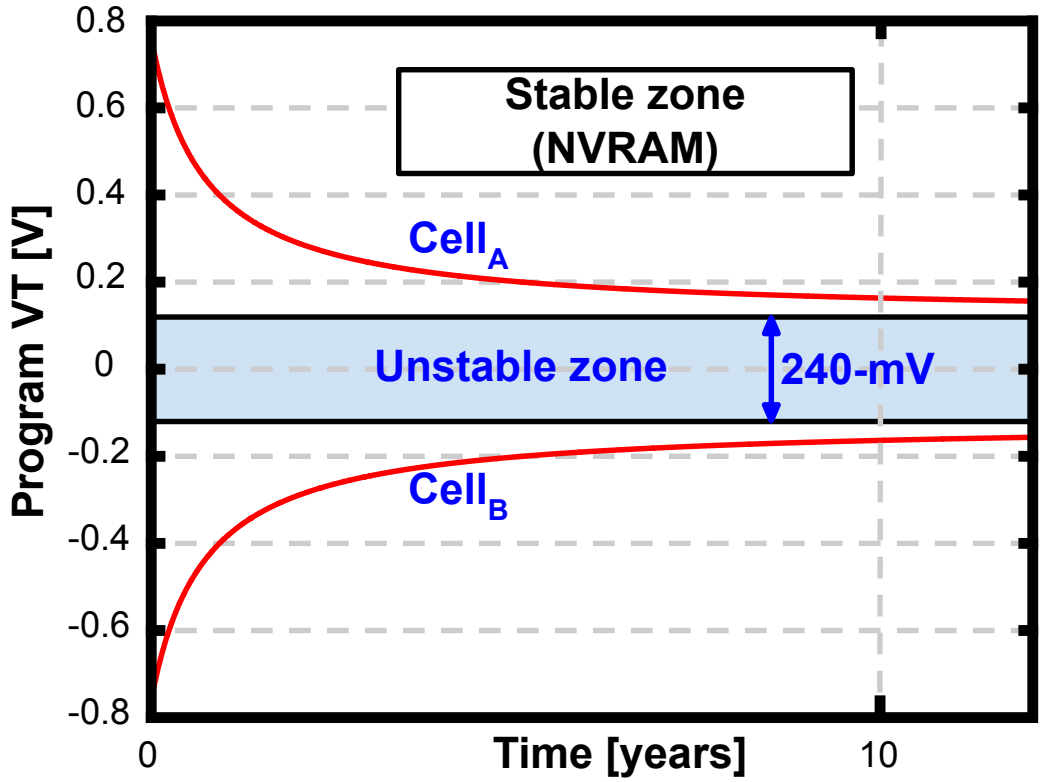


FIGURE 4.4: Retention characteristics of the programmed bit cell.

Figure 4.4 shows the retention characteristics of the differential NVRAM bit cell. The minimum margin to obtain a stable response is 120-mV in the NVRAM bit cell's Threshold Voltage (VT). The retention margin is obtained by applying a high temperature (150°C) for a long time in the *Read* step [65].

4.2 Unified NVRAM-TRNG-PUF

The differential NVRAM implementation described in section 4.1.3 presents an unstable zone response. The unstable response is caused by the low VT in the bit cell, generating metastability in the sense amplifier. Typically, the metastability effects are undesirable for the reliability of the NVRAM implementation. Therefore, the control gate methodology can be solved due to the high VT obtained in the program step. The SFG implementation can not obtain a high VT compared to the control gate implementation methodology. However, the undesirable effect in the sense amplifier's cross-coupled inverters can be exploited to obtain a PUF-TRNG based on metastability. Conventionally, the PUF-TRNG based on metastability needs a calibration step to obtain a good implementation.

Figure 4.5 compares the typical and proposed implementation of the TRNG-PUF based on metastability. The typical implementations implement a delay controller to calibrate the entropy source in the PUF or TRNG application [43,

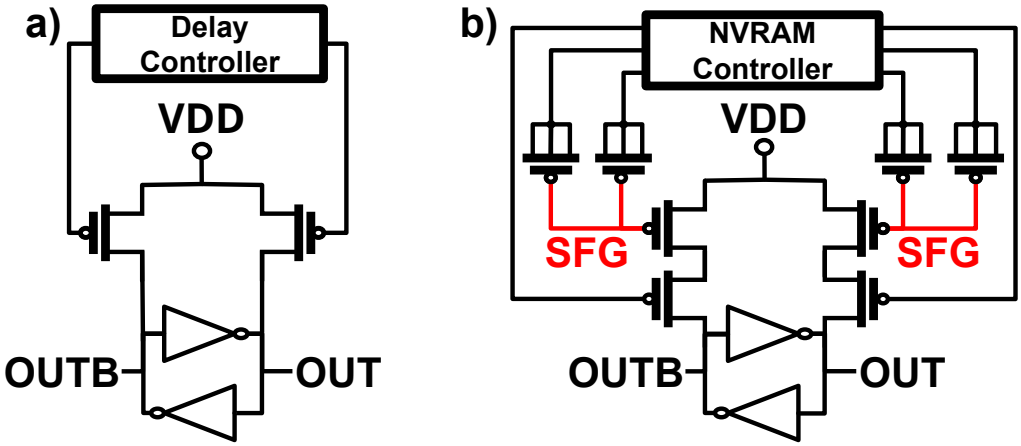


FIGURE 4.5: Architecture of typical (a) and proposed (b) TRNG-PUF based on metastability.

45]. The proposed unification uses the charges stored in the SFG to calibrate the unstable response in the sense amplifier. The mismatch variations can be reduced/increased to improve the TRNG or PUF implementation results.

4.2.1 Physical Phenomena of Unified TRNG-PUF

The physical phenomena of the unified TRNG-PUF are based on the metastability response in the sense amplifier. The differential NVRAM bit cell described in section 4.1 uses the FN phenomena to store some charges into the SFG. However, when the number of charges is similar, the sense amplifier can be introduced in an unstable mode. The two response modes of the sense amplifier are shown in Figure 4.6. The *Stable mode* is generated when the currents I_a and I_b are different enough to be sensed. On the other hand, the *Unstable mode* occurs when the currents are similar to differentiated. The *Unstable mode* can generate a metastable response in the sense amplifier.

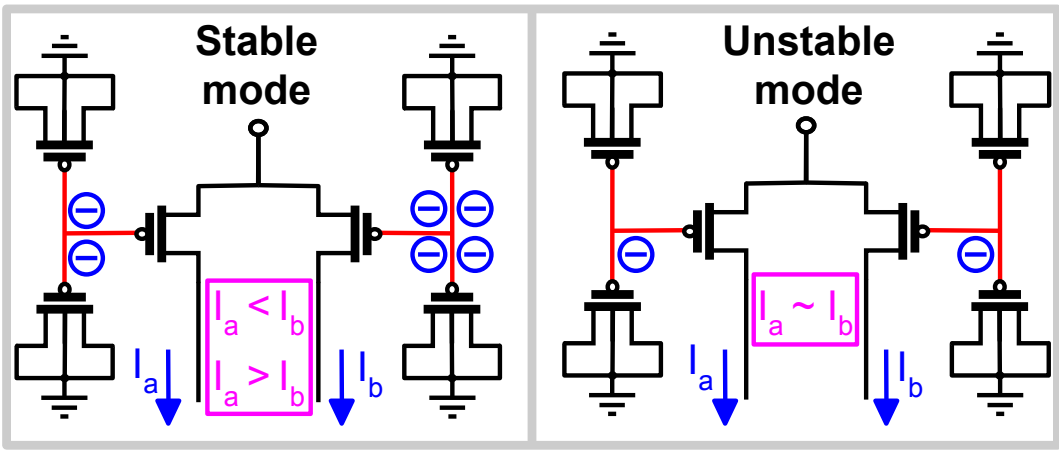


FIGURE 4.6: Stable and unstable modes in the sense amplifier caused by the NVRAM.

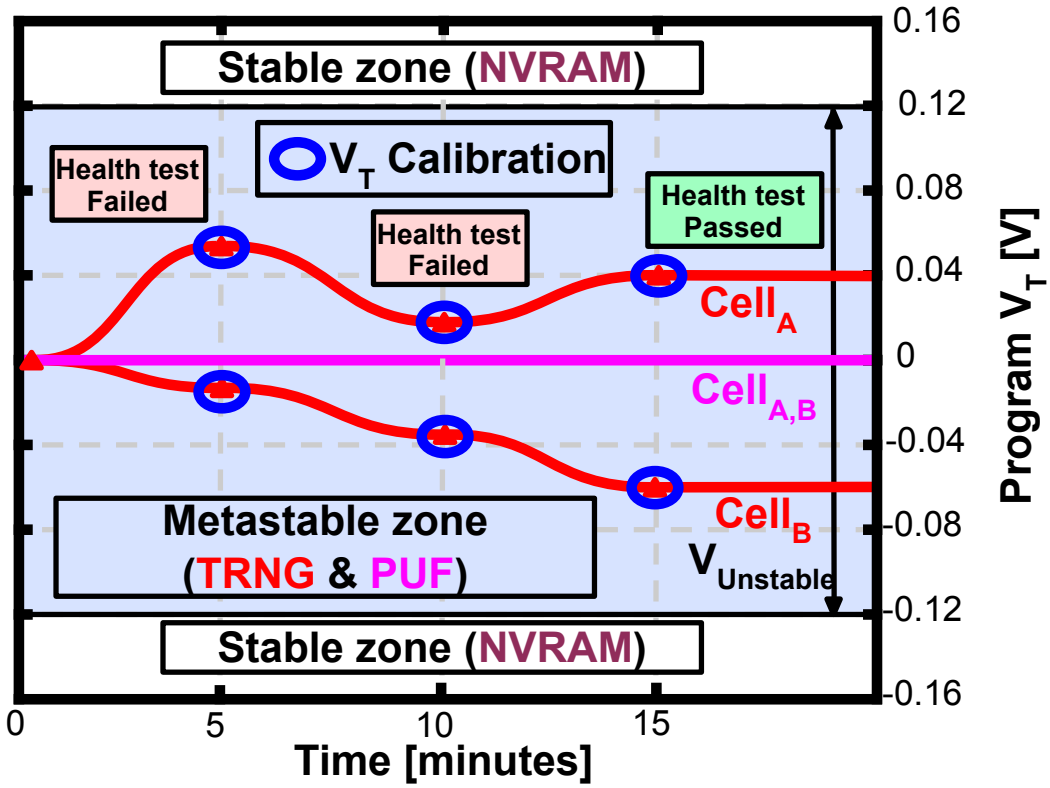


FIGURE 4.7: Metastable zone in the unified NVRAM-PUF-TRNG implementation.

Fig. 4.7 depicts programmed cell characteristics for implementing the PUF and TRNG primitives in the NVRAM implementation. The sense amplifier in the NVRAM implementation exhibits 120-mV of V_T to obtain a stable response. When V_T is under 120-mV, the sense amplifier starts to respond with metastability. Conversely, after an erase step in the differential NVRAM bit cell, the charges stored in the SFG are removed, obtaining approximately $V_T = 0$. In this case, the sense amplifier presents a biased metastability response obtained from a PUF response depending on the process variations of the sense amplifier and the NVRAM bit cell. However, the TRNG response must calibrate the V_T in each branch of the differential NVRAM bit cell to obtain an unbiased metastability response. The calibration process applied is based on the health test provided by NIST SP800-90B [85].

4.2.2 Analytical Model of Unified TRNG-PUF

The TRNG-PUF application is based on the metastability in the sense amplifier generated for the unstable mode in the NVRAM bit cell. The sense amplifier is a conventional circuit presented in a memory implementation used to sense the bit line, generating the binary response. Typically, the sense amplifier response is designed to mitigate the metastability response in the latches, such as an SRAM implementation using a loop gain model [92]. The latch response is characterized using the Static Noise Margin (SNM) in the response, as shown in Figure 4.8. The output voltage ($V_L = f(V_R)$ and $V_R = g(V_L)$) of

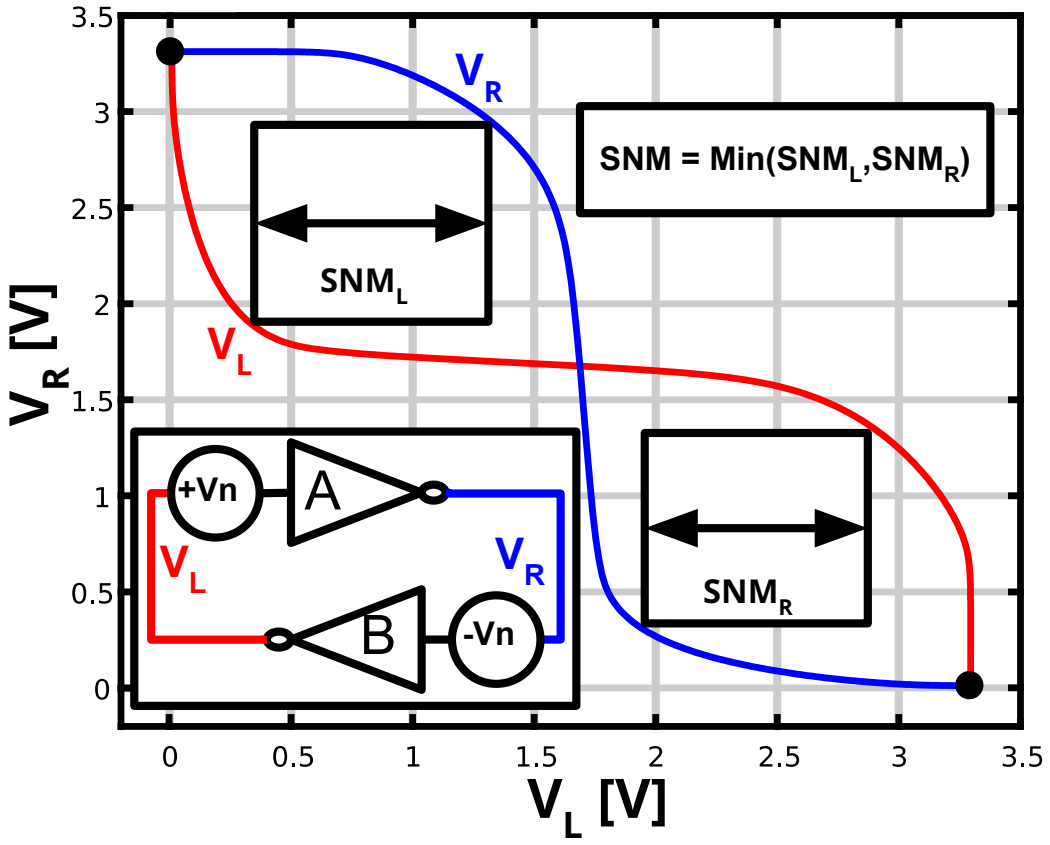


FIGURE 4.8: Metastability response of sense amplifier based on latches.

each inverter is modeled using a function correlated with each voltage node. The functions are identical in the ideal scenario for the symmetric implementation. Nevertheless, the process variations cause an unbalanced response in the metastable response of the sense amplifier. The unbalanced response is generated with the current value variations necessary to overcome each node's potential state (V_{LF} and V_{RF}). The differences in each voltage node depend on the mismatch ($\delta_{m(L)}$ and $\delta_{m(R)}$) and the noise ($\delta_{n(L)}$ and $\delta_{n(R)}$), as shown in (4.1).

$$V_L = g(V_R) + \delta_{m(R)} + \delta_{n(R)} \qquad V_R = f(V_L) + \delta_{m(L)} + \delta_{n(L)} \tag{4.1}$$

The loop gain model approximates $V_R = (a * V_L) + b$, using the a and b coefficients. When the coefficients are obtained with V_{LF} and V_{RF} (4.2), respectively.

$$a = \frac{f(V_{LF}) - V_{RF}}{V_{LF} - g(V_{RF})} \qquad b = \frac{V_{LF}V_{RF} - f(V_{LF})g(V_{RF})}{V_{LF} - g(V_{RF})} \tag{4.2}$$

Concluding, the metastable response in the sense amplifier based on latches is denoted by $[(a * V_L) + b] > [f(V_L) - \delta_{m(R)} - \delta_{n(R)}]$. The latches' output depends on the relation between the voltage nodes, including the mismatch

TABLE 4.3: NIST SP800-90B results with IID assumption.

NIST	PVT	CHIP # 1			CHIP #2			CHIP #3		
800-90B	V[V]	3.0	3.3	3.6	3.0	3.3	3.6	3.0	3.3	3.6
IID	T[°C]	-10	25	125	-10	25	125	-10	25	125
Min.	P_{value}	0.5047	0.5031	0.5022	0.5047	0.5029	0.5024	0.5049	0.5031	0.5024
Entropy	Est.	0.9865	0.9910	0.9935	0.9864	0.9914	0.9931	0.9859	0.9910	0.9930

and noise presented in the latch implementation. The NVRAM unification can exploit the noise and mismatch variations in the TRNG or PUF implementation. The PUF implementation is obtained by removing the charges in the SFG with the *Erase* step. On the other hand, the TRNG implementation needs a calibration step to eliminate the mismatch variations in the latches, using the health test described in NIST SP800-90B [82, 85].

4.2.3 Unified NVRAM-TRNG-PUF Measured Results

The NVRAM implementation shown in Figure 4.3 reproduces a TRNG implementation, exploiting the metastability response in the latches using the sense amplifier. Typically, the IID assumption in TRNGs based on metastability in latches is non-IDD. However, the calibration process applied in the NVRAM bit cell mitigates the mismatch variations, obtaining an IDD assumption in the entropy source. Table 4.3 reports the NIST SP800-90B entropy test with an IID assumption. The entropy test is applied with PVT

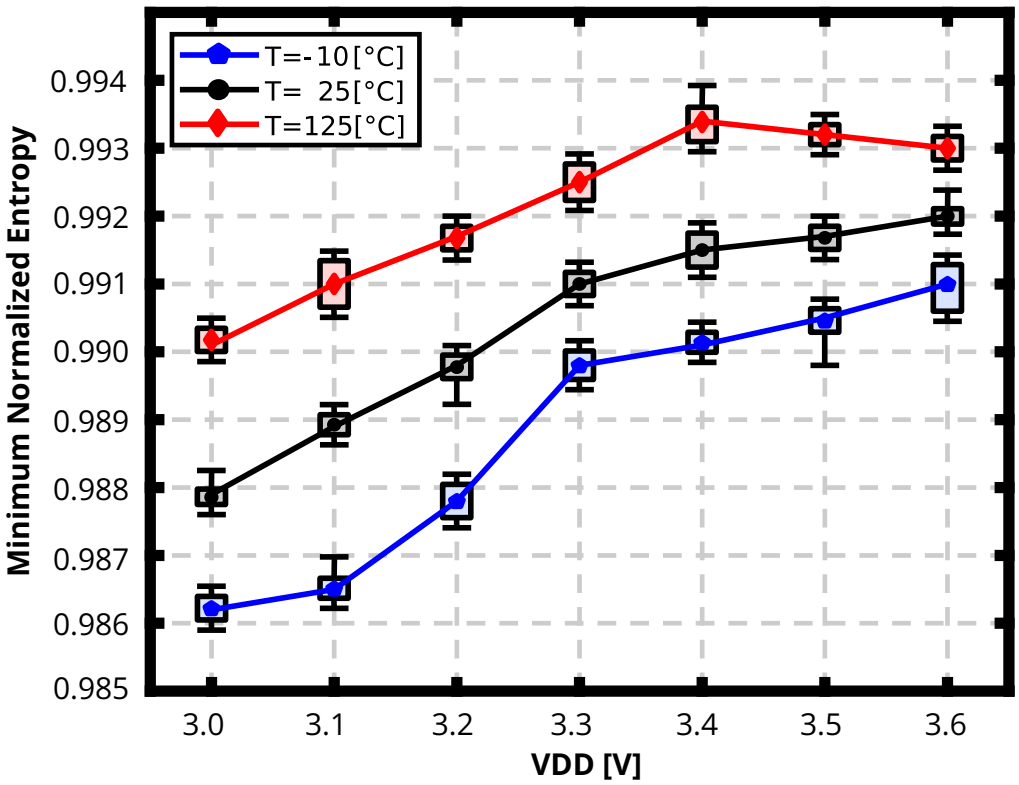


FIGURE 4.9: Unified NVRAM-TRNG minimum entropy with PVT variations.

TABLE 4.4: Minimum entropy comparison of Unified NVRAM-TRNG.

	Type of Data	Min. Entropy Normalized †	Physical Phenomena	PVT Variations
This work	IID	0.985	Metastability in NVRAM	Yes
[46] 2016	IID	0.991	Metastability in Latches	Yes
[52] 2017	Non-IID	0.407	Metastability in Latches	Yes
[63] 2020	IID	0.887	Self-time in Chaotic system	No
[66] 2020	IID	0.996	Charge-Trapping in FinFET	Yes
[70] 2021	IID	0.993	Leakage in SRAM	Yes
[76] 2022	IID	0.997	Resistance Variations in RRAM	Yes
[80] 2022	Non-IID	0.978	Frequency Collapse in Multi-Modal RO	Yes

† The ideal and maximum value is 1.

variations in three chips. The minimum entropy in an IID assumption is obtained only with the Most Common Value (MCV) estimator compared to the non-IID assumption.

Figure 4.9 illustrates the minimum entropy results of the unified NVRAM-TRNG implementation with PVT variations. The entropy source presents the minimum entropy normalized of 0.9859 with a 3.0-V and -10$^{\circ}C$ of operational conditions, respectively. The sense amplifier shows less minimum entropy in the low voltage and temperature due to the noise dependence in the physical phenomena.

Table 4.4 depicts the minimum entropy comparison results of the unified NVRAM-TRNG implementation. The entropy results are presented with an IID assumption in the worst case with PVT variations. The implementation performs well compared to other implementations with different physical phenomena. Therefore, the unified implementation based on metastability in a sense amplifier demonstrates the reliability of the TRNG application, using the SFG methodology in an NVRAM bit cell implemented in standard CMOS technology with a calibration step.

Table 4.5 reports the NIST SP800-22 statistical test results in the unified NVRAM-TRNG with PVT variations. The statistical test is applied with an $\alpha = 0.01$ level of confidence. The data is recollected after the calibration process in the TRNG mode, using 5-MB of random sequences in each test. The P-values of the non-overlap template, random excursion, and random excursion variant are the averages of the P-value sub-test.

On the other hand, the unified NVRAM-PUF needs an array implementation

TABLE 4.5: NIST SP800-22 results with PVT variations.

Statistical test	PVT	CHIP # 1			CHIP #2			CHIP #3		
	V[V]	3.0	3.3	3.6	3.0	3.3	3.6	3.0	3.3	3.6
	T[°C]	-10	25	125	-10	25	125	-10	25	125
Frequency	P_{value}	0.9642	0.5141	0.6579	0.1453	0.9978	0.4011	0.8977	0.8831	0.3041
	Rate	99	100	99	99	99	99	100	100	99
Block frequency	P_{value}	0.4011	0.4372	0.7399	0.4943	0.9114	0.6993	0.9093	0.7399	0.1626
	Rate	97	99	99	100	99	99	99	100	100
Cumulative sums	P_{value}	0.7791	0.8831	0.5341	0.1815	0.5141	0.4943	0.7197	0.5749	0.2757
	Rate	97	100	99	99	99	98	100	100	100
Cumulative sums	P_{value}	0.4372	0.8513	0.7597	0.5544	0.5341	0.2881	0.4559	0.3345	0.4871
	Rate	100	100	100	100	99	98	100	99	99
Runs	P_{value}	0.8343	0.5749	0.5341	0.1626	0.4011	0.8879	0.1025	0.1918	0.9834
	Rate	99	97	100	99	99	100	99	100	99
Longest run	P_{value}	0.9240	0.2022	0.3041	0.7597	0.5544	0.4559	0.3041	0.1223	0.2368
	Rate	100	97	99	98	99	99	99	100	99
Rank	P_{value}	0.2022	0.9535	0.7597	0.2248	0.3669	0.8879	0.6163	0.5544	0.2694
	Rate	100	99	100	100	98	98	98	100	99
FFT	P_{value}	0.2248	0.5141	0.3504	0.5955	0.4559	0.5544	0.5544	0.5955	0.6371
	Rate	100	98	100	99	100	100	99	97	99
Non-overlap template	P_{value}	0.1916	0.4372	0.4749	0.9240	0.2368	0.8977	0.4190	0.4871	0.5955
	Rate	99	99	99	100	99	100	99	98	98
Overlap template	P_{value}	0.4372	0.9093	0.1025	0.3669	0.1916	0.2022	0.1223	0.9780	0.8676
	Rate	100	98	99	99	99	99	99	100	99
Approx. entropy	P_{value}	0.9988	0.9935	0.9924	0.9931	0.9878	0.9995	0.9558	0.9878	0.9977
	Rate	99	99	100	98	100	99	100	100	100
Universal	P_{value}	0.7791	0.7579	0.9657	0.6163	0.1025	0.5749	0.5341	0.7596	0.5194
	Rate	97	99	99	99	99	99	99	100	99
Random excursions	P_{value}	0.2133	0.1025	0.5544	0.5341	0.6786	0.6163	0.8676	0.4943	0.1718
	Rate	97	98	100	100	99	99	99	99	100
Random excursions variant	P_{value}	0.4749	0.6371	0.1687	0.7399	0.1815	0.4372	0.3190	0.2519	0.4559
	Rate	98	100	99	100	100	100	99	100	100
Serial	P_{value}	0.5341	0.8676	0.3504	0.1815	0.5749	0.9114	0.5544	0.7981	0.5544
	Rate	99	100	99	99	99	100	99	100	99
Serial	P_{value}	0.2248	0.7597	0.1223	0.1453	0.8676	0.4567	0.2248	0.1916	0.5749
	Rate	98	99	99	99	99	100	98	99	99
Linear complexity	P_{value}	0.1916	0.4559	0.9357	0.7197	0.7694	0.4749	0.8165	0.9093	0.5955
	Rate	99	100	100	100	99	99	98	98	98

∗ The P-values for the non-overlap template, random excursion and random excursion variant are the average of the P-values of all sub-test.

to test the PUF application. The NVRAM array is implemented in ROHM $0.18\mu m$ standard CMOS technology. The array implementation consists of 128-bit cells (8 columns and 16 rows), a high-to-low (H2L) and low-to-high (L2H) level shifter, and sense amplifiers. The NVRAM array is implemented in a RISC-V environment for measuring purposes. The array implementation is shown in Figure 4.10.

The PUF implementation is measured using the uniformity, uniqueness, and reliability described in [93–95]. Figure 4.11 illustrates the intra- and inter-chip Hamming distances in the NVRAM-PUF implementation. The implementation exhibits an average intra-die Hamming distance of 0.4% with 0.21% standard variation. Besides, the inter-chip Hamming distance reports 48.12% with a a 1.83% standard variation on average. In nominal conditions, the intra- and inter-die Hamming distances are measured in three chips.

Table 4.6 shows the quality results between the unified NVRAM-PUF with

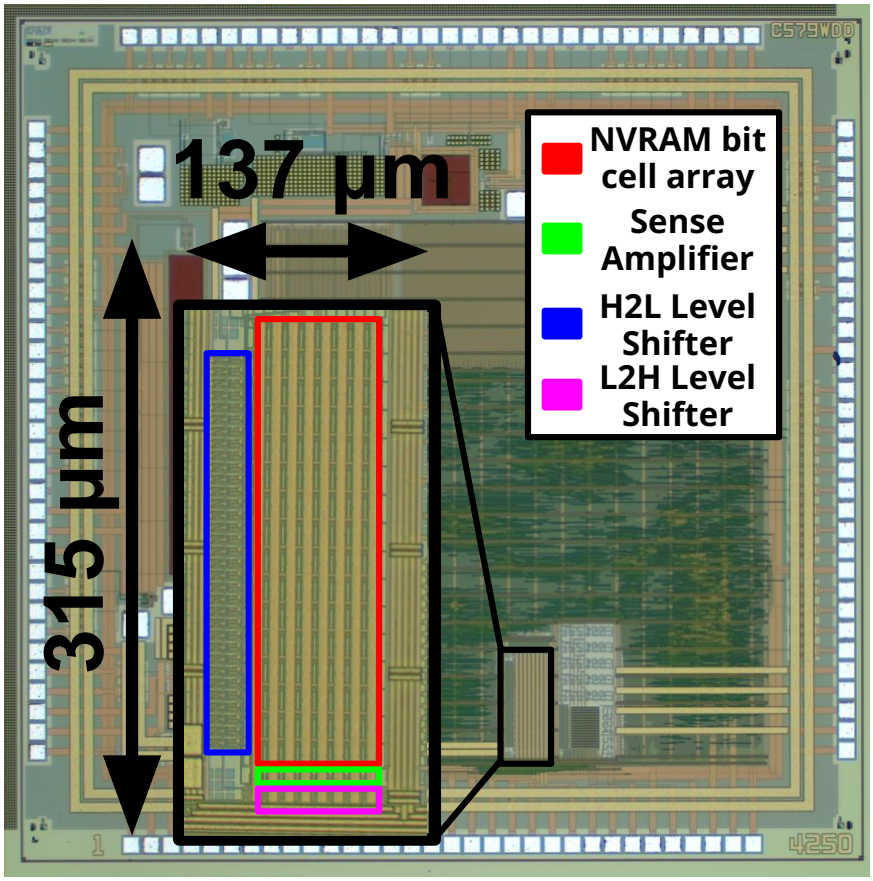


FIGURE 4.10: ASIC implementation of an NVRAM array in ROHM 0.18μm standard CMOS technology.

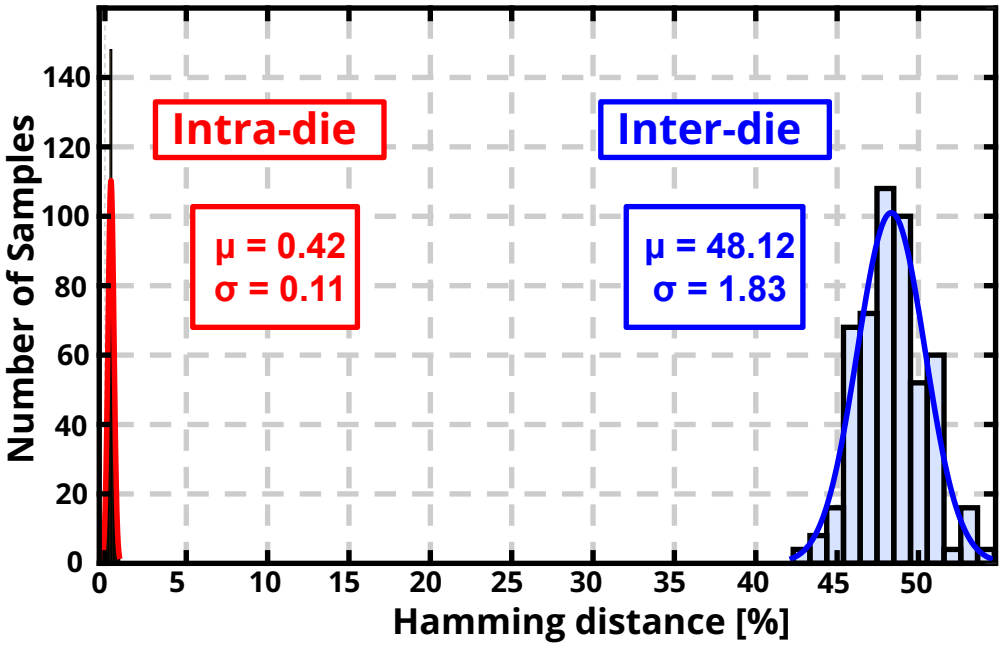


FIGURE 4.11: Inter- and Intra-die Hamming distance of unified NVRAM-PUF in nominal conditions.

TABLE 4.6: Quality results of the Unified NVRAM-PUF implementation.

	This work	[20] 2022	[77] 2022
Uniformity [%]†	49.85	49.80	53.40
Uniqueness [%]†	48.12	50.30	48.89
Reliability [%]‡	99.58	98.40	96.90
Topology	NVRAM memory	SRAM memory	SRAM memory

† The ideal value is 50%. ‡ The ideal value is 100%.

other implementations based on unified memory implementations. The ideal values for uniformity, uniqueness and reliability are 50%, 50% and 100%, respectively. The uniformity is measured using the average Hamming weight result in the PUF response. The unified implementation exhibits 49.85% of uniformity, 48.12% of uniqueness, and 99.58% of reliability. The unified NVRAM-PUF demonstrates enough quality results to use the PUF in a secure application.

Table 4.7 compiles the results of the unified TRNG-PUF in the NVRAM implementation. The Unified NVRAM-TRNG-PUF is compared with other unified implementations in SRAM memory and 3D Threshold Switching (TS). The TRNG function reports 49.5μW of power consumption and a 50-Mbps bit rate, showing a 0.99-pJ/bit energy efficiency. In addition, the NVRAM array implementation with the analog circuits occupies a 47950-μm^2 area with 191-μm^2 per NVRAM bit cell. On the other hand, the PUF implementations show a 7116-F^2 of area normalized per bit. The unified implementation reports a competitive performance compared to another unified function based on SRAM memory and TS.

TABLE 4.7: Summary and comparison of the NVRAM-TRNG-PUF implementation

	Technology Implemented [nm]	**TRNG function**			**Area [μm^2]**	**PUF function**	**Unified Functions**
		Power [μW]	**Bit rate [Mb/s]**	**Energy [pJ/bit]**		**PUF Cell Area/Bit [F^2]**$^+$	
This work	180	49.5	50	0.99	47950 (191†)	7116	NVRAM TRNG PUF
[70] 2021	28	21.12 - 61.92	2.2 - 3.6	9.6 - 17.2	15400	1125	SRAM TRNG PUF
[72] 2021	40	—	—	0.75	0.8192	2	TS TRNG PUF

† Only the NVRAM bit cell. ‡
$^+$ Normalized Area = (area)/(minimum feature size of the process)^2

4.3 Summary

A Unified NVRAM-TRNG-PUF is implemented in standard CMOS technology. The NVRAM function uses the FN phenomena to reproduce the non-volatile memory using an SFG. The ASIC implementation in the ROHM-180nm chip was made to test the differential NVRAM bit cell is explained in section 4.1. The measured results denote 8.5-V for the program and erase steps in the NVRAM function. The NVRAM implementation can be unified with a TRNG-PUF function, exploiting the metastability in the sense amplifier described in section 4.2. The TRNG application is generated using a calibration step to mitigate the mismatch variations in the entropy source. The entropy source is calibrated using the charges stored in the SFG. The Unified NVRAM-TRNG passed the NIST SP800-22 statistical and SP800-90B entropy, respectively. The entropy source based on metastability in the latches of the sense amplifier denotes a 0.985 minimum entropy normalized with PVT variations. On the other hand, a 128-bit NVRAM array was implemented in a ROHM-180nm chip. The PUF application is generated by removing all charges inside the SFG, obtaining a metastable response in the sense amplifier depending only on the process variations. The unified NVRAM-PUF exhibits 49.85% uniformity, 48.12% uniqueness, and 99.85% reliability in nominal conditions. The unified NVRAM-TRNG-PUF demonstrates exemplary performance in secure systems, exploiting the metastability in the sense amplifier of non-volatile memory.

This page intentionally left blank.

Chapter 5

Proposed One Time Program (OTP) memory in Standard CMOS Technology

The primary goal of this chapter is to describe the OTP implementation, explaining the physical phenomena of implementing it into standard CMOS technology. In addition, the characterization of the OTP based on anti-fuse bit cells is described. The physical phenomenon of the OTP anti-fuse bit cell is shown to obtain an implementation in standard CMOS technology. The high voltage driver used to program the bit cell is described with the restrictions to implement in standard CMOS technology. Next, the results of the OTP implementation are presented. Moreover, the unification of OTP-PUF is reported using the metastability in a sense amplifier. Then, the results of the unified OTP-PUF are presented and compared with the related works. Finally, the summary of the chapter is described.

5.1 OTP memory in Standard CMOS Technology

An OTP application is a non-volatile memory to store vital information in the system. However, the OTP memory presents the characteristic of being programmed only once, preventing the corruption of the information typically represented in the keys of the system. Typically, the OTP memory is implemented in CMOS technology with the fuse topology using additional layers [49,56,96]. The fuse methodology increases a wire's resistance and applies a high current to burn the wire. On the other hand, the Anti-Fuse (AF) methodology tries to program a bit differently than the fuse methodology. In the AF methodology, high voltage breaks the gate, reducing the resistance between the gate and the transistor body [74,97,98]. Conventionally, the AF OTP memory is implemented in CMOS technology with additional masks and processes to increase the reliability of the implementation. Nonetheless, the AF methodology can be implemented in a standard CMOS technology without additional layers compared with the fuse methodology [27–30].

5.1.1 Physical Phenomena

The AF OTP implementation is based on the oxide breakdown phenomena in the gate of the transistors. The oxide breakdown phenomenon is modeled using the Time-Dependent Dielectric Breakdown (TDDB) model [99–101] derived from Direct Tunneling (DT) and Fowler-Nordheim (FN) phenomena. The TDDB model defines the Time-to-Breakdown (T_{BD}) of the oxide used in the transistor construction to reproduce the OTP application. The T_{BD} can be calculated using (5.1).

$$T_{BD} = \frac{qT_{OX}N_{BD}(A_{OX},T_{OX})}{J(V_G,T_{OX})\zeta(V_G)} \tag{5.1}$$

Where $J(V_G, T_{OX})$ is the tunneling current generated by the DT and FN phenomenons based on the assumption of linear dependence of the defect generation fluency [102]. The tunneling current depends on the critical breakdown defect density (N_{BD}), the defect generation efficiency ($\zeta(V_G)$) and the oxide thickness (T_{OX}), respectively. The N_{BD} is taken to be voltage independence but only depends on the area and Weibull slope (β) within the percolation mode [103]. The modeled function of the N_{BD} is shown in (5.2).

$$N_{DB} = \frac{t_{OX}}{a_o^3} exp\left\{-\frac{1}{\beta(T_{OX})} Ln\left(\frac{A_{OX}}{a_o^2}\right)\right\} \tag{5.2}$$

Here, the a_o represents the defect size after assuming the linear time evolution of the defect density during the stress applied. The β thickness dependence is shown in (5.3), where the T_{INIT} is the inter-facial thickness of the implementation. Finally, the oxide breakdown phenomena denote a relationship between the T_{OX} of the transistor and the V_G with the respective T_{BD} necessary to program the AF OTP bit cell.

$$\beta = (T_{INIT} + T_{OX})/a_o \tag{5.3}$$

5.1.2 AF OTP bit cell ASIC implementation

Figure 5.1 illustrates the AF bit cell implementation in a $0.18\mu m$ standard CMOS technology. Two-bit AF bit cells are implemented to characterize the voltage necessary to obtain the oxide breakdown of the OTP implementation. The AF bit cells are implemented with different dimensions, using 360nm and 720nm in each bit cell, where the AF bit cell has the same weight and length dimensions in the PMOS transistor implementation. In addition, Electro-Static Discharge (ESD) protection is implemented to reduce the risk of gate breakdown by antenna problems.

Figure 5.2 shows the characterization of the AF OTP bit cell. The AF bit cell is characterized using 10-nA and 100-nA of current limitations. The program procedure with current limitations shows undesirable effects on the

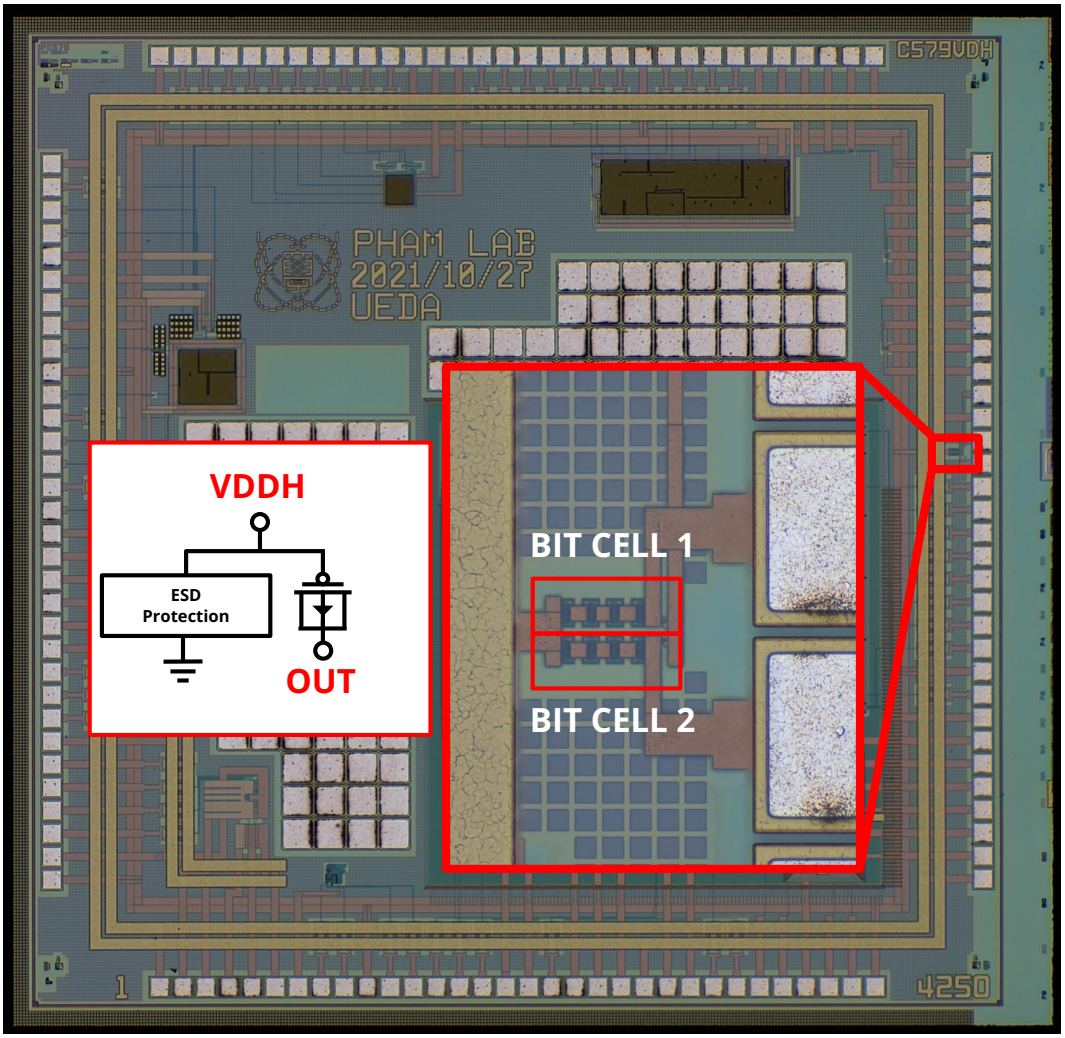


FIGURE 5.1: PMOS AF micrograph in a 0.18μm standard CMOS technology.

OTP memory in terms of security, causing a soft breakdown or no breakdown in the oxide of the AF bit cell. On the other hand, the characterization without a current limitation shows a properly programmed bit cell. The AF bit cell denotes a 5.4-V in the gate to start the defect generation in the oxide. Besides, the V_G necessary to obtain the hard breakdown in the bit cell is 6-V. The characterization results are presented with 720nm bit cell dimensions. However, the 360nm bit cell needs a higher V_G to obtain the oxide breakdown. In this way, the implementation follows the relationship presented in (5.2). In addition, a constant T_{BD} is used in the characterization. The AF bit cell uses the thickness transistor to endure the high voltage applied in the program step. Finally, the current operation of the AF bit cell after hard breakdown shows in the range of units of micro-Amperes.

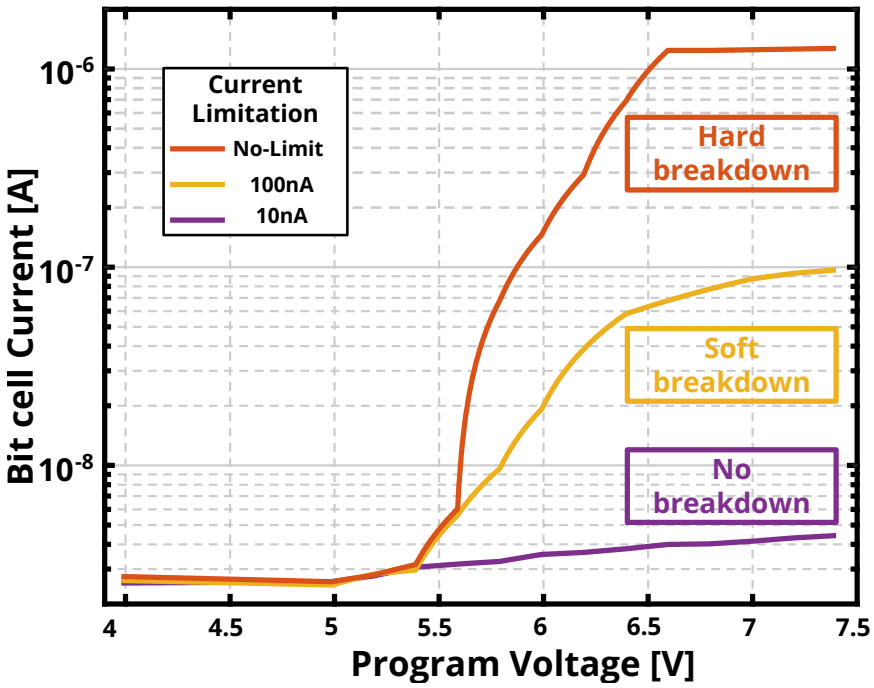


FIGURE 5.2: PMOS AF characterization with current limitations.

5.1.3 High-Voltage Driver

The OTP application needs to manage the high voltage used in the program step to break the oxide of the bit cell. Moreover, the high voltage applied time

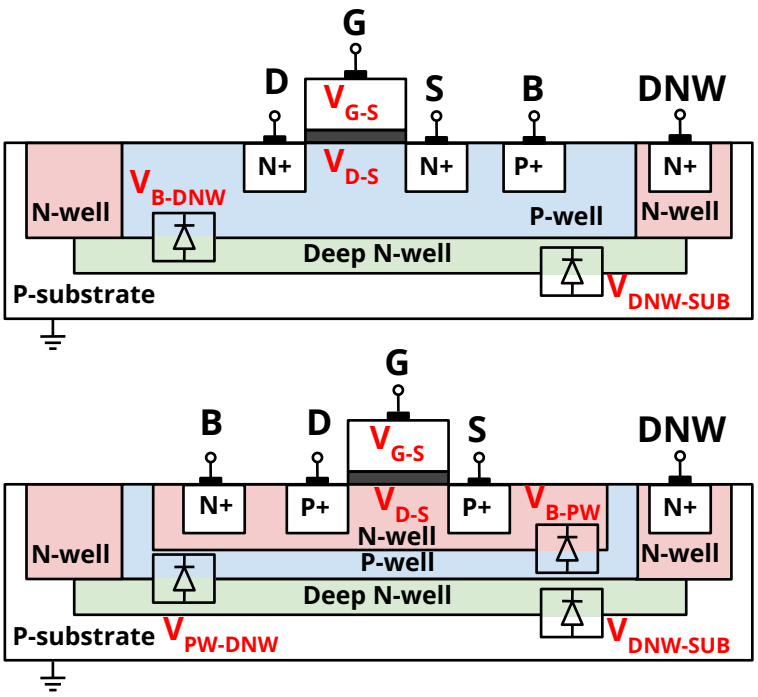


FIGURE 5.3: CMOS devices cross section in a Standard technology.

needs to be appropriately managed to reduce the over-programming stress in the AF bit cell. Therefore, a high-voltage driver circuit is necessary to manage the program step of the memory. However, high-voltage driver implementation presents challenges due to employing standard CMOS technology. The Absolute Maximum Voltage Voltage Ratio (AMRV) in standard CMOS technology restricts high-voltage applications. The AMRV determines the voltage tolerance in each type of transistor. Typically, the AMRV is 1.3 times the conventional voltage in low and high-voltage transistors. If the voltage applied in V_{GS}, V_{BS}, V_{DS}, V_{B-DNW}, and $V_{DNW-SUB}$ are higher than the AMRV, the transistors can permanently lose their functionality. The only way to support a high-voltage application in standard CMOS technology is to use a transistor in a deep N-well. Figure 5.3 illustrates the physical construction of the transistor using a deep N-well. The voltage limit is defined as the breakdown voltage of the inverted diodes generated into the different layers. For example, the diode generates between the deep N-well and P-well. The uses of deep N-well can support high voltages for different applications [23, 28].

Figure 5.4 illustrates the high-voltage driver architecture for standard CMOS

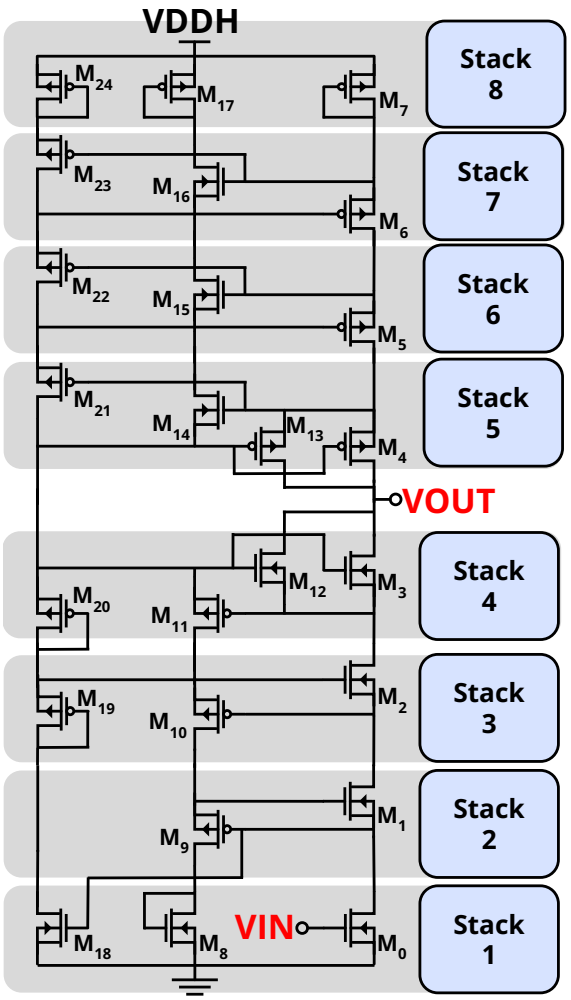


FIGURE 5.4: Schematic of the high-voltage driver implementation.

technology. The architecture is based on the stacked driver topology presented in [104]. However, the architecture is modified to increase the driver's output current to prevent the soft breakdown in the AF bit cell. In addition, the transistors follow the restrictions generated by the Absolute Maximum Rating Voltage (AMRV) described in this section. The implementation can be driver high-voltage using eight stacks in the topology. In addition, the trigger signal VIN is in the digital domain eliminating the requirements of level shifter circuits and reducing the area overhead of the implementation. The 8-stack high-voltage driver topology uses 24-thickness transistors to resist the high voltage implemented in standard CMOS technology.

5.1.4 Measurement Results and Comparison

Figure 5.5 illustrates the ASIC implementation of the high-voltage driver in a $0.18\mu m$ standard CMOS technology. The implementation is designed to manage enough high-voltage to program an OTP bit cell occupying a $7314\mu m^2$ area. The deep N-well in the layout supports the high-voltage operation for the AF bit cell programming. In addition, the stacked construction of the implementation prevents the use of additional deep N-well, reducing the area.

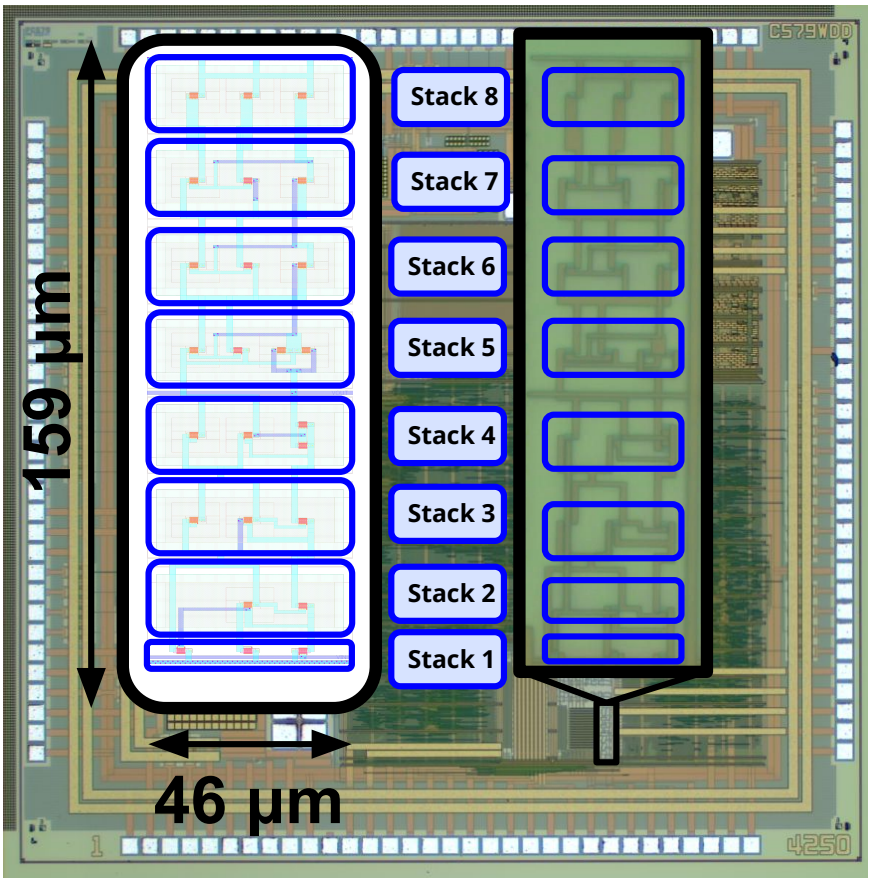


FIGURE 5.5: ASIC implementation of the high-voltage driver in ROHM $0.18\mu m$ standard CMOS technology.

TABLE 5.1: Performance summary and comparison.

	Technology [nm]	Max. HV [V]	Area [μm^2]	Area Normalized [10^6 F^2]	Auxiliary circuits +	Min. Load	Power consumption	Rise and Fall Time
This work‡	180	7.8	7314	0.22	None	720-kΩ	188.3-μW @7V	16.2-ns – 0.54-ns @7V
[105]† 2006	1500	9	1613333	0.71	3-LS 2-VR	1-kΩ	N.R	N.R
[106]‡ 2015	65	6	148113	35.06	2-LS 8-VR	12-Ω	132.2-mW @6V	1.22-ns – 1.15-ns @6V
[107]† 2016	65	5.5	187000	44.26	2-LS	12-Ω	295-mW @5.5V	3.1-ns – 2.5-ns @5.5V
[104]‡ 2020	180	8.7	5654	0.17	None	1-GΩ	78.18-μW @6V	13.1-μs – N.R @6V

+ LS and VR denotes Level Shifter and Voltage Reference, respectively. † Measured results. ‡ Simulated results.

Table 5.1 shows the comparison results with the high-voltage driver in state-of-the-art. The 8-stacks high-voltage driver implemented no need for an additional level shifter or voltage references for the functionality, reducing the area consumption compared to the conventional architectures. In addition, the minimum load of the implementation increased by 1388% compared to the related works based on the stacked high-voltage drivers with a 32% area overhead.

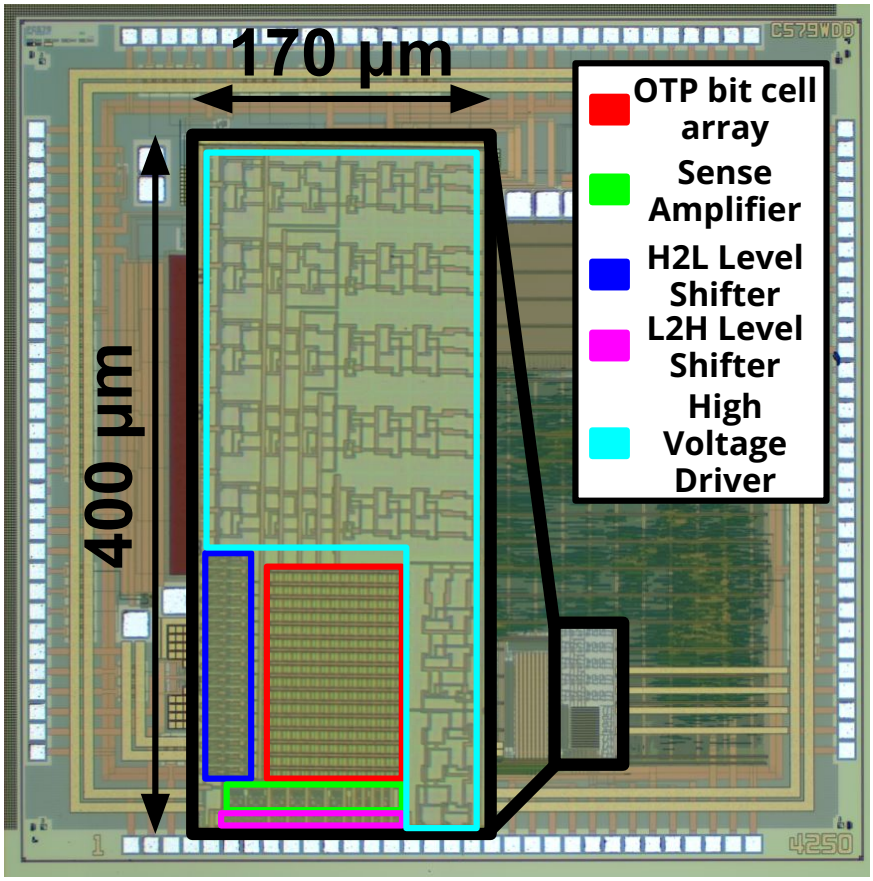


FIGURE 5.6: ASIC implementation of the 128-bit OTP array in ROHM 0.18μm standard CMOS technology.

TABLE 5.2: Operational results of the OTP memory implemented.

	Technology [nm]	Program Voltage [V]	Area/bit [μm^2]	Read Power [μW]
This work	180	6.7	27.22^{+}	6.32
[27] 2003	180 250	6.5 9.5	$21^{\dagger}$ —	36.7 —
[28] 2013	180	6.6	4.88^{+}	17

$^{+}$ 2T bit cell implementation. † 3T bit cell implementation.

Figure 5.6 illustrates the ASIC implementation of a 128-bit OTP array in standard 0.18μm standard CMOS technology. The implementation consists of an 8x16 array with the respective sense amplifier. The OTP array with the respective analog circuits is integrated on a peripheral into a microcontroller based on a RISC-V processor for measuring purposes. In addition, the high voltage drivers and level shifters (L2H and H2L) are implemented to connect with the RISC-V microcontroller. The trigger signal of the high-voltage drivers is managed with the RISC-V microcontroller in the digital power domain. The OTP implementation occupies 68000μm^2, representing 16.17% in the OTP array, 2% in the sense amplifiers, 1.45% in the L2H level shifter, 6.08% in the H2L level shifter and 74.3% in the high-voltage drivers, respectively.

Table 5.2 reports the operational results of the OTP memory in a 0.18μm standard CMOS technology. The OTP memory needs a 6.7-V program voltage using a two-transistor (2T) bit cell. The area occupied by one 2T bit cell is 27.22-μm^2. In addition, the power consumption of the 2T bit cell after the program is 6.32-μW. The other OTP implementation in a 0.18μm denotes a similar program voltage due to the equitable bit cell dimensions.

5.2 Unified OTP-PUF

The OTP memory implementation can be unified with a PUF application using the metastable response of the sense amplifier described in section 4.2.2. However, the OTP bit cell is not a differential implementation compared to the NVRAM bit cell described in chapter 4. Therefore, the OTP array implemented in section 5.1 has an additional column to reproduce the differential behavior necessary in sense amplifiers, as shown in 5.7. Cell A or B is programmed in the OTP memory application, increasing the current to store the bit. On the other hand, the PUF application needs to program the two cells to reproduce the metastable response in the sense amplifier described in section 4.2.2. However, the PUF unification in the OTP memory also exploits the variations in the oxide break process compared to the NVRAM implementation, increasing the robustness of the PUF application.

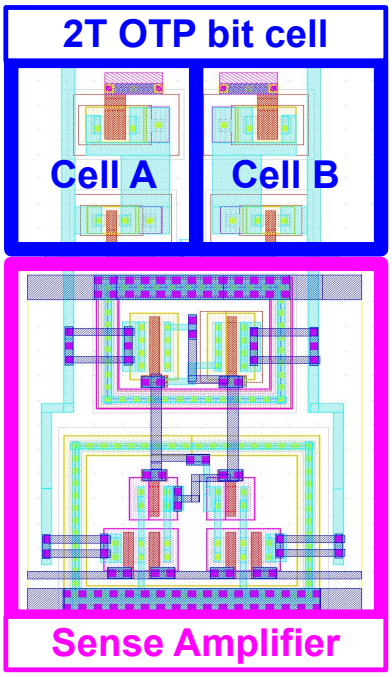
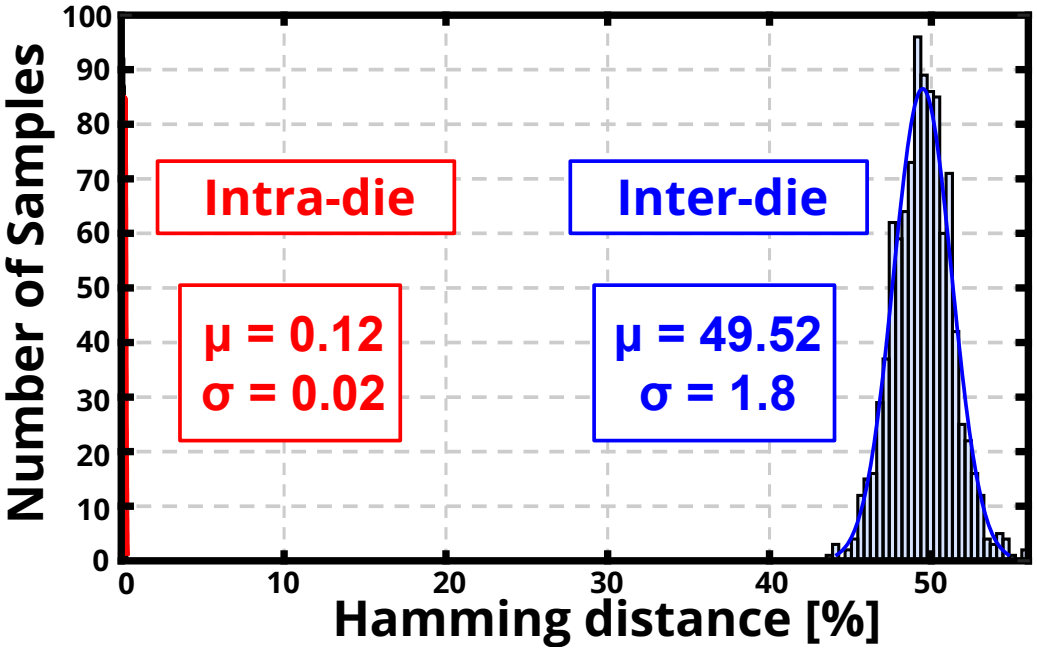


FIGURE 5.7: Layout of the basic 2 OTP bit cell with the sense amplifier.

5.2.1 Results and Comparison

Figure 5.8 illustrates the inter- and intra-Hamming distance of the unified OTP-PUf in nominal conditions. The unified OTP-PUF exhibits 0.12% of



]

FIGURE 5.8: Inter- and Intra-die Hamming distance of unified OTP-PUF in nominal conditions.

TABLE 5.3: Quality results of the Unified OTP-PUF implementation.

	This work	[20] 2022	[77] 2022
Uniformity [%]†	49.02	49.80	53.40
Uniqueness [%]†	49.52	50.30	48.89
Reliability [%]‡	99.88	98.40	96.90
Topology	OTP memory	SRAM memory	SRAM memory

† The ideal value is 50%. ‡ The ideal value is 100%.

intra-die with 0.02% standard variation. Besides, the implementation shows a 49.52% of inter-die with 1.8% standard variation. The Hamming distance results are reported to measure 1000 challenges, respectively. The challenges in the unified OTP-PUF are applied by changing the row used in each bit line response.

Table 5.3 reports the quality results of the unified OTP-PUF implementation. The implementations exhibit 49.02% uniformity, 49.52 % uniqueness, and 99.88% reliability, respectively. The results of the implementations are measured in three chips to obtain the different figures of merit in the quality of the OTP-PUF unification.

Table 5.4 compares the OTP-PUF results with other unified implementations. The OTP array with the analog circuits occupies 68000-μm^2, while the two-bit cell with the sense amplifier occupies 71-μm^2. Besides, the PUF implementations occupy 2992-F per bit of normalized area. The area normalized is reported only with the area occupied by the OTP array and the sense amplifiers. The unified OTP-PUF is not implemented with the special technology rules for memories, obtaining a higher area than related works.

TABLE 5.4: Summary and comparison of the OTP-PUF implementation

	Technology Implemented [nm]	**Area [μm^2]**	**PUF function** **PUF Cell Area/Bit [F^2]**$^+$	**Unified Functions**
This work	180	68000 (71†)	2992	OTP PUF
[70] 2021	28	15400	1125††	SRAM TRNG PUF
[72] 2021	40	0.8192	2‡	TS TRNG PUF

† Only two OTP bit cell.
†† Considering total SRAM macro area, including peripherals for SRAM and PUF.
‡ Only 3D-bit cell. $^+$ Normalized Area = (area)/(minimum feature size of the process)^2

5.3 Summary

A unified OTP-PUF is implemented in standard CMOS technology. The OTP function uses the oxide breakdown phenomena to reproduce the non-volatile memory. An ASIC implementation in a ROHM-180nm chip was made to characterize the oxide breakdown phenomena explained in section 5.1.1. The measured results denote a 6.7-V to break the oxide, obtaining a programmed bit cell. The bit cell consists of two transistors, one for reproducing the oxide breakdown and the other for control and protection purposes. The OTP implementation can be unified with a PUF function using process variations in the sense amplifier. The process variation generates a biased metastability response, reproducing the PUF application. An OTP array with the sense amplifier, high-voltage driver, H2L and L2H level shifter is implemented in ROHM-180nm, occupying a 68000-μm^2 into a RISC-V microcontroller for measuring purposes. The high-voltage driver is implemented using an 8-stack topology without additional analog circuits, occupying a 7314-μm^2. The driver can manage enough voltage to reproduce an oxide breakdown, managing with a signal in the digital voltage domain. The PUF application exhibits 49.02% of uniformity, 49.52% of uniqueness, and 99.88% of reliability in nominal conditions, respectively. The implementation denotes a 2992-F per bit area normalized. Finally, the unified OTP-PUF demonstrates the possibility to generated a random key using the PUF application storing the value in non-volatile memory with the OTP application and generating a new approach for secure systems.

This page intentionally left blank.

Chapter 6

Proposed ChaCha20-Poly1305 Authenticated Encryption with Additional Data (AEAD)

The primary goal of this chapter is to describe the AEAD construction using the ChaCha20 and Poly1305 primitives. The algorithm and hardware implementation of the ChaCha20 and Poly1305 primitives are described with the respective results in FPGA and ASIC implementations. In addition, a unified ChaCha20-PUF is implemented, exploiting the metastability in latches, generating a new approach in the generation of the One-Time Key (OTK) used in the AEAD implementation. The latches used in the PUF implementation are part of the ChaCha20 architecture to obtain a low-area overhead. Next, the AEAD construction compatible with Transport Layer Security (TLS) version 1.3 is related to the ChaCha20 and Poly1305 primitives. Finally, the results and comparison of the AEAD implemented in FPGA and ASIC are reported.

6.1 ChaCha20 primitive

The ChaCha20 cipher is based on the Salsa20 algorithm [108]. Initially, the algorithm generates an Initial Matrix (I_M) using a 256-bit key, a 32-bit initial counter, a 96-bit nonce, and four constants defined in the documentation [109]. The I_M has 512-bit, represented as a 4x4 matrix, where each matrix entry is interpreted as a 32-bit unsigned integer. Also, the organization of I_M is in little-endian form. The illustration of the matrix is as follows (6.1).

$$I_M = \begin{bmatrix} 61707865 & 3320646e & 79622d32 & 6b206574 \\ Key_0 & Key_1 & Key_2 & Key_3 \\ Key_4 & Key_5 & Key_6 & Key_7 \\ Counter & Nonce_0 & Nonce_1 & Nonce_2 \end{bmatrix} \tag{6.1}$$

Algorithm 1 shows the ChaCha20 cipher suite. Initially, an Operation Matrix (O_M) is initialized with the values of I_M. The ChaCha20 algorithm applies ten double-rounds to the internal state O_M, where each double-round consists of 8 applications of the quarter-round function. The rounds finalize when all

Algorithm 1 ChaCha20 cipher suite algorithm

Require: $K \in (0,1)^{256}$,$N \in (0,1)^{96}$,$C \in (0,1)^{32}$, $PT \in (0,1)^*$
Ensure: $CT = \text{ChaCha20}(K, N, C, PT)$
1: $I_M \leftarrow \text{Init}(K, N, C)$ ▷ The Initial matrix is organized as (6.1).
2: **for** $x \leftarrow 0$ **to** $([P/512] - 1)$ **do**
3: $O_M \leftarrow I_M$
4: **for** $y \leftarrow 0$ **to** 9 **do**
5: $O_M[0,4,8,12] \leftarrow \text{QR}(O_M[0,4,8,12])$
6: $O_M[1,5,9,13] \leftarrow \text{QR}(O_M[1,5,9,13])$
7: $O_M[2,6,10,14] \leftarrow \text{QR}(O_M[2,6,10,14])$
8: $O_M[3,7,11,15] \leftarrow \text{QR}(O_M[3,7,11,15])$
9: $O_M[0,5,10,15] \leftarrow \text{QR}(O_M[0,5,10,15])$
10: $O_M[1,6,11,12] \leftarrow \text{QR}(O_M[1,6,11,12])$
11: $O_M[2,7,8,13] \leftarrow \text{QR}(O_M[2,7,8,13])$
12: $O_M[3,4,9,14] \leftarrow \text{QR}(O_M[3,4,9,14])$
13: **end for**
14: $S \leftarrow Serialize(O_M + I_M)$
15: **for** $z \leftarrow 0$ **to** 511 **do**
16: $CT[512x + z] \leftarrow PT[512x + z] \oplus S[z]$
17: **end for**
18: $I_M[12] \leftarrow I_M[12] + 1$ ▷ Update the value of the counter in the Initial Matrix.
19: **end for**
20: **return** CT

states are updated in O_M. The result of the QR operation is saved in the same position as the state take of the input. Finally, the keystream is obtained by adding the I_M and O_M. In another way, the decryption process is an identical encryption operation but uses the same keystream and cipher stream. Additionally, the counter value is incremented by one per block of plaintext processed to obtain the next I_M.

The column and diagonal rounds consist of QR function $(A, B, C, D) = \text{QR}(a, b, c, d)$ acts on the state as follows (6.2). The addition denoted in the QR algorithm is a carry-less addition on a 32-bit word.

$$\left\{\begin{array}{ll} x = a + b; & y = (d \oplus x) \lll 16; \\ w = x + y; & z = (b \oplus w) \lll 12; \\ A = x + f; & D = (y \oplus A) \lll 8; \\ C = w + D; & B = (z \oplus C) \lll 7; \end{array}\right\} \tag{6.2}$$

Fig. 6.1 shows the implementation of the ChaCha20 primitive. The ChaCha20 implementation is divided into two parts. First, a *Block Function* highlighted in blue takes a 256 bits key, 96 bits nonce, and 32 bits counter to generate an I_M in the *Initial regs*. The states of the I_M are organized in little-endian form. After, the *Operation regs* obtain the initial states of the *Initial regs*. All

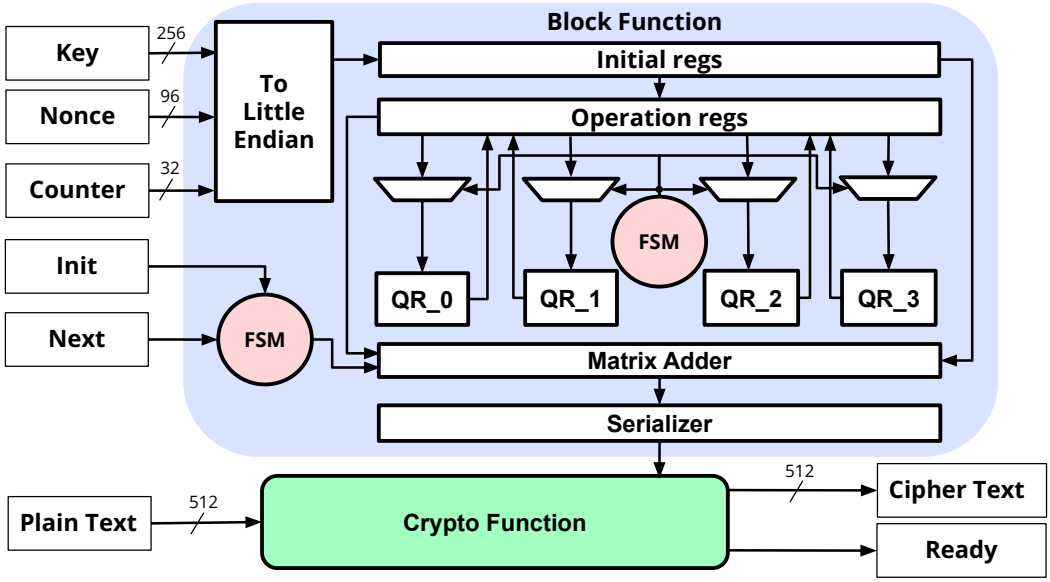


FIGURE 6.1: Architecture of the ChaCha20 primitive.

matrices have 16 states, and each state is a 32-bit register. A Finite State Machine (FSM) highlighted in red runs the 20 rounds using the QR modules. The ChaCha20 primitive can have 1, 2, and 4 Quarter Round operations to paralleling the round. However, the ChaCha20 algorithm presents the impossibility of paralleling between column and diagonal rounds. In this way, the maximum number of Quarter Round operations is four. The FSM controls the input of the Quarter Round operators in each column and diagonal round, respectively. Each round depends only on the result of the previous rounds in O_M. In addition, the *Init* and *Next* signals indicate the start of the process and the next 512-bits to processing, respectively. After 20 rounds, the *Matrix Adder* obtains the Final Matrix (F_M), adding the *Initial regs* and *Operation regs*. Second, a *Crypto Function* highlighted in green takes the F_M generated in the *Block function*. The 512-bit of *Cipher Text* is obtained by the XOR operation between F_M and the *Plain Text* . When the Crypto-Function is finished, a *Valid* signal is triggered.

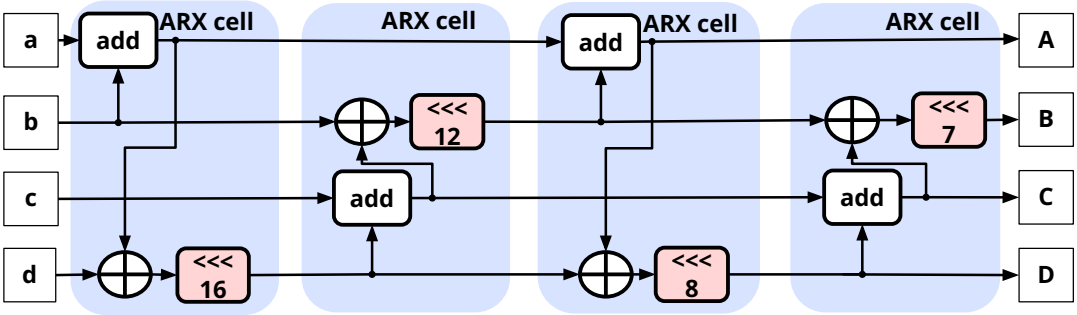


FIGURE 6.2: Quarter Round operation architecture.

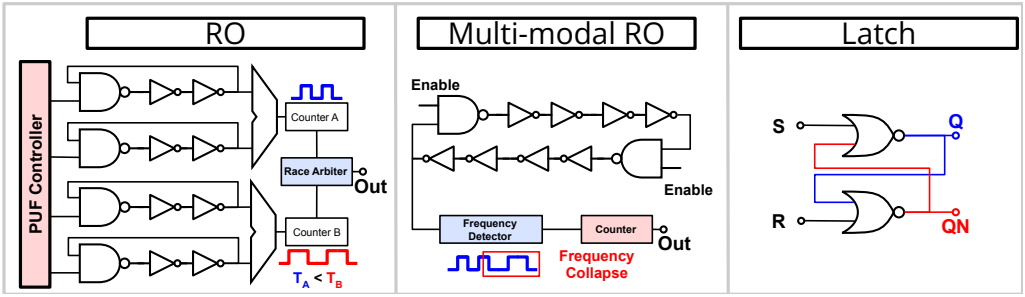


FIGURE 6.3: State of the art summary in PUF based on ring oscillators.

6.1.1 Unified ChaCha20-PUF

Figure 6.3 illustrates the typical physical phenomena that generate static entropy for a PUF application. Commonly, static entropy is obtained by the process variation of the implementation, which causes alterations in the frequency response in ring oscillators (ROs) [67, 110] and the time of frequency collapse in multi-modal RO [73, 78]. However, external circuits are necessary for collecting the static entropy. On the other hand, the dynamic entropy caused by metastability can affect the typical response of the latch, generating a random number [43, 46, 52]. Nevertheless, the process variation can generate enough static entropy in latches.

Figure 6.4 shows the response scenarios in a metastable latch. First, the latch presents the same probability of obtaining a one and zero in the response when the latch is balanced. The balanced latches are used for a TRNG application. However, the process variations impose problems when balancing the latch, which requires a calibration step [43, 46, 80]. On the other hand, the unbalanced latches produce a stable and unique response. Therefore, the stable and unique response in the unbalanced latches can be used for PUF applications, exploiting the process variations in the latch. In addition, the latches are used as a memory, independent of the balance. Therefore, the

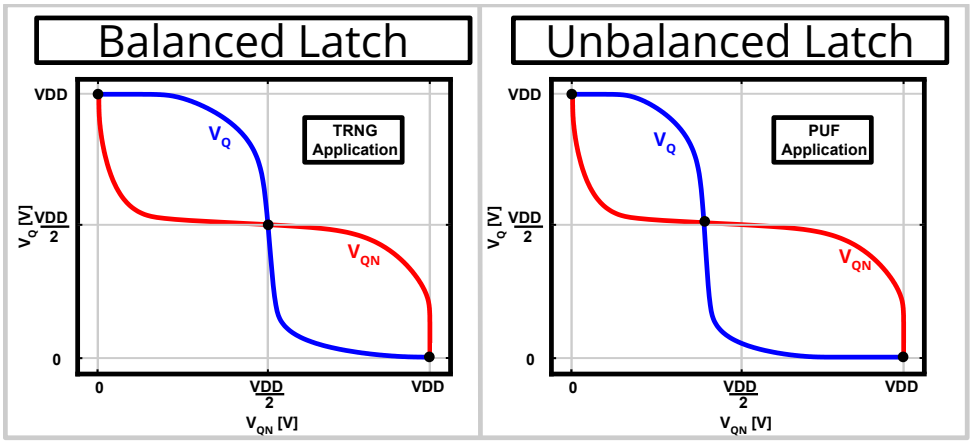


FIGURE 6.4: Metastability response in balanced and unbalanced latches.

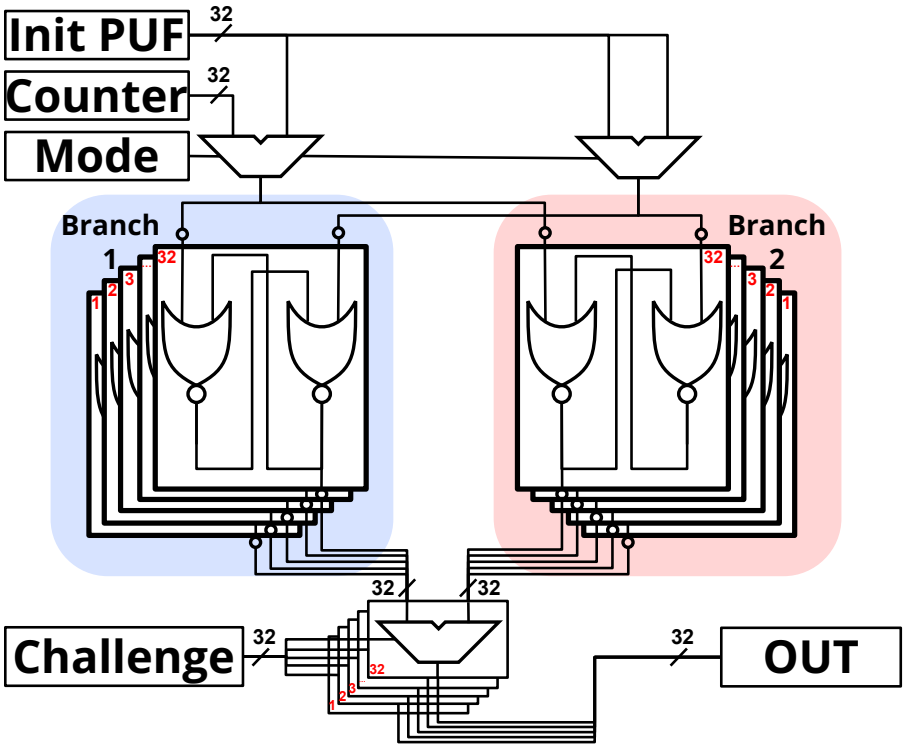


FIGURE 6.5: Schematic of PUF implementation in ChaCha20 core.

unbalanced and balanced latches can be combined with digital implementations, unifying the logic functions with a PUF or a TRNG application.

Figure 6.5 shows the architecture of the implemented PUF. The PUF exploits the metastability in NOR latches. The PUF architecture is designed to be unified with a crypto core, changing the operational mode of the latches between PUF and memory with the signal *mode*. Therefore, the *counter's* value can be generated in the PUF mode or stored in the crypto core mode. The PUF implementation consists of two branches of latches, and the 32-bit challenge selects which branch is used for the PUF output response. Each branch consists of 32 PUF latches. The implemented latches present an unbalanced response in all bits due to the lack of calibration and implementation strategies to improve the mismatch. The length of the challenge and response of the PUF implementation is 32 bits. In addition, the unified PUF generates an overhead of one cycle in the crypto core function.

6.1.2 Results and Comparison

Figure 6.6 illustrates the techniques applied to reduce the noise and improve the process variation to obtain unbalanced latches in the implementation. The latches are constructed with NOR cells using the LUT-6 primitive of the FPGA. In addition, the *INIT* value of the NOR function in the LUT differs in each latch. Therefore, the mismatch of the implemented latch increases in the PUF application. Additionally, a placement blockage is implemented

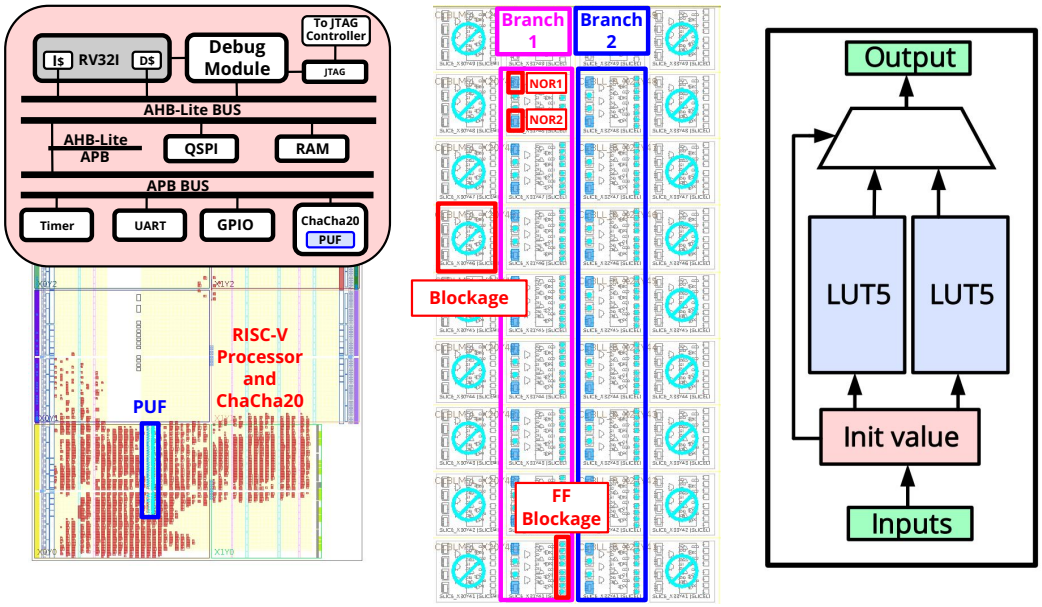


FIGURE 6.6: Placement constraints proposed in the PUF in FPGA implementation.

in the slices of the latch, reducing the noise introduced by external circuits into the latch. On the other hand, the static entropy source implemented in the latch must operate as an elementary bit memory. The entropy source is implemented in one of the internal stages of a ChaCha20 crypto core, as described in Section 6.1. Finally, the PUF-ChaCha20 is connected to a System-on-Chip (SoC) based on a RISC-V for testing the implementation [7].

Table 6.1 shows the resources occupied in the PUF-ChaCha20 implementation in an FPGA. The implementation presents a 3.1% resource overhead caused by the unified functions compared to [111]. On the other hand, the performance is compared with other ChaCha20 implementations, denoting a 0.8% performance reduction in comparison to [111] caused by the cycle necessary to configure the mode of the PUF implementation. The implementation exhibits 0.343 cycles/byte in the crypto core mode.

Fig. 6.7 illustrates the $0.18\mu m$ micrograph that includes the ChaCha20 with

TABLE 6.1: Performance summary and comparison in the FPGA implementation.

	Slices			Performance [Cycles/Byte]
Module	**LUT**	**FF**	**Total**	**Standalone**
This work	2416	1026	3442	0.343
[59] 2019	2369	2152	4521	0.530
[111] 2021	2288	1058	3338	0.340

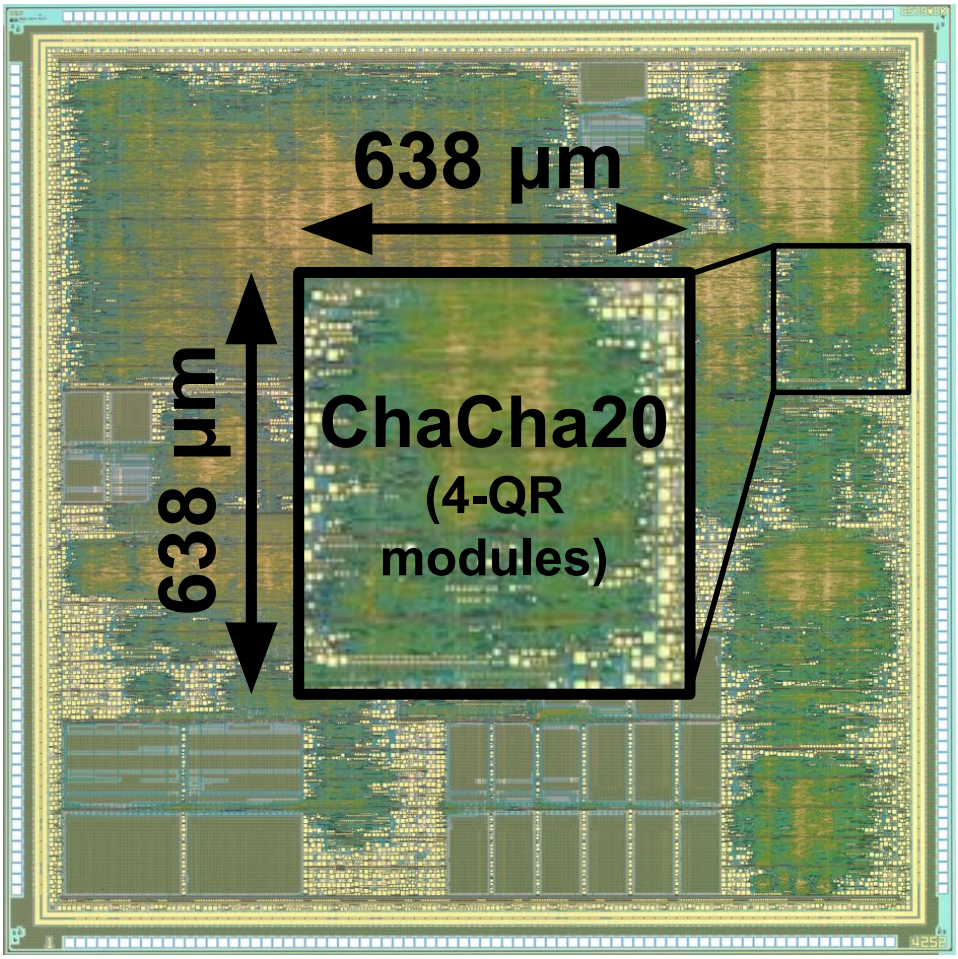


FIGURE 6.7: Micrograph of ChaCha20 core on 0.18μm GP CMOS technology.

a 4-Quarter Round in a secure SoC based on a RISC-V processor [13]. The SoC implemented is used for the management of the ChaCha20 core. The ChaCha20 occupies a 408042-μm^2 area with 31.63-kGE, using 4-QR modules. In addition, the implementation presents a 55.66-mW of power consumption.

Table 6.2 summarizes the comparison of ASIC implementation of the ChaCha core. The ChaCha20 with four QR modules reports 1.16-Gbps at 50-MHz.

TABLE 6.2: Comparison results of ChaCha20 primitive in ASIC implementation

	Area [μm^2]	**Gate count** [kGE]	**Througput** [Gbps]	**Power** [mW]	**Technology** [nm]
This Work	408042	31.63†	1.16@50MHz	55.66@50MHz	180
[41] 2008	217668	22.44	0.31@50MHz	—	180
[58] 2019	106700	11.00	0.32@50MHz	—	180

† In ROHM 0.18μm, gate area NAND2 = 12.9μm^2.

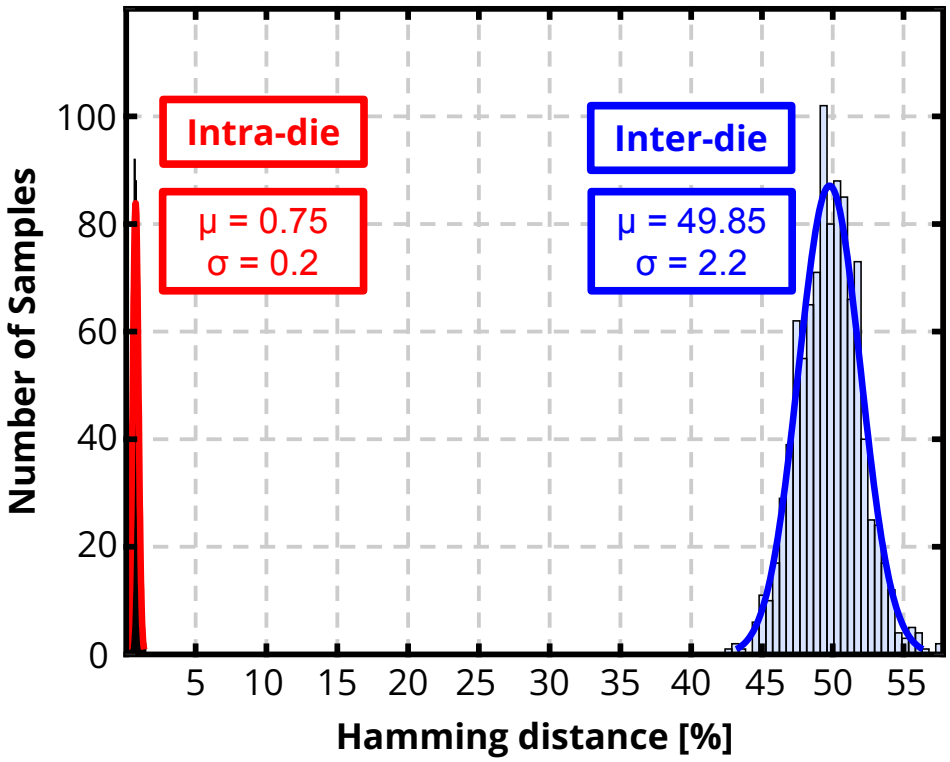


FIGURE 6.8: Hamming distances response of PUF based on unbalanced latches.

The implementation increases a 3.6 × the throughput at the same frequency with 87% [41] and 282% [58] area overhead, respectively.

Figure 6.8 illustrates the intra- and inter-chip Hamming distances in our PUF implementation. The intra- and inter-chip Hamming distances are measured in three different FPGAs using 1000 different challenges. The challenges used in the test are randomly selected and are not repeated. The PUF exhibits an average intra-chip Hamming distance of 0.75% with a 0.2% standard variation. Additionally, the implementation exhibits an average intra-chip Hamming distance of 49.85% with a 2.2% standard variation. Finally, the quality of the unified ChaCha20-PUF is compared with the related works in Table 6.3.

TABLE 6.3: Comparison of the PUF quality in FPGA implementation.

	This work	[67] 2020	[68] 2020
Uniformity [%]†	49.15	49.61	47.20
Uniqueness [%]†	49.85	49.95	39.10
Reliability [%]‡	99.25	99.13	98.89
Topology	Unbalanced Latches	RO with Hybrid Logic	Galois RO

† The ideal value is 50%. ‡ The ideal value is 100%.

6.2 Poly1305 primitive

The Poly1305 algorithm is a one-time authenticator using a 32-byte one-time key and an arbitrary-length message to obtain a 16-byte Message Authentication Code (MAC). Algorithm 2 shows the Poly1305 authenticator algorithm. Initially, the key is partitioned into two parts s and r, respectively. The pair (s,r) should be unique and unpredictable for each call of the Poly1305 algorithm. However, the r and s are possible to generate pseudorandomly. Also, r can have a constant but needs to be modified [112]. Pad1305 takes the message's length and divides the arbitrary-length message into q fragments of 16 bytes. The arbitrary-length message is read in little-endian format, and the r is clamped. Then, the Clamp function clears some bits of r, such that $\bar{r} = r_0 + r_1 + r_2 + r_3$. Where $r_0 \in \{0, 1, 2, .., 2^{28} - 1\}$, $r_1/2^{32} \in \{0, 4, 8, .., 2^{28} - 4\}$, $r_1/2^{64} \in \{0, 4, 8, .., 2^{28} - 4\}$ and $r_1/2^{96} \in \{0, 4, 8, .., 2^{28} - 4\}$. The accumulator h is initialized, adding the polynomial result (6.3). Also, the MAC is truncated in 128 bits.

$$T = \left(\sum_{i=1}^{q} m_i r^{-q-i+1} \ mod \ 2^{130} - 5 \right) + s \ mod \ 2^{128} \tag{6.3}$$

Fig. 6.9 shows the implementation of the Poly1305 core divided into two parts. First, a *PBlock* highlighted in blue generates an initial r and s, using the 256-bit of the *key*. After, the 128-bit of *Block* is operated using a Multi-Multiplier and Accumulator (MulAcc), reproducing the polynomial described in (6.3). The MulAcc implementation consists of a 32-bit unsigned multiplier with a 32-bit accumulator. The Poly1305 primitive has 4 *MulAcc* in the architecture, processing the 128-bit of the block in one cycle. The documentation [112] determines that the length of the message is arbitrary. In this way, the core processes 128 bits in each step. The signals *Block len* indicates the number of bytes in each *Block*. Besides, an FSM highlighted in red controls all *Block* of the message to authenticate and the *MulAcc* interaction. The signal *Init* and *Next* indicate the start of the new message process and

Algorithm 2 Poly1305 authenticator algorithm

Require: $K \in (0,1)^{256}, M \in (0,1)^{L}, L \in \mathbb{N}$
Ensure: $MAC = \text{Poly1305}(K, M, L)$
1: $(r, s) \leftarrow K$
2: $m = (m_1, .., m_q) \leftarrow \text{Pad1305}(L)$
3: $\bar{r} \leftarrow \text{Clamp}(r)$
4: $h \leftarrow 0$
5: **for** $i \leftarrow 0$ **to** $(q - 1)$ **do**
6: $\quad h \leftarrow h + \text{Polynomial}\ (m, \bar{r}, q, i)$
7: **end for**
8: $MAC \leftarrow h + s \bmod 2^{128}$
9: **return** MAC

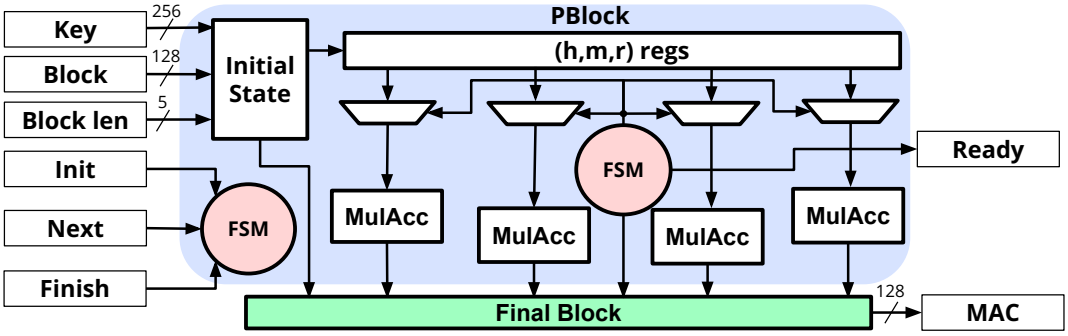


FIGURE 6.9: Architecture of Poly1305 primitive.

another *Block* of the message, respectively. When the last part of the message is introduced, the *Finish* signal is triggered to run the end *MulAcc* operation. Consequently, the data is pushed to the *Final Block*. Second, the *Final Block* highlighted in green takes the data accumulation and *s*, generating the MAC. The MAC is obtained by adding the *s* and the accumulated data with each *MulAcc*. Also, the result of the sum is truncated in 128-bit. Finally, the signal *Ready* indicates the *MAC* is generated.

6.2.1 Results and Comparison

Table 6.4 shows the performance summary and comparison of Poly1305 primitive in FPGA implementation. The Poly1305 with 4-MulAcc modules denotes a 5044-LUT and 1856-FF. In addition, the FPGA implementation shows a 1.22-Cycles/Byte of performance. The implementation reports 140% and 194% of resource overhead compared with the related implementation based on Carry-Save-Adder (CSA) and Double-Add-Reduce (DAR) Poly1305 implementations, respectively. However, the implementation based on MulAcc modules denotes a 1342% and 1345% increase in performance compared to the CSA and DAR implementations.

Figure 6.10 illustrates the ASIC implementation of the Poly1305 primitive in ROHM 0.18μm standard CMOS technology. The Poly1305 primitive was implemented with 1-MulAcc modules, reporting a 307384-μm^2 with 23.82-kGE, respectively. In addition, the implementations show synthesis results

TABLE 6.4: Performance summary and comparison of Poly1305 in FPGA implementation

Module	This Work	[58] 2019	
	Poly1305	Poly1305 CSA	Poly1305 DAR
LUT	5044	1742	1376
FF	1856	1087	963
Total	6900	2829	2339
Cycles/Byte	1.22	16.38	16.42

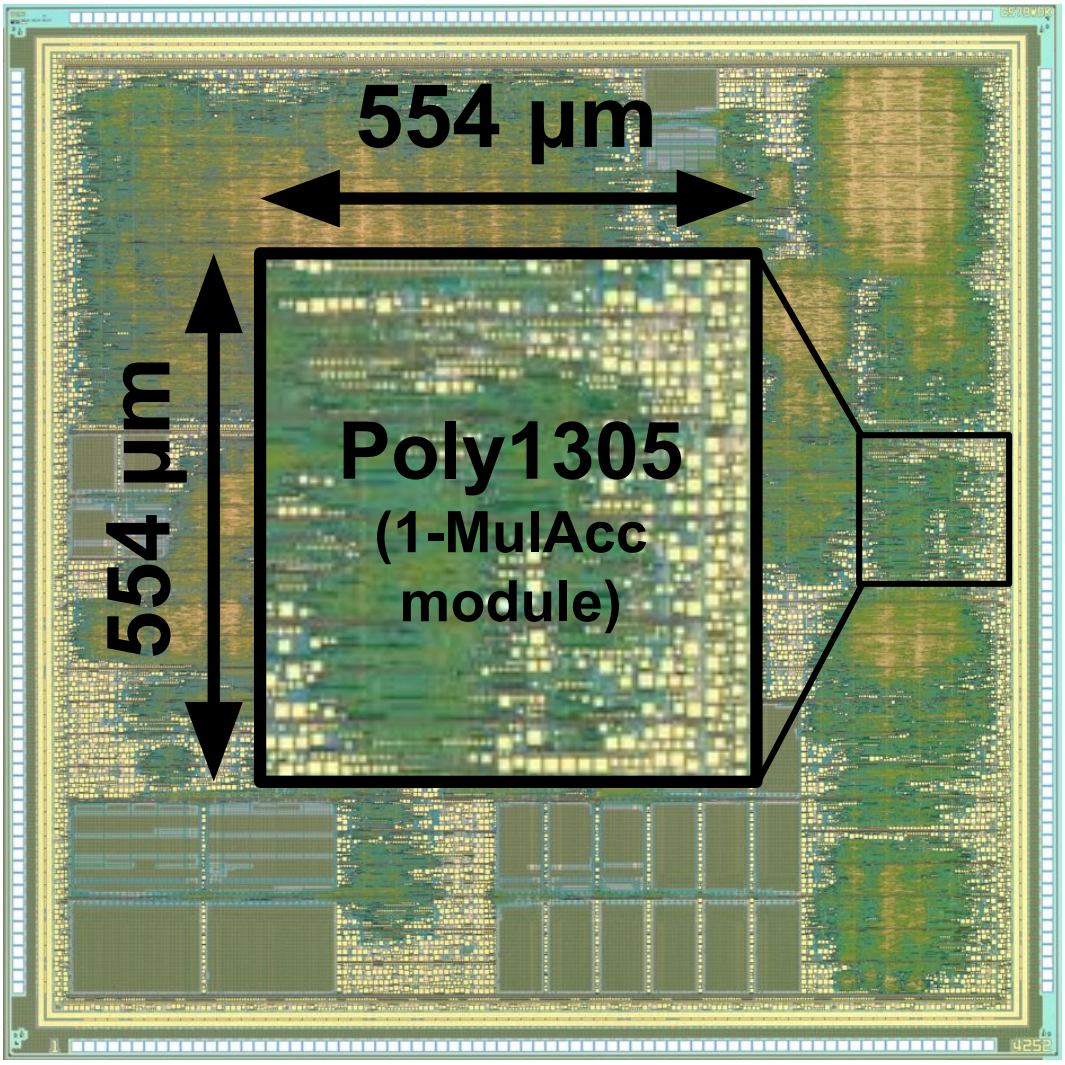


FIGURE 6.10: ASIC implementation of the Poly1305 primitive in ROHM 0.18μm CMOS technology.

of 53.32-mW power consumption at 50-MHz of frequency operation. Table 6.5 shows the summary and comparison of Poly1305 primitive in the ASIC implementation. The comparison with the related work denotes a 51% and 98% area overhead with the low-performance related work based on CSA and DAR architectures. However, the ASIC implementation shows a 13.43 times throughput compared to the related works.

TABLE 6.5: Comparison results of Poly1305 primitive in ASIC implementation

		Area [μm^2]	Gate count [kGE]	Througput [Mbps]	Power [mW]	Technology [nm]
This Work		307384	23.82[†]	40.98 @50MHz	53.32 @50MHz	180
[58] 2019	**CSA**	202633	20.89	3.05 @50MHz	—	180
	DAR	155200	16.00	3.04 @50MHz	—	180

[†] In ROHM 0.18μm, gate area NAND2 = 12.9μm^2.

6.3 Authenticated Encryption with Additional Data (AEAD)

Fig. 6.11 illustrates the implementation of the ChaCha20-Poly1305 AEAD peripheral. The standard determines an AEAD construction using a 256-bit key, 96-bit nonce, an arbitrary length of plaintext, and an arbitrary length of Additional Authenticated Data (AAD) [112]. The AEAD implementation uses a TileLink bus in the RISC-V environment described in [13]. The peripheral consists of the ChaCha20 and Poly1305 primitives highlighted in blue and green. Besides, an FSM highlighted in red controls the internal signals of ChaCha20 and Poly1305 primitives in the peripheral for the correct functionality. Finally, the implementation needs an accumulator and filter to introduce a final block in the Poly1305 primitive and when the plaintext is not a multiple of 512-bits, respectively.

Fig. 6.12 illustrates the functionality of the AEAD peripheral in the RISC-V environment. Steps 1 to 7 describe the conventional operation with the system memory. The first step defines the initial configuration of the AEAD in decryption or encryption, depending on the scenario. In the AEAD construction, a one-time key is used in the Poly1305 primitive with two options for creation in the second step. A pseudo-random number generator creates the one-time key. In another way, the ChaCha20 primitive is used with the same key, nonce of the keystream. However, the counter and the plaintext of the ChaCha20 are initialized by zero. The One Time Key is the first 255-bits of the ciphertext generated for the ChaCha20 primitive. Besides, the AAD and the length in bytes are introduced and processed in the Poly1305 primitive and the accumulator, respectively. One bit in the signal *Control AEAD* indicates the AEAD construction is ready for the next block when the block is processed. Also, the last block of the AAD is indicated with one bit in the signal *Control AEAD* in the peripheral. Steps 3 to 5 describe the AEAD process to generate the ciphertext or plaintext in the encrypt or decryption mode

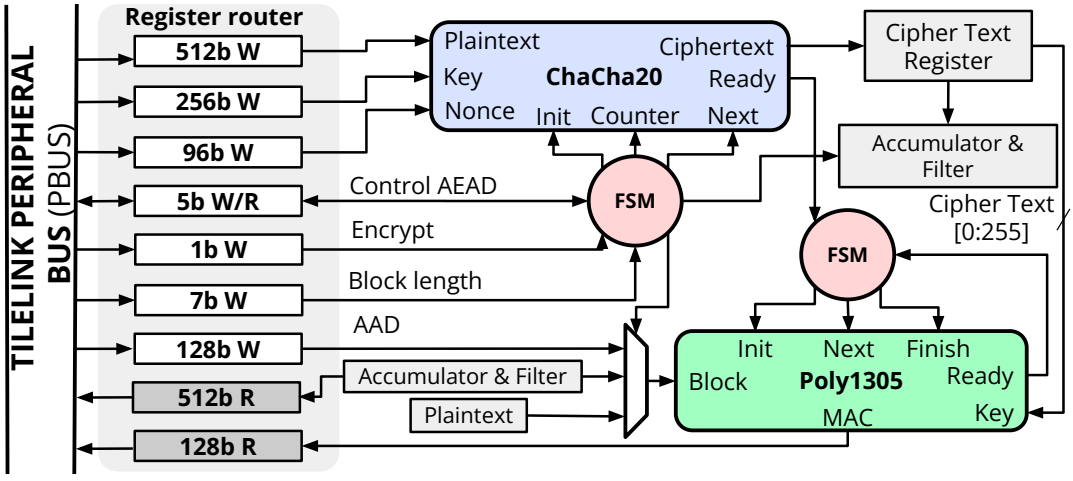


FIGURE 6.11: AEAD construction based on ChaCha20 and Poly1305 primitives.

of the implementation. The mode of the AEAD implementation is defined in the signal *Encrypt* in the peripheral. The counter of the ChaCha20 primitive starts in one in the first block processed, and the plaintext length is necessary per each block introduced. When the final block of the plaintext is not a multiple of 512-bit, the peripheral filter the response of the ChaCha20, eliminating the extra bits generated in the rounds of the primitive. In each result of the ChaCha20 primitive, the results are introduced in the Poly1305 primitive in blocks of 128-bits. Step 6 introduces the final block in the Poly1305 primitive. The final block consists of the result of the accumulator in the peripheral. When the first part is the length of the ADD, and the second part is the length of the plaintext blocks. Finally, the *MAC* is generated in step 7.

Figure 6.13 shows the ChaCha20-Poly1305 AEAD procedure. The procedure starts with the generation of the one-time key (OTK) used in the Poly1305 message authentication code (MAC) process of the AEAD. Conventionally, ChaCha20 generates an OTK using the values of the key and nonce with a counter and plaintext initialized in zero. In addition, the OTK can also be created using a Linear-Feedback Shift Register (LFSR) [109]. Next, the Additional Authenticated Data (AAD) with an arbitrary length is introduced in the Poly1305. Then, the encryption/decryption process is applied to the *plaintext* or *ciphertext*. However, sometimes the *plaintext* or *ciphertext* in step 3 is not a full block with 512 bits, requiring an output filter in the ChaCha20. When the encryption/decryption process is finished, a final block is introduced in Poly1305, containing the lengths of data processed and the introduced AAD. Finally, the MAC is generated.

Figure 6.14 illustrates the typical and proposed PUF-based OTK generation. The ChaCha20 takes the key, nonce, zero-initialized *plaintext*, and zero-initialized

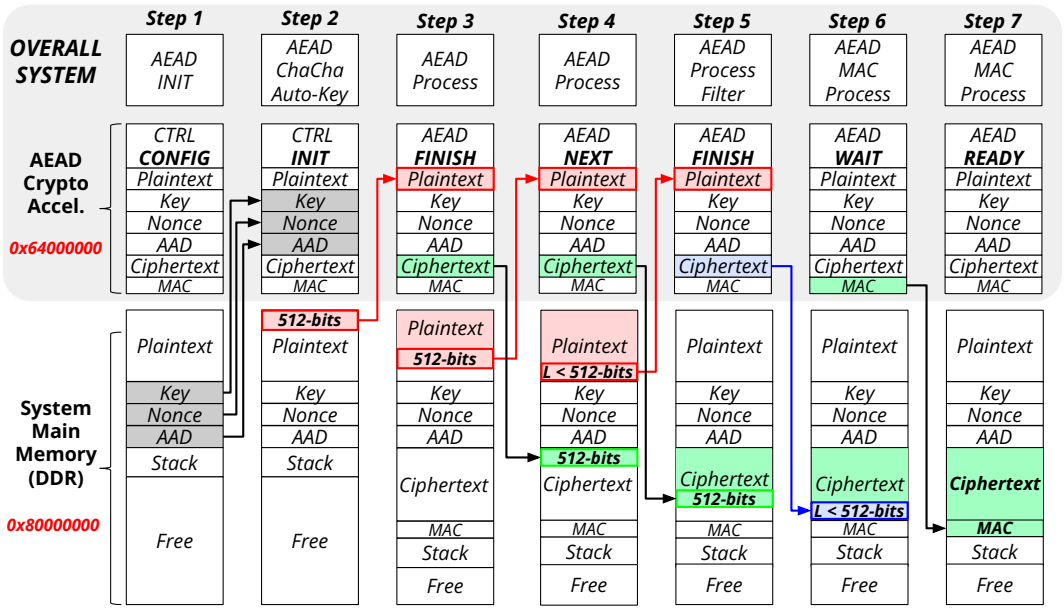


FIGURE 6.12: AEAD interaction with the memory of the system.

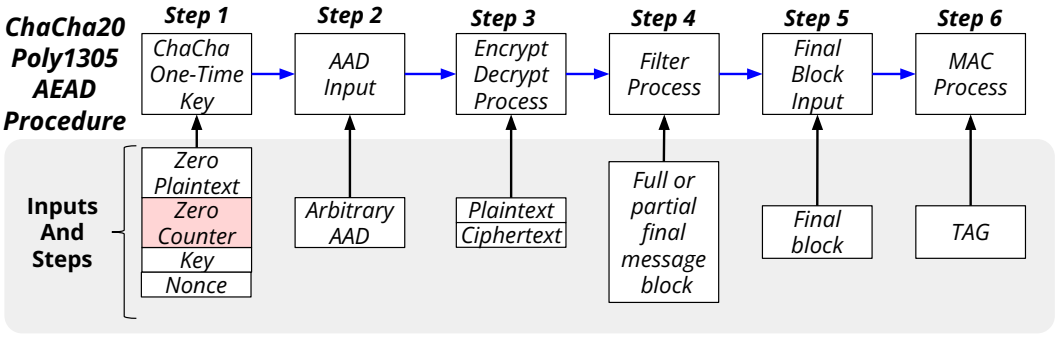


FIGURE 6.13: Procedure of ChaCha20-Poly1305 AEAD compatible with TLS version 1.3.

counter in the typical OTK generation. After 20 rounds of ChaCha20, the first 256 bits of the *CipherText* is the OTK. On the other hand, the proposed OTK generation obtains the counter value using the PUF response based on latches compared to the typical approach. The proposed OTK generation uses a PUF unified with the ChaCha20 primitive. The typical approach exhibits 100% of reliability in the OTK generation. In another way, the reliability of the proposed OTK generation is based on the intra-die Hamming distance of the PUF used to obtain the counter value. However, the unified ChaCha20-PUF exhibits 99.25% of reliability in nominal conditions. Therefore, an Up-Down Counter (UDC) post-processing stage is applied to obtain 100% reliability in the proposed approach.

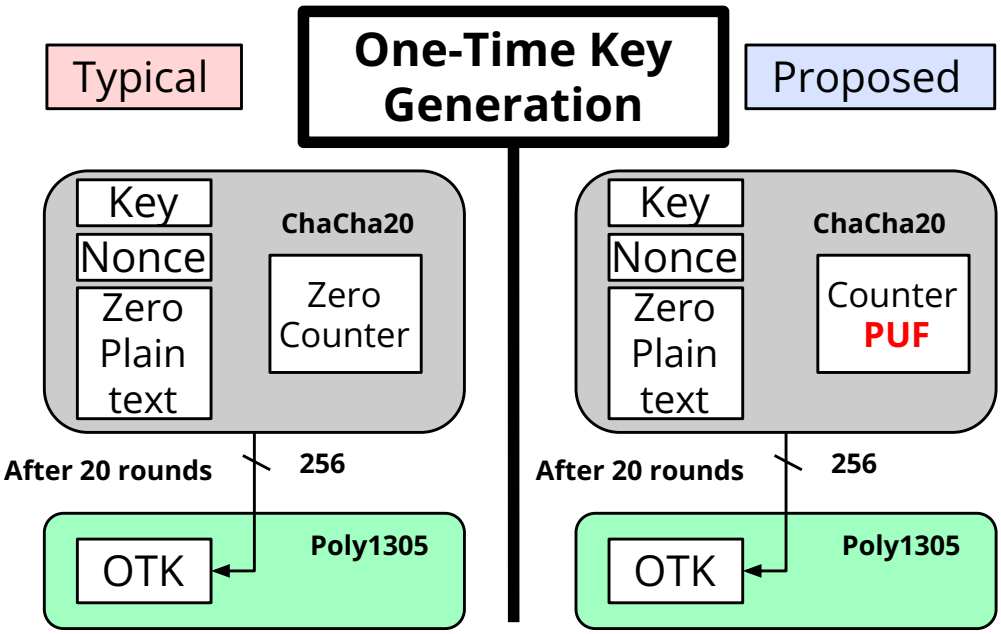


FIGURE 6.14: Typical and proposed OTK generation procedures.

TABLE 6.6: Performance summary and comparison of AEAD implementation in FPGA.

	Slices			Performance [Cycles/Byte]	
Module	**LUT**	**FF**	**Total**	**Standalone**	**System**
This work Typical OTK	10808	3731	14539	1.40	11.56‡
This work PUF OTK	11000	3733	14733	1.41	11.57‡
[58] 2019	3383	4921	8304	21.26†	N/A

† Measured with 8-bytes of AAD. ‡ Measured in the same system.

6.3.1 Results and Comparison

Table 6.6 summarizes the results of the ChaCha20-Poly1305 AEAD with typical and PUF OTK generation implementation in FPGA. The AEAD with a typical OTK generation exhibits a 1.4-cycles/byte with 10808-LUT and 3731-FF, using 4-QR and 4-MulAcc modules in the ChaCha20 and Poly1305 primitives, respectively. In addition, the performance of the AEAD in a system based on a RISC-V environment achieves 11.56-cycles/byte. The performance of the AEAD implementation with the system considers the cycles of the process, the control signals, and the data movement in the TileLink bus in the system implemented. The AEAD in a standalone implementation increases the performance by 15× with an overhead of 75.08% of the resources compared with the related work. The time for the control signals and the movement of the TileLink bus represents 70.32% of overhead compared to the standalone AEAD implementation. The performance is obtained by 5-MB of data, using 16 bytes of AAD. The principal differences of the related work are the *Accumulator and Filter* sub-module, removing the errors in fragmented blocks and the performance of each primitive in the

TABLE 6.7: Comparison of AEAD compatibles with TLS version 1.3

	Module	Device	Resources Utilization		Standalone Imp.			System Imp.
			LUT FF	Slices BRAM	Max. Freq. [MHz]	Througput [Gbps]	Cycles/ Byte	Cycles/ Byte
[42] 2008	AES-CCM	Virtex4 XC4VLX40	1995 0	1200 18	152	1.951*	0.635	—
[48] 2016	AES-CCM	Artix7 XC7A200TL	— —	554 76	177	0.119	11.89	—
[55] 2018	AES-GCM	Virtex7	— —	4194 40	125 ‡	4*	—	—
[60] 2019	AES-GCM	Cyclone V 5CSEMA4U23C6	4572 2407	— 0	36.46	0.275	1.058	19.19
This work	ChaCha20 Poly1305	Virtex7 C7VX485T	10808 3731	3034 0	166	0.948	1.4	11.56

‡ Max. Frequency obtained with synthesis results.
* Parallel AES implementation.

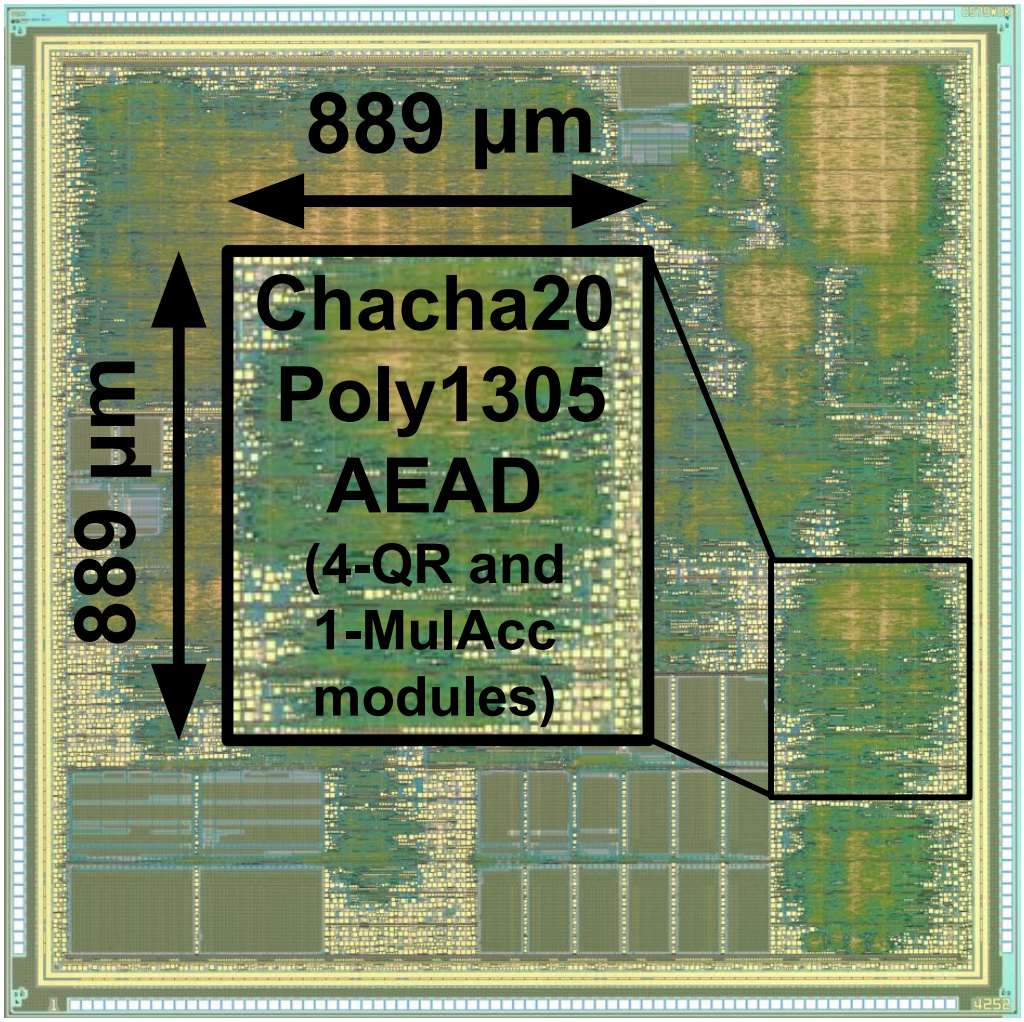


FIGURE 6.15: ChaCha20-Poly1305 AEAD ASIC implementation in ROHM 0.18μm CMOS technology.

AEAD implementation. On the other hand, the ChaCha20-Poly1305 AEAD with OTK generation exhibits a 1.33% and 0.7% resource overhead and performance reduction compared with the typical OTK generation. The UDC post-processing stage represents 0.67% of resource overhead in the AEAD construction with PUF OTK generation in FPGA implementation.

Table 6.7 compares the FPGA implementation with the other cipher suits of TLS 1.3. The AES implementations present a high performance with relatively high resource utilization compared to ChaCha20-Poly1305. However, the high throughput implementations use a parallel AES core [42, 55] . The standalone implementation of AES-GCM [60] reports an improvement of 324% in cycles-per-byte operation, using tower field optimizations in the AES module. However, the ChaCha20-Poly1305 reports an increase of 166% of cycles-per-byte in the system implementation. The ChaCha20-Poly1305 depicts a 0.948-Gpbs in the standalone implementation with 3034-slices, demonstrating competitive performance resources with the AES-based cipher suites.

TABLE 6.8: Summary and comparison in ASIC implementation

	Area [μm^2]	Gate count [kGE]	Througput [Mbps]	Power [mW]	Technology [nm]
This Work	791991	61.39†	285.71@50MHz	119.5@50MHz	180
[58] 2019	485970	50.10	18.80@50MHz	—	180

† In ROHM 0.18μm, gate area NAND2 = 12.9μm^2.

Figure 6.15 shows the ChaCha20-Poly1305 AEAD construction in a 0.18-μm standard CMOS technology. The AEAD construction is implemented using a ChaCha20 primitive with 4-QR round modules and a Poly1305 primitive with 1-MulAcc module, respectively. The implementation occupies a 791991-μm^2 with 61.39-kGE, respectively. The summary and comparison results of the AEAD in ASIC are shown in Table 6.8. The AEAD construction presents an increase of 1519% bit rate with a 62% area overhead. In addition, the implementation reports 119.5-mW of power consumption at 50-MHz.

Figure 6.16 illustrates the run time comparison of the ChaCha20-Poly1305 AEAD in a software implementation, a hardware implementation with a standalone implementation of the primitives, and the AEAD proposed. The software implementation is measured in the same system as the standalone primitives, and the proposed AEAD are based on a RISC-V processor connected with a TileLink bus. The AEAD proposed exhibits a 91.91% and 60.36% run time reduction compared to the software and standalone implementation. The run time results are measured with 50-MHz frequency operation and 16-bytes of Additional Authenticated Data (AAD).

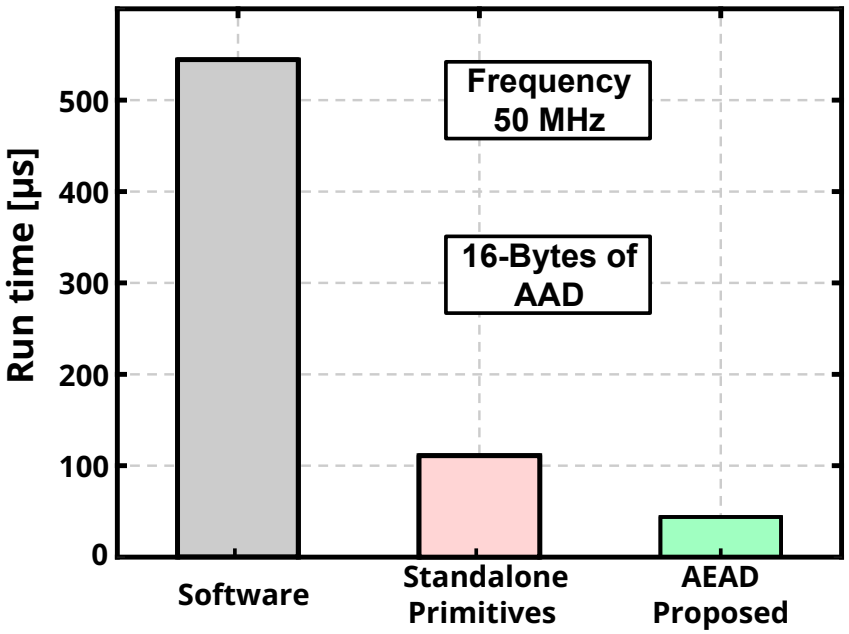


FIGURE 6.16: ChaCha20-Poly1305 AEAD run time comparison.

6.4 Summary

An Authenticated Encryption with Additional Data (AEAD) construction based on ChaCha20-Poly1305 compatible with Transport Layer Security (TLS) version 1.3 is presented with the typical OTK generation and PUF generation, respectively. The ChaCha20 and Poly1305 primitives are described in FPGA and ASIC implementation, reporting 0.343 and 1.22-Cycles per Byte, respectively. The ChaCha20 primitive presents 2416-LUT and 1026-FF of resource utilization in the FPGA implementation, using 4-QR modules. Besides, a unified ChaCha20-PUF is presented, exploiting the metastability in latches, generating a new One-Time Key (OTK) approach in the AEAD construction. The PUF implemented in FPGA denotes 49.15% uniformity, 49.85% uniqueness, and 99.25% reliability in nominal conditions, respectively. On the other hand, the ASIC implementation occupies a 408042-μm^2 area with 31.63-kGE using 4-QR modules, respectively. Besides, the ChaCha20 reports 55.66-mW of power consumption at 50-MHz of frequency operation. The Poly1305 primitive denotes 5044-LUT and 1856-FF resource utilization in FPGA implementation, using 4-MulAcc modules. The ASIC implementation of Poly1305 primitive shows a 307384-μm^2 area occupied with 23.82-kGE using 1-MulAcc modules, respectively. In addition, the Poly1305 reports 53.32-mW of power consumption at 50-MHz of frequency operation. The AEAD construction is presented based on the ChaCha20 and Poly1305 primitives. In addition, an accumulator and filter are implemented to support fragmented blocks in the AEAD implementation. The AEAD implementation depicts 10808-LUT and 3731-FF of resources occupied in FPGA, reaching 1.4-Cycles/Byte of bit rate with the conventional OTK generation approach. On the other hand, the AEAD with the PUF OTK generation reports an 11000-LUT and 3733-FF of resource occupied in FPGA, reaching a 1.1-Cycles/Byte of bit rate, denoting a 1.33% and 0.7% resource overhead and performance reduction compared with the typical OTK generation. Besides, a UDC post-processing stage is implemented to reach a 100% reliability to compete with the typical OTK generation approach. The AEAD ASIC implementation shows a 791991-μm^2 area occupied with 61.39-kGE using 4-QR and 1-MulAcc modules in the ChaCha20 and Poly1305 primitives, respectively. Also, the AEAD reports 119.5-mW of power consumption. The implementation presents a fully compatible ChaCha20-Poly1305 AEAD with TLS version 1.3, reporting a high bit rate compared to the related works.

This page intentionally left blank.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

7.1.1 Achievements

This dissertation proposed multiple secure primitives to make a Root of Trust (RoT) for secure systems. The group of secure primitives is presented for FPGA and ASIC implementation, exploiting different physical phenomena to obtain a random number generation, key generation, secure storage and secure data transmission, respectively. The primitives are implemented and tested in standard CMOS technology to obtain a heterogeneous application in the same chip. The key achievements are summarized as follows:

1. **TRNG based on Frequency Collapse:** This secure primitive generates a random number into a standalone peripheral. The standalone random number generation is used to provide a salt into a Key Derivation Function (KDF) using an existing key of the system and a random sequence, generating a new key for authentication or booting processes.
2. **Unified NVRAM-TRNG-PUF:** The unification of secure primitives allows generating of a key using the static or dynamic entropy in the PUF and TRNG application with a low area overhead, respectively. In addition,the unification with an NVRAM memory allows storing the key generated in non-volatile memory, providing resistance against power-off events.
3. **Unified OTP-PUF:** A robust RoT implementation allows storing a manufacturer key. However, this kind of key provided by the device manufacturer is mandatory to store in a non-volatile memory different from the NVRAM. A One-Time Program (OTP) memory exhibits the characteristic of being programmed only once, preventing alterations by a malicious user or software. Moreover, the unified OTP-PUF primitives allow the option to make an additional key that can be used in a KDF module with a low area overhead.
4. **Authenticated Encryption with Additional Data (AEAD):** A secure system needs to exchange information with different devices. In this way, a ChaCha20-Poly1305 AEAD is constructed to be compatible with

TLS, the most used protocol for providing an end-to-end secure channel in computer networks. The hardware implementation denotes a high performance and security strength compared to the software implementations.

7.1.2 Limitations

In this dissertation, there are several limitations have been recognized as follows:

1. The non-volatile memory implementation in a standard CMOS technology needs a high voltage in the program step in the NVRAM and OTP implementations. In this way, an entirely homogeneous application needs a DC-DC converter inside the chip to obtain the voltage necessary for the program step. In addition, an analog high-voltage multiplexer is also required to manage the program step properly.
2. Typically, the memories are implemented using a special Design Rule Check (DRC) rule to improve the area efficiency. However, due to the Process Design Kit (PDK) restriction, the NVRAM and OTP memories are implemented using basic DRC rules, generating an area overhead.

7.2 Future Work

7.2.1 Develop High-Voltage Analog Circuits

The NVRAM and OTP memories proposed needs high-voltage analog circuits to manage the different steps in these memories inside of the chip without external supplies. Therefore, some analog circuits are necessary to develop in standard CMOS technology to obtain a fully integrated implementation. First, in the program step, a DC-DC converter must provide the high voltage requested in the NVRAM and OTP memory. Second, a high-voltage multiplexer must efficiently manage the bit cell to program.

7.2.2 Improve the area efficiency in the non-volatile arrays

The NVRAM and OTP memory arrays are implemented using conventional DRC rules. However, the memory arrays are implemented using special DRC rules, obtaining a compact implementation and increasing the area efficiency of the memory applications. In addition, a study of the bit cell dimensions is necessary to reduce the high-voltage necessary in the non-volatile primitives, increasing the reliability of standard CMOS technology.

7.2.3 Verify the side-channel attacks in the AEAD primitives

The hardware implementation presents a higher resistance to side-channel attacks than the software implementations. However, the side channel attacks depict a vulnerability in the AEAD implemented in the secure system

to exchange vital information. In this way, the ChaCha20 and Poly1305 primitives need to measure the resistance against side-channel attacks.

This page intentionally left blank.

Bibliography

[1] Intel Corp. Intel Software Guard Extensions (Intel SGX) Developer Guide. [Online]. Available: https://download.01.org/intel-sgx/linux-1.7/docs/Intel_SGX_Developer_Guide.pdf

[2] V. Costan and S. Devadas, "Intel SGX Explained," Cryptology ePrint Archive, Report 2016/086, Jan. 2016, https://eprint.iacr.org/2016/086.

[3] V. Costan, I. Lebedev, and S. Devadas, *Secure Processors Part I: Background, Taxonomy for Secure Enclaves and Intel SGX Architecture*. Now Foundations and Trends, Jul. 2017.

[4] ——, *Secure Processors Part II: Intel SGX Security Analysis and MIT Sanctum Architecture*. Now Foundations and Trends, Jul. 2017.

[5] ARM Ltd., "ARM Security Technology: Building a Secure System using TrustZone Technology," ARM Ltd., Tech. Rep. PRD29-GENC-009492C, Apr. 2009. [Online]. Available: https://documentation-service.arm.com/static/5f212796500e883ab8e74531?token=

[6] R. Buhren, C. Werling, and J.-P. Seifert, "Insecure Until Proven Updated: Analyzing AMD SEV's Remote Attestation," in *ACM SIGSAC Conf. on Computer and Comm. Secu. (CCS)*, London, UK, Nov. 2019, p. 1087–1099.

[7] A. Waterman, Y. Lee, D. A. Patterson, and K. Asanović, "The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Version 2.1," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-118, May 2016. [Online]. Available: http://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-118.html

[8] Hex Five Security, Inc. MultiZone Hex-Five Security. [Online]. Available: https://hex-five.com/

[9] S. Weiser, M. Werner, F. Brasser, M. Malenko, S. Mangard, and A.-R. Sadeghi, "TIMBER-V: Tag-Isolated Memory Bringing Fine-grained Enclaves to RISC-V," in *Network and Distributed Syst. Secu. Symp. (NDSS)*, San Diego, CA, USA, Feb. 2019, pp. 1–15.

[10] D. Lee, D. Kohlbrenner, S. Shinde, K. Asanovic, and D. Song, "Keystone: An Open Framework for Architecting Trusted Execution Environments," in *European Conf. on Computer Syst. (EUROSYS)*, Heraklion, Greece, Apr. 2020, pp. 1–16.

[11] V. Costan, I. Lebedev, and S. Devadas, "Sanctum: Minimal Hardware Extensions for Strong Software Isolation," in *Secu. Symp. (USENIX)*, Aug. 2016, pp. 857—874.

[12] R. Bahmani, F. Brasser, G. Dessouky, P. Jauernig, M. Klimmek, A.-R. Sadeghi, and E. Stapf, "CURE: A Security Architecture with CUstomizable and Resilient Enclaves," in *USENIX Secu. Symp. (USENIX Security)*, Virtual Event, Aug. 2021, pp. 1073–1090.

[13] T.-T. Hoang, C. Duran, R. Serrano, M. Sarmiento, K.-D. Nguyen, A. Tsukamoto, K. Suzaki, and C.-K. Pham, "Trusted Execution Environment Hardware by Isolated Heterogeneous Architecture for Key Scheduling," *IEEE Access*, vol. 10, pp. 46 014–46 027, 2022.

[14] K. Yang, Q. Dong, D. Blaauw, and D. Sylvester, "14.2 A physically unclonable function with BER 10^{-8} for robust chip authentication using oscillator collapse in 40nm CMOS," in *IEEE International Solid-State Circuits Conference - (ISSCC) Digest of Technical Papers*, 2015, pp. 1–3.

[15] K. Yang, D. Blaauw, and D. Sylvester, "An All-Digital Edge Racing True Random Number Generator Robust Against PVT Variations," *IEEE Journal of Solid-State Circuits*, vol. 51, no. 4, pp. 1022–1031, 2016.

[16] Q. Zhou, Y. He, K. Yang, and T. Chi, "12.3 Exploring PUF-Controlled PA Spectral Regrowth for Physical-Layer Identification of IoT Nodes," in *IEEE International Solid- State Circuits Conference (ISSCC)*, vol. 64, 2021, pp. 204–206.

[17] T. Chen, Y. Ma, J. Lin, Y. Cao, N. Lv, and J. Jing, "A Lightweight Full Entropy TRNG With On-Chip Entropy Assurance," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 12, pp. 2431–2444, 2021.

[18] S. Kiamehr, M. S. Golanbari, and M. B. Tahoori, "Leveraging aging effect to improve SRAM-based true random number generators," in *Design, Automation & Test in Europe Conference & Exhibition (DATE), 2017*, 2017, pp. 882–885.

[19] P.-S. Yeh, C.-A. Yang, Y.-H. Chang, Y.-D. Chih, C.-J. Lin, and Y.-C. King, "Self-Convergent Trimming SRAM True Random Number Generation With In-Cell Storage," *IEEE Journal of Solid-State Circuits*, vol. 54, no. 9, pp. 2614–2621, 2019.

[20] S. Taneja, V. K. Rajanna, and M. Alioto, "In-Memory Unified TRNG and Multi-Bit PUF for Ubiquitous Hardware Security," *IEEE Journal of Solid-State Circuits*, vol. 57, no. 1, pp. 153–166, 2022.

[21] K. Ohsaki, N. Asamoto, and S. Takagaki, "A single poly EEPROM cell structure for use in standard CMOS processes," *IEEE Journal of Solid-State Circuits*, vol. 29, no. 3, pp. 311–316, 1994.

[22] R. McPartland and R. Singh, "1.25 volt, low cost, embedded flash memory for low density applications," in *Symposium on VLSI Circuits. Digest of Technical Papers (Cat. No.00CH37103)*, 2000, pp. 158–161.

[23] J. Raszka, M. Advani, V. Tiwari, L. Varisco, N. Hacobian, A. Mittal, M. Han, A. Shirdel, and A. Shubat, "Embedded flash memory for security applications in a 0.13 μm CMOS logic process," in *IEEE International Solid-State Circuits Conference (IEEE Cat. No.04CH37519)*, 2004, pp. 46–512 Vol.1.

[24] C. Li, J. Li, W. Li, S. Li, and J. Wu, "Multitime programmable (MTP) memory cell with pseudo differential architecture," in *IEEE International Conference on Electron Devices and Solid-State Circuits (EDSSC)*, 2015, pp. 582–585.

[25] S.-K. Park, N.-Y. Kim, and K.-I. Choi, "Array Operation Mode Characterization of Select Gate Lateral Coupling Single-Poly Embedded Nonvolatile Memory," *IEEE Transactions on Electron Devices*, vol. 63, no. 4, pp. 1495–1501, 2016.

[26] J. Ardila, J. Santamaria, K. Florez, and E. Roa, "A Stable Physically Unclonable Function Based on a Standard CMOS NVR," in *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2020, pp. 1–4.

[27] J. Kim and K. Lee, "Three-transistor one-time programmable (OTP) ROM cell array using standard CMOS gate oxide antifuse," *IEEE Electron Device Letters*, vol. 24, no. 9, pp. 589–591, 2003.

[28] N. D. Phan, I. J. Chang, and J.-W. Lee, "A 2-Kb One-Time Programmable Memory for UHF Passive RFID Tag IC in a Standard 0.18μ m CMOS Process," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 60, no. 7, pp. 1810–1822, 2013.

[29] Z. Liu, R. Zheng, and J. Sun, "A gate-oxide-breakdown antifuse OTP ROM array based on TSMC 90nm process," in *International Symposium on Next-Generation Electronics (ISNE)*, 2015, pp. 1–3.

[30] J. Cañada, Y. Yoshida, H. Miura, and N. Nakano, "An On-Chip Self-Powered Non-Volatile One-Time-Programmable Memory System in Standard CMOS Technology," in *27th IEEE International Conference on Electronics, Circuits and Systems (ICECS)*, 2020, pp. 1–4.

[31] J. Li, R. Chen, J. Su, X. Huang, and X. Wang, "ME-TLS: Middlebox-Enhanced TLS for Internet-of-Things Devices," *IEEE Internet of Things Journal*, vol. 7, no. 2, pp. 1216–1229, 2020.

[32] S. Pinto and N. Santos, "Demystifying Arm TrustZone: A Comprehensive Survey," *ACM Comput. Surv.*, vol. 51, no. 6, pp. 1–36, Nov. 2019.

[33] ISO/IEC, "Information Technology — Trusted Platform Module Library — Part 1: Architecture," ISO/IEC, Tech. Rep. 11889-1:2015, Aug. 2015. [Online]. Available: https://www.iso.org/standard/66510.html

[34] E. Barker, A. Roginsky, and R. Davis, "Recommendation for Cryptographic Key Generation," National Institute of Standards and Technology (NIST), Gaithersburg, MD, Tech. Rep., 2020. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-133r2

[35] E. Barker, "Recommendation for Key Management: Part 1 – General," National Institute of Standards and Technology (NIST), Gaithersburg, MD, Tech. Rep., 2020. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-57pt1r5

[36] E. Barker and W. C. Barker, "Recommendation for Key Management: Part 2 – Best Practices for Key Management Organizations," National Institute of Standards and Technology (NIST), Gaithersburg, MD, Tech. Rep., 2019. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-57pt2r1

[37] E. Barker and Q. Dang, "Recommendation for Key Management Part 3: Application-Specific Key Management Guidance," National Institute of Standards and Technology (NIST), Gaithersburg, MD, Tech. Rep., 2015. [Online]. Available: http://dx.doi.org/10.6028/NIST.SP.800-57pt3r1

[38] Y. Yuan, L. Wu, X. Zhang, and Y. Yang, "Side-channel collision attack based on multiple-bits," in *11th IEEE International Conference on Anti-counterfeiting, Security, and Identification (ASID)*, 2017, pp. 1–5.

[39] Y. Gao, W. Cheng, H. Zhang, and Y. Zhou, "Cache-Collision Attacks on GPU-Based AES Implementation with Electro-Magnetic Leakages," in *17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/ 12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*, 2018, pp. 300–306.

[40] J. B. Djoko, J. Lange, and A. J. Lee, "NeXUS: Practical and Secure Access Control on Untrusted Storage Platforms using Client-Side SGX," in *49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2019, pp. 401–413.

[41] L. Henzen, F. Carbognani, N. Felber, and W. Fichtner, "VLSI hardware evaluation of the stream ciphers Salsa20 and ChaCha, and the compression function Rumba," in *Int. Conf. on Signals, Circ. and Syst. (SCS)*, Nov. 2008, pp. 1–5.

[42] I. A.-Badillo, C. F.-Uribe, R. Cumplido, and M. M.-Sandoval, "FPGA Implementation and Performance Evaluation of AES-CCM Cores for Wireless Networks," in *Proc. of Int. Conf. on Reconfigurable Computing and FPGAs (Reconfig)*, Dec. 2008, pp. 421–426.

[43] S. K. Mathew, S. Srinivasan, M. A. Anders, H. Kaul, S. K. Hsu, F. Sheikh, A. Agarwal, S. Satpathy, and R. K. Krishnamurthy, "2.4 Gbps, 7 mW All-Digital PVT-Variation Tolerant True Random Number Generator

for 45 nm CMOS High-Performance Microprocessors," *IEEE Journal of Solid-State Circ.*, vol. 47, no. 11, pp. 2807–2821, Oct. 2012.

[44] K. Yang, D. Fick, M. B. Henry, Y. Lee, D. Blaauw, and D. Sylvester, "16.3 A 23Mb/s 23pJ/b fully synthesized true-random-number generator in 28nm and 65nm CMOS," in *IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, 2014, pp. 280–281.

[45] T.-K. Kuan, Y.-H. Chiang, and S.-I. Liu, "A 0.43pJ/bit True Random Number Generator," in *2014 IEEE Asian Solid-State Circuits Conference (A-SSCC)*, 2014, pp. 33–36.

[46] N. Torii, D. Yamamoto, and T. Matsumoto, "Evaluation of Latch-Based Physical Random Number Generator Implementation on 40 Nm ASICs," in *Proceedings of the 6th International Workshop on Trustworthy Embedded Devices*, ser. TrustED '16. New York, NY, USA: Association for Computing Machinery, 2016, p. 23–30. [Online]. Available: https://doi.org/10.1145/2995289.2995292

[47] S. K. Mathew, D. Johnston, S. Satpathy, V. Suresh, P. Newman, M. A. Anders, H. Kaul, A. Agarwal, S. K. Hsu, G. Chen, and R. K. Krishnamurthy, "μ RNG: A 300–950 mV, 323 Gbps/W All-Digital Full-Entropy True Random Number Generator in 14 nm FinFET CMOS," *IEEE Journal of Solid-State Circuits*, vol. 51, no. 7, pp. 1695–1704, 2016.

[48] V.-P. Hoang, T.-T.-D. Phan, V.-L. Dao, and C.-K. Pham, "A compact, ultra-low power AES-CCM IP core for wireless body area networks," in *Proc. of IFIP/IEEE Int. Conf. on Very Large Scale Integration (VLSI-SoC)*, Sep. 2016, pp. 1–4.

[49] S. Chung, W.-K. Fang, Y. Hsu, J. Hsiao, L. Lin, and W.-H. Yu, "Ultra-small and ultra-reliable innovative fuses scalable from 0.35um to 28nm," in *International Conference on Microelectronic Test Structures (ICMTS)*, 2016, pp. 148–151.

[50] J. Cartagena, H. Gomez, and E. Roa, "A Fully-Synthesized TRNG with Lightweight Cellular-Automata Based Post-Processing Stage in 130nm CMOS," in *IEEE Nordic Circuits and Systems Conference (NORCAS)*, 2016, pp. 1–5.

[51] C. Li, Q. Wang, J. Jiang, and N. Guan, "A Metastability-Based True Random Number Generator on FPGA," in *IEEE 12th International Conference on ASIC (ASICON)*, 2017, pp. 738–741.

[52] S. Tao and E. Dubrova, "TVL-TRNG: Sub-Microwatt True Random Number Generator Exploiting Metastability in Ternary Valued Latches," in *IEEE 47th International Symposium on Multiple-Valued Logic (ISMVL)*, 2017, pp. 130–135.

[53] M. A. Sarkisla and S. Ergun, "An Area Efficient True Random Number Generator Based on Modified Ring Oscillators," in *IEEE Asia Pacific Conference on Circuits and Systems (APCCAS)*, 2018, pp. 274–278.

[54] B. Ray and A. Milenković, "True Random Number Generation Using Read Noise of Flash Memory Cells," *IEEE Transactions on Electron Devices*, vol. 65, no. 3, pp. 963–969, 2018.

[55] M. Rodríguez, A. Astarloa, J. Lázaro, U. Bidarte, and J. Jiménez, "System-on-Programmable-Chip AES-GCM implementation for wire-speed cryptography for SAS," in *Proc. of Conf. on Design of Circ. and Integrated Syst. (DCIS)*, Nov. 2018, pp. 1–6.

[56] S. Chung, W.-K. Fang, and F.-H. Chen, "A 4Kx8 Innovative Fuse OTP on 22-nm FD-SOI," *IEEE Journal of the Electron Devices Society*, vol. 7, pp. 837–845, 2019.

[57] S. Park, B. G. Choi, T. W. Kang, K. W. Park, J. J. Lee, S. W. Kang, and J. B. Kim, "Analysis of Entropy Estimator of True Random Number Generation Using Beta Source," in *34th International Technical Conference on Circuits/Systems, Computers and Communications (ITC-CSCC)*, 2019, pp. 1–3.

[58] G. Kanda and K. Ryoo, "High-Throughput Low-Area Hardware Design of Authenticated Encryption with Associated Data Cryptosystem that Uses ChaCha20 and Poly1305," *Int. Journal of Recent Tech. and Engi.*, vol. 8, no. 2S6, pp. 86–94, Jul. 2019.

[59] J. Pfau, M. Reuter, T. Harbaum, K. Hofmann, and J. Becker, "A Hardware Perspective on the ChaCha Ciphers: Scalable Chacha8/12/20 Implementations Ranging from 476 Slices to Bitrates of 175 Gbit/s," in *32nd IEEE Int. System-on-Chip Conf. (SOCC)*, 2019, pp. 294–299.

[60] S. Koteshwara, A. Das, and K. K. Parhi, "Architecture Optimization and Performance Comparison of Nonce-Misuse-Resistant Authenticated Encryption Algorithms," *IEEE Trans. on Very Large Scale Integration (VLSI) Syst.*, vol. 27, no. 5, pp. 1053–1066, Feb. 2019.

[61] Y. Lu, H. Liang, L. Yao, X. Wang, H. Qi, M. Yi, C. Jiang, and Z. Huang, "Jitter-Quantizing-Based TRNG Robust Against PVT Variations," *IEEE Access*, vol. 8, pp. 108 482–108 490, 2020.

[62] H. Gomez, J. Arenas, and E. Roa, "Low-cost TRNG IPs," *IET Circuits, Devices & Systems*, vol. 14, no. 7, pp. 942–946, 2020.

[63] Y. Luo, W. Wang, S. Best, Y. Wang, and X. Xu, "A High-Performance and Secure TRNG Based on Chaotic Cellular Automata Topology," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 67, no. 12, pp. 4970–4983, 2020.

[64] S. T. Chandrasekaran, V. E. G. Karnam, and A. Sanyal, "0.36-mW, 52-Mbps True Random Number Generator Based on a Stochastic Delta–Sigma Modulator," *IEEE Solid-State Circuits Letters*, vol. 3, pp. 190–193, 2020.

[65] S.-K. Park, Y.-S. Lee, D.-H. Kim, N.-Y. Kim, and K.-I. Choiv, "Characterization of Single-Poly SGLC Cell Using 4kbits Array Test Chip for

a Low Density Code Storage Embedded NVM Applications," in *IEEE International Memory Workshop (IMW)*, 2020, pp. 1–4.

[66] J. Yang, Q. Ding, T. Gong, Q. Luo, X. Xue, Z. Gao, H. Yu, J. Yu, X. Xu, P. Yuan, X. Li, L. Tai, S. S. Chung, H. Lv, and M. Liu, "Robust True Random Number Generator using Stochastic Short-Term Recovery of Charge Trapping FinFET for Advanced Hardware Security," in *IEEE Symposium on VLSI Technology*, 2020, pp. 1–2.

[67] D. Deng, S. Hou, Z. Wang, and Y. Guo, "Configurable Ring Oscillator PUF Using Hybrid Logic Gates," *IEEE Access*, vol. 8, pp. 161 427–161 437, 2020.

[68] M. Garcia-Bosque, G. Díez-Señorans, C. Sánchez-Azqueta, and S. Celma, "Proposal and Analysis of a Novel Class of PUFs Based on Galois Ring Oscillators," *IEEE Access*, vol. 8, pp. 157 830–157 839, 2020.

[69] F. Pebay-Peyroula, T. Dalgaty, and E. Vianello, "Entropy source characterization in HfO2 RRAM for TRNG applications," in *15th Design & Technology of Integrated Systems in Nanoscale Era (DTIS)*, 2020, pp. 1–2.

[70] S. Taneja, V. K. Rajanna, and M. Alioto, "36.1 Unified In-Memory Dynamic TRNG and Multi-Bit Static PUF Entropy Generation for Ubiquitous Hardware Security," in *IEEE International Solid- State Circuits Conference (ISSCC)*, vol. 64, 2021, pp. 498–500.

[71] D. Lane, P. D. Hodgson, R. J. Potter, R. Beanland, and M. Hayne, "ULTRARAM: Toward the Development of a III–V Semiconductor, Nonvolatile, Random Access Memory," *IEEE Transactions on Electron Devices*, vol. 68, no. 5, pp. 2271–2274, 2021.

[72] Q. Ding, H. Jiang, J. Li, C. Liu, J. Yu, P. Chen, Y. Zhao, Y. Ding, T. Gong, J. Yang, Q. Luo, Q. Liu, H. Lv, and M. Liu, "Unified 0.75pJ/Bit TRNG and Attack Resilient 2F2/Bit PUF for Robust Hardware Security Solutions with 4-layer Stacking 3D NbOx Threshold Switching Array," in *IEEE International Electron Devices Meeting (IEDM)*, 2021, pp. 39.2.1–39.2.4.

[73] A. A. Zayed, H. H. Issa, K. A. Shehata, and H. F. Ragai, "Ultra-Low Power Oscillator Collapse Physical Unclonable Function Based on FinFET," *IEEE Access*, vol. 9, pp. 27 696–27 707, 2021.

[74] S.-Y. S. Chou, S. Chen, J.-H. Chang, W.-H. Cheng, Y.-D. Chih, P. Fan, C.-E. Huang, C.-M. Hung, G.-H. Li, Y. Wang, S.-D. Wu, and T.-Y. J. Chang, "A 16-kb Antifuse One-Time-Programmable Memory in 5-nm High-K Metal-Gate FinFET CMOS Featuring Bootstrap High-Voltage Scheme, Read Endpoint Detection, and Pseudodifferential Sensing," *IEEE Solid-State Circuits Letters*, vol. 4, pp. 170–173, 2021.

[75] S. Taneja and M. Alioto, "Fully Synthesizable Unified True Random Number Generator and Cryptographic Core," *IEEE Journal of Solid-State Circuits*, vol. 56, no. 10, pp. 3049–3061, 2021.

[76] B. Gao, B. Lin, X. Li, J. Tang, H. Qian, and H. Wu, "A Unified PUF and TRNG Design Based on 40-nm RRAM With High Entropy and Robustness for IoT Security," *IEEE Transactions on Electron Devices*, vol. 69, no. 2, pp. 536–542, 2022.

[77] J.-W. Nam, J.-H. Ahn, and J.-P. Hong, "Compact SRAM-Based PUF Chip Employing Body Voltage Control Technique," *IEEE Access*, vol. 10, pp. 22 311–22 319, 2022.

[78] J. Park, B. Kim, and J.-Y. Sim, "A BER-Suppressed PUF With an Amplification of Process Mismatch Effect in an Oscillator Collapse Topology," *IEEE Journal of Solid-State Circuits*, vol. 57, no. 7, pp. 2208–2219, 2022.

[79] R. Serrano, C. Duran, T.-T. Hoang, M. Sarmiento, K.-D. Nguyen, A. Tsukamoto, K. Suzaki, and C.-K. Pham, "A Fully Digital True Random Number Generator With Entropy Source Based in Frequency Collapse," *IEEE Access*, vol. 9, pp. 105 748–105 755, 2021.

[80] R. Serrano, C. Duran, M. Sarmiento, T.-T. Hoang, A. Tsukamoto, K. Suzaki, and C.-K. Pham, "A Robust and Healthy Against PVT Variations TRNG Based on Frequency Collapse," *IEEE Access*, vol. 10, pp. 41 852–41 862, 2022.

[81] R. Serrano, C. Duran, M. Sarmiento, C.-K. Pham, and T.-T. Hoang, "ChaCha20-Poly1305 Authenticated Encryption with Additional Data for Transport Layer Security 1.3," *Cryptography*, vol. 6, no. 2, 2022. [Online]. Available: https://doi.org/10.3390/cryptography6020030

[82] R. Serrano, C. Duran, M. Sarmiento, and C.-K. Pham, "A Unified NVRAM and TRNG in Standard CMOS Technology," *IEEE Access*, vol. 10, pp. 79 213–79 221, 2022.

[83] R. Serrano, C. Duran, M. Sarmiento, T.-K. Dang, T.-T. Hoang, and C.-K. Pham, "A Unified PUF and Crypto Core Exploiting the Metastability in Latches," *Future Internet*, vol. 14, no. 10, 2022. [Online]. Available: https://doi.org/10.3390/fi14100298

[84] A. Rukhin, J. Soto, J. Nechvatal, M. Smid, E. Barker, S. Leigh, M. Levenson, M. Vangel, D. Banks, A. Heckert, J. Dray, S. Vo, and L. Bassham, "NIST Special Publication 800-22: A Statistical Test Suite for the Validation of Random Number Generators and Pseudo Random Number Generators for Cryptographic Applications," *NIST Special Publication 800-22*, Apr. 2010.

[85] E. Barker, A. Roginsky, and R. Davis, "Recommendation for the Entropy Sources Used for Random Bit Generation," National Institute of Standards and Technology (NIST), Gaithersburg, MD, Tech. Rep., 2018. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-90B

[86] J.-S. Kang, H. Park, and Y. Yeom, "On the Additional Chi-Square Tests for the IID Assumption of NIST SP 800-90B," in *15th Annual Conference on Privacy, Security and Trust (PST)*, 2017, pp. 375–3757.

[87] S. Zhu, H. Chen, W. Xi, M. Chen, L. Fan, and D. Feng, "A Worst-Case Entropy Estimation of Oscillator-Based Entropy Sources: When the Adversaries Have Access to the History Outputs," in *18th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/13th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*, 2019, pp. 152–159.

[88] P. Canet, V. Bouquet, F. Lalande, J. Devin, and B. Leconte, "Fowler-Nordheim erasing time prediction in flash memory," in *Symposium Non-Volatile Memory Technology 2005.*, 2005, pp. 4–18.

[89] L. Zhou and S. Chakrabartty, "Self-Powered Timekeeping and Synchronization Using Fowler–Nordheim Tunneling-Based Floating-Gate Integrators," *IEEE Transactions on Electron Devices*, vol. 64, no. 3, pp. 1254–1260, 2017.

[90] D. Mehta, B. Raman, and S. Chakrabartty, "Differential Fowler-Nordheim Tunneling Dynamical System for Attojoule Sensing and Recording," in *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2019, pp. 1–5.

[91] D. Lane and M. Hayne, "Simulations of Ultralow-Power Nonvolatile Cells for Random-Access Memory," *IEEE Transactions on Electron Devices*, vol. 67, no. 2, pp. 474–480, 2020.

[92] K. Agarwal and S. Nassif, "Statistical analysis of SRAM cell stability," in *43rd ACM/IEEE Design Automation Conference*, 2006, pp. 57–62.

[93] A. Maiti, V. Gunreddy, and P. Schaumont, *A Systematic Method to Evaluate and Compare the Performance of Physical Unclonable Functions*. New York, NY: Springer New York, 2013, pp. 245–267. [Online]. Available: https://doi.org/10.1007/978-1-4614-1362-2_11

[94] C. Gu, N. Hanley, and M. O'neill, "Improved Reliability of FPGA-Based PUF Identification Generator Design," *ACM Trans. Reconfigurable Technol. Syst.*, vol. 10, no. 3, may 2017. [Online]. Available: https://doi.org/10.1145/3053681

[95] J. Miskelly and M. O'Neill, "Fast DRAM PUFs on Commodity Devices," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 11, pp. 3566–3576, 2020.

[96] S. Chung, W.-K. Fang, M. Wendt, and H.-K. Lee, "High temperature reliability studies of innovative fuse programming mode," in *IEEE SOI-3D-Subthreshold Microelectronics Technology Unified Conference (S3S)*, 2019, pp. 1–2.

[97] C.-Y. Wu, M.-C. Shih, C.-Y. Chang, W. Wang, J.-H. Lee, Y.-H. Lee, T.-C. Hsu, and C.-Y. Ko, "Origin of Initial Rapid Cell Current Reduction in Non-Volatile One-Time Programming Memory," *IEEE Electron Device Letters*, vol. 37, no. 2, pp. 176–178, 2016.

[98] B. A. Tsai, S. J. Chang, C. S.Ho, K. C. Chou, I. S. Wei, and P. H. Tseng, "The OTP Data Retention Improvement on CESL and SAB Film Scheme," in *IEEE International Symposium on the Physical and Failure Analysis of Integrated Circuits (IPFA)*, 2018, pp. 1–4.

[99] R. Degraeve, G. Groeseneken, R. Bellens, M. Depas, and H. Maes, "A consistent model for the thickness dependence of intrinsic breakdown in ultra-thin oxides," in *Proceedings of International Electron Devices Meeting*, 1995, pp. 863–866.

[100] E. Y. Wu, J. Sune, and R.-P. Vollertsen, "Comprehensive physics-based breakdown model for reliability assessment of oxides with thickness ranging from 1 nm up to 12 nm," in *IEEE International Reliability Physics Symposium*, 2009, pp. 708–717.

[101] M. Deloge, B. Allard, P. Candelier, J. Damiens, E. Le-Roux, and M. Rafik, "Application of a TDDB Model to the Optimization of the Programming Voltage and Dimensions of Antifuse Bitcells," *IEEE Electron Device Letters*, vol. 32, no. 8, pp. 1041–1043, 2011.

[102] D. J. DiMaria, "Electron energy dependence of metal-oxide-semiconductor degradation," *Applied Physics Letters*, vol. 75, no. 16, pp. 2427–2428, 1999. [Online]. Available: https://doi.org/10.1063/1.125036

[103] J. Sune, "New physics-based analytic approach to the thin-oxide breakdown statistics," *IEEE Electron Device Letters*, vol. 22, no. 6, pp. 296–298, 2001.

[104] J. Cañada, Y. Yoshida, H. Miura, and N. Nakano, "An On-Chip Scalable Low Power Consumption High-Voltage Driver Based on Standard CMOS Technology," in *International SoC Design Conference (ISOCC)*, 2020, pp. 17–18.

[105] P. Swaroop, A. J. Vasani, and M. Ghovanloo, "A High-Voltage Output Driver for Implantable Biomedical Stimulators and I/O Applications," in *49th IEEE International Midwest Symposium on Circuits and Systems*, vol. 1, 2006, pp. 566–569.

[106] S. Pashmineh and D. Killat, "Design of a high-voltage driver based on low-voltage CMOS with an adapted level shifter optimized for a wide range of supply voltage," in *IEEE International Conference on Electronics, Circuits, and Systems (ICECS)*, 2015, pp. 5–8.

[107] ——, "A high-voltage driver based on stacked low-voltage transistors with minimized on-resistance for a buck converter in 65 nm CMOS," in *IEEE Canadian Conference on Electrical and Computer Engineering (CCECE)*, 2016, pp. 1–5.

[108] Daniel J. Bernstein, *The Salsa20 Family of Stream Ciphers*. Berlin, Heidelberg: Springer, 2008, pp. 84–97.

[109] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, Jun. 2018. [Online]. Available: https://rfc-editor.org/rfc/rfc8439.txt

[110] M. Garcia-Bosque, R. Aparicio, G. Díez-Señorans, C. Sánchez-Azqueta, and S. Celma, "An analysis of the behaviour of a PUF based on ring oscillators depending on their locations," in *17th Conf. on Ph.D Research in Microelectronics and Electronics (PRIME)*, 2022, pp. 361–364.

[111] R. Serrano, C. Duran, T.-T. Hoang, M. Sarmiento, A. Tsukamoto, K. Suzaki, and C.-K. Pham, "ChaCha20-Poly1305 Crypto Core Compatible with Transport Layer Security 1.3," in *Proc. of Int. SoC Design Conf. (ISOCC)*, Oct. 2021, pp. 17–18.

[112] Eric Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018.

This page intentionally left blank.

Appendix A

Related Publications

A.1 Journal

[1] Ronaldo Serrano, Ckristian Duran, Trong-Thuc Hoang, Marco Sarmiento, Khai-Duy Nguyen, Akira Tsukamoto, Kuniyasu Suzaki, and Cong-Kha Pham, "A Fully Digital True Random Number Generator with Entropy Source Based in Frequency Collapse," in *IEEE Access*, vol. 9, pp. 105748-105755, Jul. 2021.

[2] Ronaldo Serrano, Ckristian Duran, Marco Sarmiento, Trong-Thuc Hoang, Akira Tsukamoto, Kuniyasu Suzaki, and Cong-Kha Pham, "A Robust and Healthy Against PVT Variations TRNG Based on Frequency Collapse," in *IEEE Access*, vol. 10, pp. 41852-41862, Apr. 2022.

[3] Ronaldo Serrano, Ckristian Duran, Marco Sarmiento, Cong-Kha Pham, and Trong-Thuc Hoang, "ChaCha20–Poly1305 Authenticated Encryption with Additional Data for Transport Layer Security 1.3," in *Cryptography*, vol. 6, n. 2, 30, Jun. 2022.

[4] Ronaldo Serrano, Ckristian Duran, Marco Sarmiento, and Cong-Kha Pham, "A Unified NVRAM and TRNG in Standard CMOS Technology," in *IEEE Access*, vol. 10, pp. 79213-79221, Jul. 2022.

[5] Ronaldo Serrano, Ckristian Duran, Marco Sarmiento, Tuan-Kiet Dang, Trong-Thuc Hoang, and Cong-Kha Pham, "A Unified PUF and Crypto Core Exploiting the Metastability in Latches," in *Future Internet*, vol. 14, n. 10, 298, Oct. 2022.

A.2 Conference

[1] Ronaldo Serrano, Ckristian Duran, Trong-Thuc Hoang, Marco Sarmiento, Akira Tsukamoto, Kuniyasu Suzaki, and Cong-Kha Pham, "ChaCha20-Poly1305 Crypto Core Compatible with Transport Layer Security 1.3," in *Proc. of Int. SoC Design Conf. (ISOCC)*, Jeju Island, South Korea, Oct. 2021, pp. 17-18.

Appendix B

Full Publication List

B.1 Journal

[1] Ronaldo Serrano, Ckristian Duran, Trong-Thuc Hoang, Marco Sarmiento, Khai-Duy Nguyen, Akira Tsukamoto, Kuniyasu Suzaki, and Cong-Kha Pham, "A Fully Digital True Random Number Generator with Entropy Source Based in Frequency Collapse," in *IEEE Access*, vol. 9, pp. 105748-105755, Jul. 2021.

[2] Marco Sarmiento, Khai-Duy Nguyen, Ckristian Duran, Trong-Thuc Hoang, Ronaldo Serrano, Van-Phuc Hoang, Xuan-Tu Tran, Koichiro Ishibashi, and Cong-Kha Pham, "A Sub-μW Reversed-Body-Bias 8-bit Processor on 65-nm Silicon-on-Thin-Box (SOTB) for IoT Applications," in *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 68, no. 9, pp. 3182-3186, Sept. 2021.

[3] Ronaldo Serrano, Ckristian Duran, Marco Sarmiento, Trong-Thuc Hoang, Akira Tsukamoto, Kuniyasu Suzaki, and Cong-Kha Pham, "A Robust and Healthy Against PVT Variations TRNG Based on Frequency Collapse," in *IEEE Access*, vol. 10, pp. 41852-41862, Apr. 2022.

[4] Trong-Thuc Hoang, Ckristian Duran, Ronaldo Serrano, Marco Sarmiento, Khai-Duy Nguyen, Akira Tsukamoto, Kuniyasu Suzaki, and Cong-Kha Pham, "Trusted Execution Environment Hardware by Isolated Heterogeneous Architecture for Key Scheduling," in *IEEE Access*, vol. 10, pp. 46014-46027, Apr. 2022.

[5] Marco Sarmiento, Khai-Duy Nguyen, Ckristian Duran, Ronaldo Serrano, Trong-Thuc Hoang, Koichiro Ishibashi, and Cong-Kha Pham, "Systems on a Chip With 8 and 32 Bits Processors in 0.18-μm Technology for IoT Applications," in *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 69, no. 5, pp. 2438-2442, May 2022.

[6] Ronaldo Serrano, Ckristian Duran, Marco Sarmiento, Cong-Kha Pham, and Trong-Thuc Hoang, "ChaCha20–Poly1305 Authenticated Encryption with Additional Data for Transport Layer Security 1.3," in *Cryptography*, vol. 6, n. 2, 30, Jun. 2022.

[7] Ronaldo Serrano, Ckristian Duran, Marco Sarmiento, and Cong-Kha Pham, "A Unified NVRAM and TRNG in Standard CMOS Technology," in *IEEE Access*, vol. 10, pp. 79213-79221, Jul. 2022.

[8] Ronaldo Serrano, Ckristian Duran, Marco Sarmiento, Tuan-Kiet Dang, Trong-Thuc Hoang, and Cong-Kha Pham, "A Unified PUF and Crypto Core Exploiting the Metastability in Latches," in *Future Internet*, vol. 14, n. 10, 298, Oct. 2022.

B.2 Conference

[1] Trong-Thuc Hoang, Ckristian Duran, Ronaldo Serrano, Marco Sarmiento, Khai-Duy Nguyen, Akira Tsukamoto, Kuniyasu Suzaki, and Cong-Kha Pham, "System-on-Chip Implementation of Trusted Execution Environment with Heterogeneous Architecture," *IEEE Hot Chips Symp. (HCS)*, Palo Alto, CA, USA, Aug. 2021, pp. 1-16.

[2] Ronaldo Serrano, Marco Sarmiento, Ckristian Duran, Khai-Duy Nguyen, Trong-Thuc Hoang, Koichiro Ishibashi, and Cong-Kha Pham, "A Low-Power Low-Area SoC based in RISC-V Processor for IoT Applications," in *Proc. of Int. SoC Design Conf. (ISOCC)*, Jeju Island, South Korea, Oct. 2021, pp. 375-376.

[3] Ronaldo Serrano, Ckristian Duran, Trong-Thuc Hoang, Marco Sarmiento, Akira Tsukamoto, Kuniyasu Suzaki, and Cong-Kha Pham, "ChaCha20-Poly1305 Crypto Core Compatible with Transport Layer Security 1.3," in *Proc. of Int. SoC Design Conf. (ISOCC)*, Jeju Island, South Korea, Oct. 2021, pp. 17-18.

Author Biography

Ronaldo Serrano received the B.Sc. degree in Electronics from the Universidad Industrial de Santander (UIS), Bucaramanga, Colombia in 2020. From 2018 to 2020, he was a junior research assistant at the research group of integrated systems ONCHIP, UIS. Since 2020, he has a research assistant at PHAM laboratory, University of Electro-Communications (UEC). Since 2022, he has also been a applications engineer at Synopsys Inc. His research interests include computer architecture, high-speed digital interfaces, and hardware for security.

THE END.